



Degree Project in Technology

Second cycle, 30 credits

From Experiment to Insight: A Comparative Study of Storage Approaches for Large-Scale Synchrotron and Neutron Scattering Data on AWS

Benchmarking HDF5/HSDS, Zarr, TileDB, and ROOT/TTree for
Random Single-Frame Reads and Full Sequential Scans

CASPER OVE KRISTIANSSON

From Experiment to Insight: A Comparative Study of Storage Approaches for Large-Scale Synchrotron and Neutron Scattering Data on AWS

Benchmarking HDF5/HSDS, Zarr, TileDB, and ROOT/TTree for Random Single-Frame Reads and Full Sequential Scans

CASPER OVE KRISTIANSSON

Master's Programme, Computer Science, 120 credits

Date: May 26, 2026

Supervisors: Sebastian Dalleiger, Ahmet Bahadir Yildiz

Examiner: Panagiotis Papadimitratos

School of Electrical Engineering and Computer Science

Host company: Scatterin AB

Swedish title: Från experiment till insikt: en jämförande studie av lagringsmetoder för storskaliga synkrotron- och neutrondata på AWS

Swedish subtitle: Prestandautvärdering av HDF5/HSDS, Zarr, TileDB och ROOT/TTree för slumpmässiga läsningar av enskilda bilder och sekventiell genomläsning av hela dataset

Abstract

Modern synchrotron and neutron beamlines can stream at gigabytes per second, turning single experiments into multi-terabyte datasets and exposing latency limits of parallel file systems. This thesis benchmarks cloud-native storage layouts on AWS S3 using a single synthetic ~ 20 GiB, $\sim 1,300$ -frame detector stack generated from 80 real CBF seed frames, focusing on slice-level latency, scan throughput, and cost drivers. Four backends (HDF5 via HSDS, Zarr v3, TileDB, ROOT/TTree) and three codecs (gzip, LZ4, zstd) are evaluated under two workloads: random single-frame reads and full sequential scans. Client timings are analyzed on a log scale with dependence-robust confidence intervals, with S3 request counts and stored size providing context.

Results are descriptive and session-conditional for this dataset and AWS setup. For interactive slices, Zarr is fastest with zstd (GM ≈ 155 ms) and with LZ4; TileDB is fastest with gzip. For full scans, TileDB completes the same logical dataset in $2.2\text{--}3.2\times$ lower GM scan time than Zarr across codecs; the transferred-byte throughput summaries follow the same ordering (e.g., zstd ≈ 1.06 GiB/s for 20 GiB). ROOT/TTree is slow for frame-random reads despite a few GETs, indicating single-object paging and decode overheads. HDF5/HSDS trails on slice latency and scan throughput with LZ4, storing 19.17 GiB versus Zarr 10.40 GiB for the same payload.

The contribution is a reproducible AWS-native benchmarking method and a decision matrix for the tested synthetic detector stack and workloads: choose Zarr+zstd when slice latency dominates, and choose TileDB+zstd/LZ4 when throughput dominates.

Keywords

Synchrotron; Neutron scattering; Object storage; AWS S3; EC2; CloudWatch; Cloud-native; HDF5; HSDS; Zarr; TileDB; ROOT; TTree; Parquet; Partial retrieval; Random reads; Full scans; Chunking; Fragments; Metadata; Compression; zstd; lz4; gzip; Latency; Throughput; Request cost; Decision matrix; Scientific data management; Benchmarking;

Sammanfattning

Vid moderna synkrotron- och neutronanläggningar genereras dataflöden på flera gigabyte per sekund. Enskilda experiment kan därmed leda till datamängder på flera terabyte, vilket tydliggör latensbegränsningar i parallella filsystem. I denna avhandling jämförs molnnativa lagringsformat på AWS S3 med en syntetiskt utökad datamängd om cirka 20 GiB och cirka 1 300 bilder, genererad från 80 verkliga CBF-bilder, som testfall. Fokus ligger på latens vid läsning av enskilda bilder, datahastighet vid sekventiell genomläsning av hela datamängden samt kostnadsdrivande faktorer. Fyra lagringslösningar (HDF5 via HSDS, Zarr v3, TileDB och ROOT/TTree) och tre kodekar (gzip, LZ4 och zstd) utvärderas för två åtkomstmönster: slumpmässig läsning av enskilda bilder och sekventiell genomläsning av hela datamängden. Körtider på klientsidan analyseras på logaritmisk skala med beroenderobusta konfidensintervall, och antalet S3-förfrågningar samt lagrad datamängd används som kompletterande mått.

Resultaten är deskriptiva och gäller den studerade datamängden i den aktuella AWS-miljön; de bör därför tolkas som sessionsbundna. För interaktiv läsning av enskilda bilder är Zarr snabbast med zstd (GM ≈ 155 ms) och även med LZ4, medan TileDB är snabbast med gzip. Vid sekventiell genomläsning av hela datamängden läser TileDB igenom samma logiska datamängd på $2,2\text{--}3,2\times$ lägre GM-skanningstid än Zarr för samtliga kodekar; sammanfattningarna av överförd datahastighet följer samma ordning (exempelvis zstd $\approx 1,06$ GiB/s för 20 GiB). ROOT är långsamt vid slumpmässig läsning av bilder trots få GET-förfrågningar, vilket tyder på overhead från paginering och avkodning i en lagringslayout med ett enda objekt. HDF5/HSDS presterar sämre både vad gäller latens vid bildläsning och datahastighet vid helavläsning med LZ4 och lagrar 19,17 GiB jämfört med 10,40 GiB för Zarr för samma nyttolast.

Avhandlingens huvudsakliga bidrag är en reproducerbar metod för prestandamätning på AWS samt en beslutsmatris för den studerade syntetiska datamängden och de studerade användningsfallen: välj Zarr+zstd när latens vid läsning av enskilda bilder är viktigast, och välj TileDB+zstd/LZ4 när datahastighet prioriteras.

Nyckelord

Synkrotron; Neutronspridning; Objektlagring; AWS S3; EC2; CloudWatch; Molnnativa lagringsformat; HDF5; HSDS; Zarr; TileDB; ROOT; TT-

ree; Parquet; Partiell dataåtkomst; Slumpmässiga läsningar; Sekventiella genömläsningar; Chunkning; Fragment; Metadata; Komprimering; zstd; lz4; gzip; Latens; Genomströmning; Förfrågningskostnader; Beslutsmatris; Vetenskaplig datahantering; Prestandajämförelse;

Contents

1	Introduction	1
1.1	Motivation and Context	2
1.1.1	Limitations of Traditional HPC Storage	2
1.1.2	Emergence of Cloud-Native Storage Models	2
1.2	Problem, Purpose, and Goals	3
1.2.1	Problem	3
1.2.2	Original problem and definition	4
1.2.3	Scientific and engineering issues	4
1.2.4	Purpose	4
1.2.5	Goals	5
1.2.6	Research Questions (RQs)	5
1.3	Methodology and Scope	6
1.3.1	Research Methodology	6
1.3.2	Delimitations	6
1.4	Structure of the Thesis	7
2	Background	9
2.1	Scientific Context and Workflows	9
2.1.1	Scientific-Data Context	10
2.1.1.1	Typical beamline architecture	10
2.1.1.2	Detector technologies and raw rates	11
2.1.1.3	Dimensionality of datasets	12
2.1.2	Experimental Workflows and Iterative Analysis	13
2.1.3	Access Patterns in Scattering Analysis	14
2.1.3.1	Small-slice interactive reads	15
2.1.3.2	Full-dataset sequential scans	16
2.1.3.3	Metadata-driven queries	16
2.2	Storage Foundations: HPC vs Object Stores	17
2.2.1	Traditional HPC Storage Paradigms	17

2.2.1.1	Parallel File Systems (Lustre, General Parallel File System (GPFS), Bee Global File System (BeeGFS))	17
2.2.1.2	Portable Operating System Interface (POSIX)-Centric Scientific Formats	19
2.2.2	Cloud Object Storage Fundamentals	20
2.2.2.1	Architectural model	20
2.2.2.2	Request classes and cost drivers	20
2.2.2.3	Latency characteristics	20
2.2.2.4	Performance-optimisation layers	21
2.3	Cloud-Native Models and Compression	22
2.3.1	Cloud-Native Storage Models	22
2.3.1.1	Hierarchical Data Format version 5 (HDF5) + Highly Scalable Data Service (HSDS)	24
2.3.1.2	Zarr	26
2.3.1.3	Apache Parquet	26
2.3.1.4	TileDB	27
2.3.1.5	ROOT data analysis framework (ROOT)/ROOT Tree data structure (TTree)	28
2.3.2	Format design trade-offs for this thesis	29
2.3.2.1	HDF5/HSDS	29
2.3.2.2	Zarr	30
2.3.2.3	Parquet	30
2.3.2.4	TileDB	30
2.3.2.5	ROOT/TTree	31
2.3.3	Compression Algorithms Reviewed	31
2.3.3.1	gzip	32
2.3.3.2	Lempel–Ziv 4 (LZ4)	32
2.3.3.3	Zstandard (ZSTD)	33
2.3.4	Positioning Relative to Previous Work	34
2.4	Summary and Link to Problem Statement	35
3	Methods	37
3.1	Research Design and Process	37
3.1.1	Research Paradigm	37
3.1.2	Research Process	38
3.1.2.1	Step 1: Planning the Experiment	38
3.1.2.2	Step 2: Conducting the Experiment	38
3.1.2.3	Step 3: Analyzing the Data	40

3.1.2.4	Step 4: Discussing the Results	40
3.1.3	RQ map and estimands	40
3.1.3.1	RQ1 — Interactive slice latency	41
3.1.3.2	RQ2 — Full-dataset throughput	41
3.1.3.3	RQ3 — Cost posture and mechanisms	41
3.1.4	Hypotheses	42
3.1.4.1	RQ1 — Interactive slice latency.	42
3.1.4.2	RQ2 — Full-dataset throughput.	42
3.1.4.3	RQ3 — Cost posture and mechanisms.	43
3.1.5	Statistical formulation and evaluation	43
3.1.5.1	Evaluation procedure (<i>estimation</i>)	45
3.1.5.2	Why bootstrap, and why not <i>t</i> -tests or analysis of variance (ANOVA)?	46
3.2	Experimental Testbed and Protocols	47
3.2.1	Test Environment and Infrastructure Setup	47
3.2.2	Data Ingestion and Configuration	48
3.2.3	Experimental Parameters and Workloads	48
3.3	Metrics, Statistics, and Scoring	48
3.3.1	Performance Metrics and Evaluation Criteria	48
3.3.2	Decision-matrix scoring (normalization and weights)	49
3.3.3	Statistical methods	49
3.3.4	Uncertainty in Amazon Web Services (AWS)	51
3.4	Data Collection and Reproducibility	51
3.4.1	Data Sources	52
3.4.2	Sampling Strategy	52
3.4.3	Instrumentation and Metric Logging	52
3.4.4	Validity and Reliability Measures	52
3.4.5	Reproducibility and availability	53
4	What you did	55
4.1	Dataset and preprocessing	55
4.1.1	Raw structure	55
4.1.2	Preprocessing and synthetic augmentation	56
4.2	Infrastructure and software	57
4.3	Storage layouts, codecs, and objectization	57
4.3.1	Common design	57
4.3.2	HDF5 via HSDS	58
4.3.3	Zarr on Simple Storage Service (S3)	58
4.3.4	Apache Parquet on S3 (single-object pilot)	59

4.3.5	TileDB dense array	60
4.3.6	ROOT/TTTree	60
4.3.7	RQ links from the layout choices	61
4.4	Execution, logging, and quality control	61
4.4.1	Execution protocol	61
4.4.2	Quality control, exclusions, and deviations	62
5	Results and Analysis	63
5.1	Inventory and data hygiene	64
5.2	Footprint and service behavior — RQ3 context	64
5.2.1	Compression and objectization outcomes	64
5.2.1.1	S3 object sizes and compression ratios	64
5.2.1.2	Object counts and layout footprint	66
5.2.2	Request intensity and S3 service latencies	67
5.2.2.1	GETs per repetition and per GiB	67
5.2.2.2	CloudWatch FirstByte and TotalRequest latencies	70
5.2.3	Mechanistic links (RQ3)	70
5.3	Core performance results	70
5.3.1	RQ1 — Interactive slice latency	70
5.3.1.1	gzip	71
5.3.1.2	LZ4	72
5.3.1.3	ZSTD	72
5.3.2	RQ2 — Full-dataset scan performance	73
5.3.2.1	gzip	74
5.3.2.2	LZ4	75
5.3.2.3	ZSTD	75
5.3.3	Codec comparison (summary)	76
5.4	Synthesis, sensitivity, and decision support	77
5.4.1	Cross-metric views	78
5.4.1.1	Slice: latency vs GETs per GiB	78
5.4.1.2	Scan: transferred-byte throughput vs GETs per GiB	80
5.4.2	Sensitivity analyses and robustness checks	82
5.4.2.1	Nonparametric confirmation	82
5.4.2.1.1	Slice	82
5.4.2.1.2	Full Scan	84
5.4.2.2	Tail sensitivity	86
5.4.2.2.1	Slice	86

5.4.2.2.2	Full Scan	87
5.4.3	Summary tables for downstream decision matrix	88
5.4.3.1	Per-codec league tables	88
5.4.3.1.1	Slice	88
5.4.3.1.2	Full Scan	89
5.4.3.2	Normalized scores matrix	89
6	Discussion	91
6.1	Answers to the research questions	91
6.1.1	Answer to RQ1 — Interactive slice latency	91
6.1.2	Answer to RQ2 — Full-dataset scan performance	93
6.1.3	RQ3 — Cost posture and mechanisms (summary)	94
6.2	RQ3 interpretation: mechanisms and cost posture	95
6.2.1	Mechanisms explaining outcomes	95
6.2.1.1	Objectization and layout	95
6.2.1.2	Request intensity vs client timing	95
6.2.1.3	Service latencies are not the driver	96
6.2.1.4	Codec effects	96
6.2.2	Cost and storage posture	97
6.3	Threats to validity and limits	97
6.3.1	Distribution shape and robustness	97
6.3.1.1	Shape and dependence	98
6.3.1.2	Center robustness	98
6.3.1.3	Tail robustness	98
6.3.2	Validity and limits	98
6.3.2.1	Internal checks	99
6.3.2.2	External validity and synthetic-data limits	99
6.4	Deviations and exclusions	99
7	Conclusions and Future Work	101
7.1	Closing synthesis	102
7.2	Practitioner choices (decision matrix hook)	102
7.2.1	RQ1 — Interactive slice latency (latency-first)	103
7.2.2	RQ2 — Full-dataset scan performance (scan-time-first)	103
7.2.3	Weight sensitivity	103
7.3	Operational implications	103
7.3.1	Zarr	104
7.3.2	TileDB	104
7.3.3	HDF5/HSDS	104

7.3.4	ROOT/TTree	104
7.3.5	Sustainability, ethics, and societal relevance	105
7.4	Future work	106
References		109
A Supporting materials		121
A.1	Methodological details and statistical settings	121
A.1.1	Cloud testbed and storage configuration	121
A.1.2	Run protocol and quality control	122
A.1.3	Statistical settings	122
A.1.4	Instrumentation and metrics	122
A.1.5	Libraries and access paths	124
A.1.6	Reproducibility and data availability	124
A.1.7	Per-format schema specifics	125
A.1.8	Data synthesis parameters	127
A.1.9	Parquet pilot notes	128
A.2	Supplementary tables and figures	129
A.2.1	Timing diagnostics (Q–Q and drift)	129
A.2.2	Request intensity and S3 service latencies	135
A.2.3	Objectization and storage footprint	144
A.2.4	Slice and full latency details	147
A.2.5	Tail sensitivity	156
A.2.6	League tables for configuration selection	158
A.2.7	Score matrices for configuration selection	159

List of Figures

2.1	End-to-end data journey at a modern synchrotron beamline. Photons are shaped by optics, interact with a sample on a multi-axis goniometer, and are converted to megapixel frames by a high-speed detector. A DAQ burst-buffer server transforms the raw stream into a data cube, which then follows two divergent paths: (i) low-latency ingestion into an on-premises parallel file system (Lustre [10]/GPFS) for traditional HPC workflows, or (ii) upload to an S3-compatible object store where cloud-native formats such as Zarr, HDF5 via HSDS, Parquet, TileDB, and ROOT/TTree enable slice-level access and long-term archival.	11
2.2	Multidimensional detector data cube. A 2-D detector frame (x - y plane) is shown as the front slice of a stack. Successive slices are color-coded to indicate measurements collected at different values of a third experimental coordinate (here labeled λ , but equally applicable to rotation angle θ , time t , or energy E). A single pixel is magnified to reveal its wavelength-dependent spectrum, emphasizing that every voxel within the cube contains a spectral (or angular/temporal) record. Original work by the author.	13
2.3	Median time-to-first-byte read across common storage tiers, plotted on a logarithmic axis. The three-order-of-magnitude gap between local Non-Volatile Memory Express (NVMe) and S3 storage motivates the use of chunking and request strategies.	21
3.1	Research process flowchart with four stages: planning, conducting, analyzing, and discussing the experiment.	40

3.2	AWS benchmarking topology. A single Elastic Compute Cloud (EC2) client issues small-slice requests (~ 15.82 MiB) and full-dataset scans to one Amazon S3 bucket that stores four format prefixes (HDF5/HSDS, Zarr, TileDB, ROOT/TTree). CloudWatch metrics collect request counts, bytes, and service latencies.	47
5.1	Total stored size on S3 by format and codec. Bars sum all objects (data + metadata) under each prefix. The raw baseline is $B_{\text{raw}} \approx 20.02$ GiB ($1296 \times 2880 \times 2880 \times 2$ bytes).	65
5.2	Compression ratios $R_{f,c} = B_{\text{raw}}/B_{f,c}^{\text{stored}}$ for the same format–codec totals. Higher is better. Values are grouped by format and codec; read this together with Figure 5.1, since compression ratios summarize relative compactness but do not show absolute stored GiB. Minor between-format differences at fixed codec are consistent with additional layout metadata.	66
5.3	Object counts by format and codec with annotated layout type. Counts include data and metadata objects. The y-axis is logarithmic only to keep the one-object ROOT/TTree case and the thousand-object Zarr/HSDS cases visible in the same plot; exact counts are listed in Appendix Table A.5.	67
5.4	S3 request intensity by format. Top: GETs per repetition; bottom: GETs per GiB. Each box shows the distribution across codecs and access patterns for a given format.	69
5.5	Codec comparison by format. Top: geometric mean (GM) slice latency (ms), where lower is better; bottom: transferred-byte GM full-scan throughput (GiB/s), where higher is better. Bars are grouped by format with one bar per codec (gzip, LZ4, ZSTD); error bars show dependence-robust 95% CIs.	77
5.6	Slice: GM latency (ms, dependence-robust Confidence Interval (CI)) versus GETs per GiB by format and codec. Points are per session and are used as descriptive mechanism evidence rather than as a fitted causal relationship.	79

5.7	Full-scan transferred-byte GM throughput versus GETs per GiB. Higher throughput and lower GET intensity are preferable, so the upper-left region is best. The y-axis is logarithmic to show both high-throughput and low-throughput formats. Points are per format-codec session; vertical bars show dependence-robust 95% CIs for throughput. Lines connect codecs within the same format.	81
A.1	Q–Q plots of $\ln(\text{time})$ for slice workloads. Rows: codecs {gzip, LZ4, ZSTD}. Columns: formats {HDF5/HSDS, Zarr, TileDB, ROOT/TTTree}. Shaded band is a 95% pointwise envelope; per-panel Shapiro p shown.	130
A.2	Q–Q plots of $\ln(\text{time})$ for full-scan workloads. Rows: codecs {gzip, LZ4, ZSTD}. Columns: formats {HDF5/HSDS, Zarr, TileDB, ROOT/TTTree}. Shaded band is a 95% pointwise envelope; per-panel Shapiro p shown.	131
A.3	Run-order traces of $\ln(\text{time})$ for slice workloads. Rows and columns as in Figure A.1. Solid line: LOWESS smoother; dashed: $\ln$ geometric mean. Per-panel robust y -limits; panel shows lag-1 autocorrelation ρ_1	132
A.4	Run-order traces of $\ln(\text{time})$ for full-scan workloads. Rows and columns as in Figure A.2. Solid line: LOWESS smoother; dashed: $\ln$ geometric mean. Per-panel robust y -limits; panel shows lag-1 autocorrelation ρ_1	133
A.5	Slice: median latency $\pm 95\%$ CI (moving-block bootstrap (MBB)); one row with three panels (gzip, LZ4, ZSTD).	137
A.6	Full scan: median latency $\pm 95\%$ CI (MBB); one row (columns = gzip, LZ4, ZSTD).	138
A.7	S3 request intensity per repetition ($\hat{\lambda}_{\text{rep}}$). Rows: codecs; columns: access patterns. Points are per session; descriptive only.	142
A.8	S3 request intensity per GiB ($\hat{\lambda}_{\text{GiB}}$). Rows: codecs; columns: access patterns. Points are per session; descriptive only.	143
A.9	FirstByteLatency per cell: per-session average (avg) and p_{95} as side-by-side bars. Rows: codecs; columns: patterns. Descriptive only.	145
A.10	TotalRequestLatency per cell: per-session average (avg) and p_{95} as side-by-side bars. Rows: codecs; columns: patterns. Descriptive only.	146

A.11	gzip/slice: geometric mean latency with dependence-robust 95% CIs.	147
A.12	gzip/slice: p_{95} latency with dependence-robust 95% CIs. . . .	148
A.13	gzip/slice: descriptive GM time ratios vs Zarr (session-conditional).	148
A.14	lz4/slice: geometric mean latency with dependence-robust 95% CIs.	149
A.15	lz4/slice: p_{95} latency with dependence-robust 95% CIs. . . .	149
A.16	lz4/slice: descriptive GM time ratios vs Zarr (session-conditional).	150
A.17	zstd/slice: geometric mean latency with dependence-robust 95% CIs.	151
A.18	zstd/slice: p_{95} latency with dependence-robust 95% CIs. . . .	151
A.19	zstd/slice: descriptive GM time ratios vs Zarr (session-conditional).	152
A.20	Slice: descriptive median latency ratios vs Zarr across codecs (gzip, LZ4, ZSTD).	153
A.21	Full scan: descriptive median latency ratios vs Zarr across codecs (gzip, LZ4, ZSTD).	154
A.22	Slice: Harrell–Davis (HD) p_{95} latency with MBB 95% CIs versus block length ℓ (columns: gzip, LZ4, ZSTD; legend shows formats).	156
A.23	Full scan: HD p_{95} latency with MBB 95% CIs versus block length ℓ (columns: gzip, LZ4, ZSTD; legend shows formats).	157
A.24	Slice/gzip: normalized scores (higher is better); column “Total” uses the stated weights.	159
A.25	Slice/lz4: normalized scores.	160
A.26	Slice/zstd: normalized scores.	160
A.27	Full/gzip: normalized scores.	161
A.28	Full/lz4: normalized scores.	161
A.29	Full/zstd: normalized scores.	162
A.30	gzip/full: geometric-mean time (ms) with dependence-robust 95% CIs.	164
A.31	gzip/full: p_{95} time with dependence-robust 95% CIs.	165
A.32	gzip/full: descriptive GM time ratios vs Zarr (session-conditional).	165
A.33	lz4/full: geometric-mean time (ms) with dependence-robust 95% CIs.	166
A.34	lz4/full: p_{95} time with dependence-robust 95% CIs.	166

A.35 <code>lz4/full</code> : descriptive GM time ratios vs Zarr (session-conditional).	167
A.36 <code>zstd/full</code> : geometric-mean time (ms) with dependence-robust 95% CIs.	168
A.37 <code>zstd/full</code> : p_{95} time with dependence-robust 95% CIs.	168
A.38 <code>zstd/full</code> : descriptive GM time ratios vs Zarr (session-conditional).	169

List of Tables

2.1	Four flagship X-ray and neutron area detectors	12
2.2	Order-of-magnitude payloads and dominant performance metrics for the three access patterns analyzed in this section. .	15
2.3	Core S3 request classes	20
2.4	Storage granularity, object-count, and dominant pain-point for each candidate format. Figures assume a 1 TB tomography volume stored with 16 MiB scientific chunks (except Parquet) or as a partitioned Parquet dataset with 256 MiB data files. . .	24
5.1	Overview of RQ1 slice-latency results by codec. Values are rounded and intended as a reader-oriented summary; detailed GM estimates, p_{95} values, and dependence-robust confidence intervals are reported in Tables 5.2–5.4.	71
5.2	Slice latency, <code>gzip</code> : GM and p_{95} (ms) with dependence-robust 95% CIs (MBB; $\ell \in \{4, 6, 8\}$; $B=10,000$).	72
5.3	Slice latency, <code>lz4</code> : GM and p_{95} (ms) with dependence-robust 95% CIs (MBB; $\ell \in \{4, 6, 8\}$; $B=10,000$).	72
5.4	Slice latency, <code>zstd</code> : GM and p_{95} (ms) with dependence-robust 95% CIs (MBB; $\ell \in \{4, 6, 8\}$; $B=10,000$).	73
5.5	Overview of RQ2 full-scan results by codec. Values are rounded and intended as a reader-oriented summary; detailed GM scan times, p_{95} values, and dependence-robust confidence intervals are reported in Tables 5.6–5.8.	74
5.6	Full-scan latency, <code>gzip</code> : GM and p_{95} (ms) with dependence-robust 95% CIs (MBB; $\ell \in \{4, 6, 8\}$; $B=10,000$).	75
5.7	Full-scan latency, <code>lz4</code> : GM and p_{95} (ms) with dependence-robust 95% CIs (MBB; $\ell \in \{4, 6, 8\}$; $B=10,000$).	75
5.8	Full-scan latency, <code>ZSTD</code> : GM and p_{95} (ms) with dependence-robust 95% CIs (MBB; $\ell \in \{4, 6, 8\}$; $B=10,000$).	76

5.9	Slice: request intensity and latency by format and codec (descriptive).	80
5.10	Full scan: request intensity and throughput by format and codec (descriptive).	81
5.11	Slice / gzip: median latency with MBB 95% CIs and ratios vs Zarr.	83
5.12	Slice / gzip: rank agreement (median vs GM).	83
5.13	Slice / lz4: median latency with MBB 95% CIs and ratios vs Zarr.	83
5.14	Slice / lz4: rank agreement (median vs GM).	83
5.15	Slice / zstd: median latency with MBB 95% CIs and ratios vs Zarr.	84
5.16	Slice / zstd: rank agreement (median vs GM).	84
5.17	Full / gzip: median latency with MBB 95% CIs and ratios vs Zarr.	84
5.18	Full / gzip: rank agreement (median vs GM).	85
5.19	Full / lz4: median latency with MBB 95% CIs and ratios vs Zarr.	85
5.20	Full / lz4: rank agreement (median vs GM).	85
5.21	Full / zstd: median latency with MBB 95% CIs and ratios vs Zarr.	85
5.22	Full / zstd: rank agreement (median vs GM).	86
5.23	Slice / gzip: HD p_{95} with widest MBB 95% CI and selected ℓ^* .	86
5.24	Slice / lz4: HD p_{95} with widest MBB 95% CI and selected ℓ^* .	86
5.25	Slice / zstd: HD p_{95} with widest MBB 95% CI and selected ℓ^* .	87
5.26	Full / gzip: HD p_{95} with widest MBB 95% CI and selected ℓ^* .	87
5.27	Full / lz4: HD p_{95} with widest MBB 95% CI and selected ℓ^* .	87
5.28	Full / zstd: HD p_{95} with widest MBB 95% CI and selected ℓ^* .	87
5.29	Slice — gzip: league table.	88
5.30	Slice — zstd: league table.	89
5.31	Full — zstd: league table.	89
A.1	Dataset facts and size definitions for the benchmark detector stack. The reported ~ 20 GiB refers to the uncompressed pixel payload B_{raw} , not the on-disk size of Crystallographic Binary File (CBF) inputs or the stored size of any backend.	127

A.2	Summary diagnostics by cell: Shapiro p on $\ln(\text{time})$ and lag-1 autocorrelation ρ_1 .	129
A.3	S3 request intensity per cell: point estimates and exact Poisson 95% CIs.	135
A.4	S3 CloudWatch service latencies per cell (ms). Values summarize 1-minute CloudWatch datapoints within the timed window of each session. Averages are request-weighted across 1-minute periods; p_{95} is the mean of the 1-minute p_{95} datapoints (descriptive; not a true session-wide per-request p_{95}).	136
A.5	Object counts by store and codec with stored size and compression ratio $R = B_{\text{raw}}/B^{\text{stored}}$ ($B_{\text{raw}} = 20.02$ GiB; metadata included). HSDS sizes tallied from shared db/ by dataset UUID.	144
A.6	Slice — lz4: league table.	158
A.7	Full — gzip: league table.	158
A.8	Full — lz4: league table.	159
A.9	Slice/gzip: score matrix (normalized 0–100).	162
A.10	Slice/lz4: score matrix (normalized 0–100).	163
A.11	Slice/zstd: score matrix (normalized 0–100).	163
A.12	Full/gzip: score matrix (normalized 0–100).	163
A.13	Full/lz4: score matrix (normalized 0–100).	163
A.14	Full/zstd: score matrix (normalized 0–100).	164

List of acronyms and abbreviations

ANOVA	analysis of variance
API	Application Programming Interface
AWS	Amazon Web Services
BeeGFS	Bee Global File System
CBF	Crystallographic Binary File
Ceph	Ceph distributed storage system
CERN	European Organization for Nuclear Research
CFOSAT	China–France Oceanography Satellite
CI	Confidence Interval
CMOS	Complementary Metal–Oxide–Semiconductor
CPU	Central Processing Unit
CSV	Comma-Separated Values
DAOS	Distributed Asynchronous Object Storage
DAQ	Data Acquisition
DNE	Distributed Namespace Environment
DoM	Data on Metadata
DSR	Design Science Research
EBS	Elastic Block Store
EC2	Elastic Compute Cloud
ECHO	ECHO object store
EFS	Elastic File System
EXAFS	Extended X-ray Absorption Fine Structure
FCBench	Floating-Point Compression Benchmark
FSE	Finite State Entropy
GDAL	Geospatial Data Abstraction Library
GM	geometric mean
GPFS	General Parallel File System
HD	Harrell–Davis
HDF5	Hierarchical Data Format version 5

HPC	High-Performance Computing
HSDS	Highly Scalable Data Service
HTTP	Hypertext Transfer Protocol
IRIS	IRIS cloud analysis service
JSON	JavaScript Object Notation
LHCb	Large Hadron Collider beauty
LZ4	Lempel–Ziv 4
LZ77	Lempel–Ziv 1977
MBB	moving-block bootstrap
MDS	Metadata Server
MinIO	MinIO object storage
MPI-IO	Message Passing Interface Input/Output
netCDF	Network Common Data Form
NeXT	Neutron and X-ray Tomography
NIST	National Institute of Standards and Technology
NREL	National Renewable Energy Laboratory
NSD	Network Shared Disk
NSLS-II	National Synchrotron Light Source II
NVMe	Non-Volatile Memory Express
OME	Open Microscopy Environment
OSS	Object Storage Server
OST	Object Storage Target
P95	95 th percentile
PFS	Parallel File System
POSIX	Portable Operating System Interface
RFC	Request for Comments
RNTuple	ROOT Ntuple
ROOT	ROOT data analysis framework
S3	Simple Storage Service
SAN	Storage Area Network

SAXS	Small-angle X-ray Scattering
sCMOS	scientific Complementary Metal–Oxide–Semiconductor
SLS	Swiss Light Source
SSD	Solid-State Drive
SWOT	Surface Water and Ocean Topography
TOMCAT	Tomographic Microscopy and Coherent Radiology Experiments
TTree	ROOT Tree data structure
VLDB	Very Large Data Bases
WAN	Wide Area Network
WAXS	Wide-angle X-ray Scattering
zlib	zlib compression library
ZSTD	Zstandard

Chapter 1

Introduction

Modern synchrotron and neutron experiments generate multi-GiB/s streams of multidimensional detector data. Parallel file systems (Lustre, **General Parallel File System (GPFS)**) deliver high throughput for large, contiguous I/O but often respond poorly to the thousands of small, partial reads used for exploratory inspection and downstream analysis. Cloud object stores such as Amazon **Simple Storage Service (S3)** provide elastic capacity and global access but add per-request latency and fees. Monolithic file-container formats (e.g., **Hierarchical Data Format version 5 (HDF5)**/NeXus) map poorly to object stores without additional translation layers.

Cloud-native layouts—chunked arrays (Zarr), fragmented dense/sparse arrays (TileDB), and tree/column layouts (**ROOT data analysis framework (ROOT)**/**ROOT Tree data structure (TTree)**)—offer alternative trade-offs in latency, throughput, compression, **Central Processing Unit (CPU)**, and request intensity. This thesis provides a side-by-side evaluation of four cloud storage backends on **Amazon Web Services (AWS)** under two canonical scattering access patterns: random single-frame reads (interactive slices) and full sequential scans (bulk post-processing).

In this benchmark, the choice of layout and codec changes interactive single-frame latency by a factor of $\approx 2.6\times$ across the tested configurations (**geometric mean (GM)** ≈ 155 ms for Zarr+**Zstandard (ZSTD)** versus **GM** ≈ 406 ms for **ROOT**+gzip). For bulk processing, TileDB reduces full-scan **GM** time by $\approx 2.2\text{--}3.2\times$ relative to Zarr across codecs (e.g., **GM** ≈ 10.4 s vs. ≈ 30.4 s under **ZSTD**). Cost-relevant proxies also vary materially: request intensity differs by up to $\approx 3\text{--}4\times$ (GETs/GiB) between layouts for the same workload, and stored size can differ by $\approx 1.8\times$ for the same logical dataset under **Lempel–Ziv 4 (LZ4)** (e.g., **Highly Scalable Data Service**

(HSDS) vs. Zarr). These effect sizes illustrate the range observed in this benchmark for how engineering choices in storage layout and compression can change interactive slice latency, end-to-end pipeline time, and object-store cost posture in an S3-based workflow.

1.1 Motivation and Context

As synchrotron and neutron facilities have evolved, detector technologies and beamline instrumentation have driven data generation rates from megabytes to multiple gigabytes per second. Modern experiments frequently combine multiple detectors and scan axes, such as energy, angle, or polarization. This results in multidimensional datasets that can reach terabyte-scale within minutes, making pure ‘download-then-analyze’ workflows increasingly impractical for iterative analysis [1].

1.1.1 Limitations of Traditional HPC Storage

Historically, researchers stored beamline data on parallel file systems such as Lustre or GPFS and accessed it through Message Passing Interface Input/Output (MPI-IO) or batch-oriented workflows. These systems perform well for large-block I/O but incur high overhead on thousands of small random reads typical of exploratory slices or metadata queries [2]. Maintaining on-premises High-Performance Computing (HPC) arrays entails capital and operational expenditures, as well as additional network complexity.

1.1.2 Emergence of Cloud-Native Storage Models

Cloud object storage, most notably Amazon S3, is now widely used for scientific data. Object stores provide virtually unlimited, durable storage, global namespace access, and a pay-as-you-go billing model that shifts capital costs to operational expenses. However, when monolithic scientific formats such as HDF5 or NeXus, originally designed for Portable Operating System Interface (POSIX) filesystems that rely on byte-addressable seek semantics, are naively stored on S3, every small slice can trigger hundreds or thousands of Hypertext Transfer Protocol (HTTP) Range-GET requests (i.e., S3’s emulation of those seeks). For S3, namespace access results in additional LIST/HEAD calls, which are billed per request.

Researchers have introduced a new generation of cloud-native data models to address these shortcomings. Zarr and related chunked layouts expose each

array chunk as an independent object, enabling parallel GET/PUT with a configurable chunk size. Organizations tune chunking to balance object count and transfer efficiency [3]. Columnar storage formats such as Apache Parquet organize data in row groups and column stripes, pruning unneeded fields to reduce I/O usage [4]. TileDB provides dense and sparse array storage with tiled chunking and object-store backends, enabling parallel range reads/writes and reducing metadata and small-read overheads. In parallel, the high-energy physics community has extended ROOT’s TTree abstractions to object-store backends, preserving branch-level selective reads [5].

Each approach offers distinct trade-offs in latency, throughput, compression efficiency, CPU overhead, and cost. Similar object-store design questions appear in adjacent domains such as segmented video delivery, geospatial tiling, cloud bioimaging, and tabular analytics: data are partitioned so that common requests fetch only the necessary chunks, tiles, row groups, or pages. Scattering detector data differ because the same dataset must support both low-latency slice reads during or shortly after acquisition and full sequential scans for reconstruction or reprocessing, while also controlling request cost. Although individual formats have been benchmarked in isolation or within specific domains, to the best of the author’s knowledge, there is no comprehensive, side-by-side evaluation of these backends for scattering-style access patterns on AWS S3 under a controlled, end-to-end benchmark. Addressing this gap informs both beamline scientists seeking responsive, cost-effective data access and data-intensive computing engineers designing next-generation, cloud-native scientific platforms.

1.2 Problem, Purpose, and Goals

This section defines the problem, states the purpose, and lists the goals. It sets the basis for the comparisons used later in the thesis.

1.2.1 Problem

Handling multi-terabyte scattering datasets on AWS S3 reveals several limitations in current data platforms. First, partial reads of beamline slices, such as retrieving a single detector channel or angular data segment, can result in high latency due to the need to perform hundreds, if not thousands, of GET operations and metadata lookups, delaying exploratory analysis and quality checks. Secondly, existing prior prototype evaluations have focused on moderate-scale workloads. This thesis benchmarks each format on a

single 20 GiB detector dataset ($\sim 1,300$ frames) to uncover performance, request behavior, and data ingestion considerations. Thirdly, the interaction between storage formats and compression algorithms (e.g., **ZSTD**, **LZ4**, **gzip**) has not been systematically evaluated, making it unclear which format codec combinations best optimize latency and throughput for **S3**-based workflows. These industrial challenges raise questions for the data-intensive computing community: How can we systematically leverage and compare different storage backends to generate guidance for scientific users? Which combinations of performance and integration trade-offs matter most when designing cloud-native workflows for interactive, partial-retrieval use cases?

1.2.2 Original problem and definition

This thesis defines the original industrial problem as optimizing the retrieval of small, targeted data slices (e.g., a single angle's worth of diffraction frames) from multi-terabyte scattering datasets stored on **AWS S3**. "Partial retrieval" denotes reading the required byte ranges or object keys rather than entire files. This study targets cloud-native, object-store workflows.

1.2.3 Scientific and engineering issues

Generalizing beyond the narrow industrial case leads to two broader research issues. For the two dominant access patterns in scattering pipelines, namely interactive single-frame reads and full sequential scans, this thesis asks how storage layouts and codecs trade latency, throughput, request intensity, and stored size on **S3**. It also asks how different compression algorithms (**ZSTD**, **LZ4**, **gzip**) interact with object-stored data and how those interactions shape end-to-end latency and throughput. These questions are directly relevant to ongoing research in **HPC** and data-intensive computing, where cloud adoption continues to challenge traditional storage paradigms.

1.2.4 Purpose

Scientifically, the study quantifies trade-offs among latency, throughput, compression ratio, and request intensity for a realistic scattering workload on **S3**, within a design-science framework [6]. Outputs include a decision matrix and best-practice guidance.

Secondly, it aims to provide Scatterin AB with a practical, evidence-based framework to optimize its cloud-based data platform for high-throughput synchrotron and neutron-scattering experiments. This work offers concrete

guidance on minimizing latency for partial-read use cases, maximizing throughput, and controlling pay-as-you-go costs across varying access patterns.

1.2.5 Goals

This thesis develops an experimentally validated decision framework to guide the selection and configuration of cloud-native storage solutions for scattering datasets on **AWS S3**, evaluated on a single 20 GiB detector stack. Concretely, the work has three operational goals. First, it builds comparable prototypes by ingesting a single 20 GiB dense 3-D detector stack ($\sim 1,300$ frames) into four timed backends: **HDF5/HSDS**, Zarr, TileDB, and **ROOT/TTree**, while applying consistent chunking and matched codec families (**ZSTD**, **LZ4**, `gzip`) across backends. Because each backend exposes these codecs through its own filter or codec stack, the comparison should be read as an implementation-level benchmark rather than as a fully harmonized comparison of raw codec kernels alone. Second, it benchmarks two read patterns, full scans and random single-frame reads, while recording stored size, requests per GiB, and client timings, and reporting geometric-mean latency for slices, geometric-mean scan time for full scans, transferred-byte throughput summaries, and 95th percentiles. Third, it synthesizes the empirical results into a decision matrix that relates workload patterns, compression choices, and integration concerns to format-specific behavior, and then turns that synthesis into ranked best-practice recommendations.

1.2.6 Research Questions (RQs)

*How do **HDF5** via **HSDS**, Zarr v3, TileDB, and **ROOT/TTree** compare on **AWS S3** in terms of random single-frame read latency and full sequential scan performance, measured by **GM** scan time and transferred-byte throughput, under `gzip`, **LZ4**, and **ZSTD** compression?*

To structure this investigation, the thesis poses three research questions. **RQ1** concerns interactive latency: for random single-frame reads on **S3**, which backend and codec minimize geometric-mean latency and control p_{95} ? **RQ2** concerns full-scan performance: for full-dataset scans, which backend and codec minimize geometric-mean scan time and, correspondingly, deliver the highest transferred-byte throughput with stable p_{95} ? **RQ3** addresses cost posture and mechanisms: how do layout and codec choices change GETs/GiB and stored size, and how do these factors explain client timings?

1.3 Methodology and Scope

This section outlines the benchmarking methodology and the study scope. It specifies datasets, workloads, metrics, and the delimitations for reproducibility.

1.3.1 Research Methodology

We use an *experimental benchmarking* approach within a design–science framework to evaluate four timed storage backends on **AWS** (**HDF5/HSDS**, Zarr, TileDB, **ROOT/TTree**). Apache Parquet is included only as a single-object pilot (ingest only, no timed runs; Appendix **A.1.9**). The benchmark dataset is a single dense detector stack of approximately 20 GiB ($\sim 1,300$ frames). Two workloads are evaluated: random single-frame reads and full scans. We report geometric-mean latency for slices, geometric-mean scan time for full scans, transferred-byte throughput summaries where relevant, and dependence-robust 95% confidence intervals with p_{95} summaries. Detailed descriptions of infrastructure, randomization, logging, and estimator choices are provided in Chapter **3** and the Appendix.

1.3.2 Delimitations

This thesis focuses exclusively on cloud-native, object-store–based workflows within the **AWS** ecosystem. Only **AWS** services (**S3**, **Elastic Compute Cloud (EC2)**) are considered, so the study does not include Azure, Google Cloud, or on-premises **HPC** clusters. Quantitative results cover four timed formats (**HDF5/HSDS**, Zarr, TileDB, **ROOT/TTree**), while Apache Parquet appears only as a single-object, ingest-only pilot and is excluded from timed comparisons (Appendix **A.1.9**). Although real beamline campaigns can reach multi-terabyte scale, this thesis deliberately limits the timed benchmark to one dense 3-D detector stack of roughly 20 GiB so that all format–codec combinations can be evaluated reproducibly under the same **AWS** setup. Other patterns, such as metadata-only queries, random single-element lookups, streaming ingestion, and multi-client contention, are treated qualitatively or left to future work. Likewise, the codec comparison is limited to three widely used alternatives, **ZSTD**, **LZ4**, and `gzip`, rather than extending to Snappy, Brotli, bzip2, or ZFP. Codec labels are aligned across backends, but implementations remain backend-specific; in particular, the Zarr `lz4` condition uses the Python stack’s `Blosc(lz4)` path, so codec effects should be

interpreted at the evaluated implementation level. These delimitations ensure a focused, reproducible comparison of core performance and cost trade-offs for scattering-style object-store access on **AWS**.

1.4 Structure of the Thesis

This thesis is organized into seven main chapters, followed by appendices.

Chapter 1 introduces the motivation, scope, research questions, methodology overview, delimitations, and overall structure. Chapter 2 provides the background by reviewing scientific-data challenges in synchrotron and neutron facilities, traditional **HPC** I/O, and emerging cloud-native formats such as **HDF5/HSDS**, Zarr, Parquet, TileDB, and **ROOT/TTTree**. Chapter 3 explains the research methodology, including dataset selection, **AWS** infrastructure, ingestion pipelines, access patterns, performance metrics, and statistical procedures, while Chapter 4 describes the implementations used for each backend together with chunking, compression, and workflow automation. Chapter 5 presents the quantitative results and their analysis, and Chapter 6 synthesizes those findings into a decision matrix and discussion of trade-offs, integration complexity, and operational implications. Chapter 7 concludes the thesis by summarizing the main contributions, reflecting on limitations, and outlining future work. The appendices provide detailed benchmark logs, code snippets, and supplementary figures.

Chapter 2

Background

Modern synchrotron and neutron facilities stream multi-GiB/s data, producing multi-terabyte experiments that strain large-block assumptions in traditional **HPC** storage. Chapter 1 posed the core question: which cloud-native layout minimizes partial-read latency while maintaining scan throughput on **AWS**? This chapter provides scientific context, storage foundations, and prior art needed for the experimental design and results.

First, this chapter surveys how modern beamlines generate and consume data, highlighting iterative analysis needs and collaboration demands that drive storage-system requirements (Section 2.1.1). It then contrasts legacy parallel file systems such as Lustre and **GPFS** with the architectural principles of **S3** and other object stores (Sections 2.2.1 and 2.2.2). Building on that foundation, Sections 2.3.1–2.3.3 introduce the five storage formats considered in this thesis: **HDF5** + **HSDS**, Zarr, Parquet, TileDB, and **ROOT** / **TTree**. Four formats are benchmarked quantitatively, while Parquet is included only as a single-object pilot for the detector stack (Appendix A.1.9). The timed benchmark matches the four timed formats with three compression algorithms (`gzip`, **LZ4**, **ZSTD**) that modulate their performance. Section 2.1.3 characterizes the access patterns typical of scattering analysis.

2.1 Scientific Context and Workflows

Modern beamlines generate multi-GiB/s data streams and often benefit from selective, timely inspection of subsets of the data.

2.1.1 Scientific-Data Context

Synchrotron light sources and spallation or continuous-beam neutron facilities produce multi-megapixel frames at kilohertz rates, pushing raw data beyond 10 GB s^{-1} [7]. High-speed area detectors, hybrid photon counters, and **Complementary Metal–Oxide–Semiconductor (CMOS)** cameras convert pulses into image sequences that accumulate to multi-TB per day.

Giant accelerator-based light and neutron sources act as kilometer-scale microscopes that let us “see through” centimeters of steel, polymers, or living tissue at kilohertz frame rates. By storing ultra-low-emittance electrons (or moderating proton-driven neutrons), the machines deliver beams that are $\sim 10^{10}–10^{12}$ times brighter than laboratory instruments, unlocking 3-D imaging and diffraction of virtually any material [7]. Each pulse is immediately converted into multi-megapixel images by high-speed area detectors, hybrid photon counters, **CMOS** cameras, or time-pix neutron imagers. Capturing these data allows a single experiment to exceed 10 GB s^{-1} of raw data.

These facilities have a direct impact on the industry. At the European Synchrotron Radiation Facility, X-ray tomography has revealed how expanded-polypropylene crash foams used in car bumpers collapse under load, informing design rules for next-generation impact absorbers [8]. Neutron radiography visualizes sub-second water build-up that strangles polymer-electrolyte fuel cells during winter cold starts, guiding stack architecture and heating strategies for fuel-cell vehicles [9].

2.1.1.1 Typical beamline architecture

A beamline shapes the beam with mirrors, slits, and monochromators, then positions the specimen on a multi-axis stage. A high-speed detector records 2-D or 3-D data at up to thousands of frames per second. A **Data Acquisition (DAQ)** server buffers frames on local **Non-Volatile Memory Express (NVMe) Solid-State Drive (SSD)**s or a burst buffer and writes to storage. Historically, the stream targets a parallel file system (Lustre/**GPFS**), but in recent years object-store workflows have been introduced as an alternative (Figure 2.1).

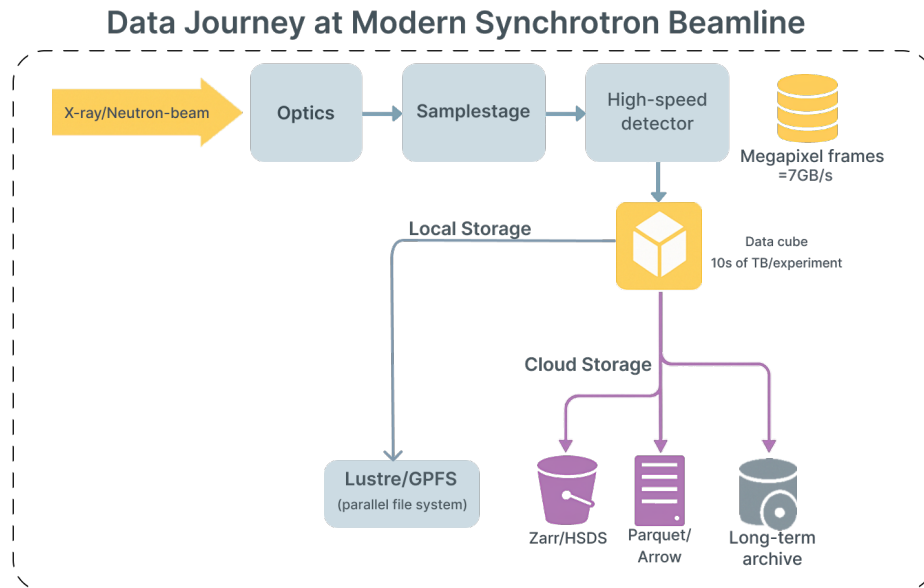


Figure 2.1: End-to-end data journey at a modern synchrotron beamline. Photons are shaped by optics, interact with a sample on a multi-axis goniometer, and are converted to megapixel frames by a high-speed detector. A DAQ burst-buffer server transforms the raw stream into a data cube, which then follows two divergent paths: (i) low-latency ingestion into an on-premises parallel file system (Lustre [10]/GPFS) for traditional HPC workflows, or (ii) upload to an S3-compatible object store where cloud-native formats such as Zarr, HDF5 via HSDS, Parquet, TileDB, and ROOT/TTree enable slice-level access and long-term archival.

2.1.1.2 Detector technologies and raw rates

- **Hybrid photon-counting detectors** (e.g. EIGER2 XE 16M) capture 16-megapixel images at ≥ 400 Hz, producing $\sim 13.5 \text{ GB s}^{-1}$ uncompressed diffraction data $\approx 1 \text{ PB/day}$ if run continuously [11].
- **High-speed CMOS cameras** for tomography, such as the GigaFRoST system at the Swiss Light Source, sustain continuous streaming at 7.7 GB s^{-1} directly to backend servers [1].
- **Neutron scintillator-coupled imagers** notably the National Institute of Standards and Technology (NIST) Neutron and X-ray Tomography (NeXT) detector, enable truly simultaneous neutron + X-ray 3-D tomography and have been applied to Li-ion batteries [12] and operando polymer-electrolyte fuel cells [13].

Table 2.1: Four flagship X-ray and neutron area detectors

Detector	Pixels	Max fps	Peak GB s ⁻¹
EIGER2 XE 16M ^a	4148 × 4362	1 280	23
JUNGFRAU 4M ^b	2048 × 2048	1 000	16.9
GigaFRoST ^c	2016 × 1800	540	7.7
Andor Balor sCMOS ^d	4096 × 4096	54	1.8

^a Serial crystallography and time-resolved Laue diffraction [14].

^b Single-shot imaging at X-ray free-electron lasers [15].

^c Ultrafast tomography on the [Swiss Light Source \(SLS\) Tomographic Microscopy and Coherent Radiology Experiments \(TOMCAT\)](#) beamline [1].

^d Time-resolved [Small-angle X-ray Scattering \(SAXS\)](#)/[Wide-angle X-ray Scattering \(WAXS\)](#) experiments [16].

Modern beamlines already saturate local networks with sustained data streams of $\mathcal{O}(10^2)$ MB s⁻¹ per instrument. For example, the Advanced Photon Source Upgrade anticipates 172 MB s⁻¹ per double sector and 4 GB s⁻¹ facility-wide [17]. Looking ahead, diffraction-limited storage rings are projected to drive detector outputs beyond 10 GB s⁻¹ and data volumes of > 10 TB per experiment, implying corresponding upgrades to storage and analysis pipelines [7].

2.1.1.3 Dimensionality of datasets

Indexing a single 2-D detector frame by additional experimental coordinates turns it into a higher-dimensional data object. Each 2-D image becomes a slice in a stacked data cube, while the depth axis stores a third coordinate, such as a rotation angle, energy, or time (Figure 2.2). Because modern beamlines vary at least three independent axes, the raw (x, y) detector image is naturally represented as a 4-D (or higher) array:

- **Rotation or scan angle:** a full tomographic or serial-crystallography scan may step the sample (or detector) through 10^2 – 10^4 projections to reconstruct 3-D real-space structure or space intensity maps (*rotation* × *energy* × *time*) [18].
- **Energy or wavelength:** in spectro-microscopy and [Extended X-ray Absorption Fine Structure \(EXAFS\)](#), the microscope sweeps energy across an absorption edge. At the same time, a fast detector records images, producing hyperspectral cubes with ~ 10 – 10^2 energy points per angle or time step [19].

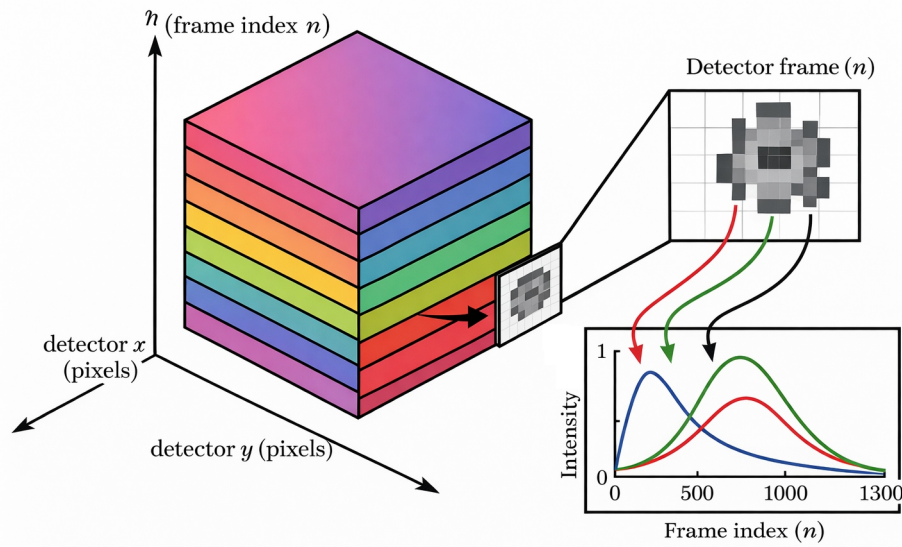


Figure 2.2: **Multidimensional detector data cube.** A 2-D detector frame (x - y plane) is shown as the front slice of a stack. Successive slices are color-coded to indicate measurements collected at different values of a third experimental coordinate (here labeled λ , but equally applicable to rotation angle θ , time t , or energy E). A single pixel is magnified to reveal its wavelength-dependent spectrum, emphasizing that every voxel within the cube contains a spectral (or angular/temporal) record. Original work by the author.

When these axes are combined, experiments can generate 5-D or 6-D datasets. For example, a 2048×2048 detector recording 1800 rotation angles, 50 energy points, and 20-time delays already contains $\sim 150 \times 10^9$ pixels (≈ 300 GB at 16-bit depth). These arrays enable multivariate analyses but impose heavy storage and I/O demands. Some beamlines further append polarization, external field strength, or environmental parameters (temperature, pressure, electrochemical potential), further increasing data dimensionality and storage complexity.

2.1.2 Experimental Workflows and Iterative Analysis

Experimental workflows at synchrotron and neutron facilities often involve iterative analysis in which raw detector frames are reconstructed, inspected, and, in some cases, used to inform later acquisition choices. Bicer et al.

demonstrated a real-time tomography workflow in which projections were streamed from the beamline to an **HPC** cluster, producing slice reconstructions within seconds and enabling autonomous steering of acquisition parameters such as exposure, angular range, and focus [20].

At Shanghai Synchrotron Radiation Facility, automated pipelines tied into the metadata catalog provide progress notifications and interactive dashboards for timely updates [21].

These workflow requirements translate directly into storage-system requirements:

- **Latency:** scientists often request slices of data (quick-look radiographs or azimuthal-integrated diffraction rings). Low retrieval latency supports practical interactive use, although exact timing requirements depend on the beamline and workflow.
- **Throughput:** full 3-D reconstruction or machine-learning inference may pull hundreds of contiguous gigabytes at the end of a scan, often requiring burst bandwidths of $\geq 10 \text{ GB s}^{-1}$.
- **Scalability & concurrency:** remote teams viewing the same dataset and automated QC pipelines often need simultaneous small-read streams. Monolithic single-file layouts trigger many Range-GETs per slice and cannot exploit per-chunk concurrency.
- **Online analysis:** some workflows use streaming reconstruction or AI-driven feature extraction as data arrive. In those cases, storage should support continuous, low-overhead partial reads and write-through metadata indexing without forcing large end-of-scan delays.

These workflow pressures motivate the performance metrics and storage formats evaluated in this thesis.

2.1.3 Access Patterns in Scattering Analysis

Workflows range from interactive slice access to terabyte-scale re-processing. For the purposes of this thesis, the relevant storage workloads are grouped into three access patterns: interactive slices, full sequential scans, and metadata-only queries.

Table 2.2 summarises the three access patterns discussed in this section. It contrasts the typical payload size, level of concurrent requests, and the single performance metric that most strongly constrains each workload. The

figures serve as order-of-magnitude guides: exact numbers vary by beamline, detector, and network topology.

A useful analogy is segmented video delivery: a video file can be divided into independently retrievable temporal pieces so that a player fetches only the next segment. Beamline detector data share the need for partial access, but differ because the access unit is often a multidimensional scientific slice, detector frame, angular wedge, or metadata subset rather than a fixed playback segment. The same dataset may also be read end-to-end for reconstruction or reprocessing, so a layout optimized only for sequential video-style playback is insufficient.

Table 2.2: Order-of-magnitude payloads and dominant performance metrics for the three access patterns analyzed in this section.

Pattern	Typical payload	Dominant metric
Interactive slice	1–50 MB	Latency (TTFB $\approx$ 100 ms to 200 ms per GET on S3)
Full dataset scan	0.1–10 TB	Sustained throughput (GB s^{-1})
Metadata-only query	10–100 kB	Metadata operation rate / metadata hot-spot

2.1.3.1 Small-slice interactive reads

At many beamlines, an *interactive slice* is a reconstructed tomographic plane or detector region of interest. For example, the GigaFRoST camera at [SLS–TOMCAT](#) streams 2016×1800 px, 16-bit projections at 7.7 GB s^{-1} , so a single projection (or a stack of a few tiled ROIs) naturally falls in this size window [1]. Likewise, the APS nanoprobe workflow achieves online neural-network inference on 512×512 diffraction frames at 100 Hz (one frame every 10 ms), illustrating that some latency-sensitive workflows operate on millisecond-to-sub-second timescales [22].

Partial reads are equally critical *after* acquisition. For example, strain- and stress-mapping routines may integrate only Bragg peaks, i.e., localized intensity maxima in diffraction images, within narrow azimuthal sectors ($\Delta\chi \approx 5^\circ$). They may also select only a few detector tiles and discard the rest of the frame. In high-energy diffraction microscopy at APS, grain-resolved strain reconstruction can require repeatedly loading only the angular wedges that intersect a given grain’s Debye–Scherrer rings. These ring-shaped

diffraction features are produced by many crystal orientations, and restricting reads to them can cut I/O by an order of magnitude [23].

2.1.3.2 Full-dataset sequential scans

A *full-dataset scan* means streaming every projection or event record, typically 10^2 – 10^3 GB per tomographic run or several thousand gigabytes per serial-crystallography campaign.

Complete reads underpin high-fidelity analyses that cannot tolerate missing frames. Iterative phase-retrieval and filtered back-projection algorithms in tomography require the entire sinogram for each refinement cycle. In serial femtosecond crystallography, clustering, indexing, and Monte Carlo integration need millions of patterns to converge on accurate structure factors [24]. Full-dataset scans also support *reference reprocessing* and video-style visualization, where sequential reads saturate available bandwidth and compression efficiency, and parallel read pipelines dominate time-to-solution.

2.1.3.3 Metadata-driven queries

A *metadata query* retrieves only descriptive information, sample-environment logs, scan parameters, detector geometry, or a limited set of columns/branches without accessing the large image stacks. In the access model used here, metadata-only queries are treated as small-payload operations, usually far smaller than detector-frame reads, and are relevant mainly because many such operations can create metadata hot spots.

Synchrotron and neutron facilities increasingly push raw detector frames straight into object stores (AWS S3, Ceph distributed storage system (Ceph), MinIO object storage (MinIO)) to curb on-premise costs and enable remote collaboration. Brookhaven’s National Synchrotron Light Source II (NSLS-II), for example, operates a 5 PB MinIO cluster exposed via the S3 Application Programming Interface (API), the Bluesky acquisition pipeline writes NeXus event documents there, and downstream notebooks read them over the Wide Area Network (WAN) [25]. The UK’s Rutherford Appleton Laboratory offers the ECHO object store (ECHO) Ceph object store, providing 100 PB of S3 capacity that underpins IRIS cloud analysis service (IRIS) cloud analysis services and beamline portals [26]. In these environments, transferring even a few gigabytes across a WAN to inspect scan headers is inefficient when lightweight, metadata-only queries can satisfy the request.

2.2 Storage Foundations: HPC vs Object Stores

This section contrasts parallel file systems with cloud object stores to highlight the core trade-offs in throughput and latency. We review how Lustre/GPFS/Bee Global File System (BeeGFS) achieve high sequential I/O and why many small reads and metadata traffic degrade performance, then summarize S3 semantics, costs, and latency.

2.2.1 Traditional HPC Storage Paradigms

Parallel file systems such as Lustre, IBM GPFS, and BeeGFS stripe files across many disks and coordinate access with distributed locking, delivering high sequential throughput [27, 28]. These designs degrade under many small reads and heavy metadata traffic, which production traces show small transfers dominate call counts and metadata can consume a large share of requests [29].

When the I/O profile shifts from a few large checkpoints to many small look-ups and interactive slices, these design assumptions break down. A study from the Summit and Cori supercomputers shows that small transfers (<1 MB) account for the overwhelming majority of read and write calls at both the file and process levels, and metadata operations can saturate metadata servers and become a primary scalability bottleneck [29]. This statement concerns operation counts rather than total transferred bytes: large sequential reads can still dominate data volume even when they account for fewer individual I/O calls.

2.2.1.1 Parallel File Systems (Lustre, GPFS, BeeGFS)

Lustre, IBM GPFS (now Spectrum Scale) [28], and BeeGFS have dominated supercomputing storage for over two decades, consistently delivering throughput of hundreds of gigabytes per second. By striping data across hundreds of disks and coordinating access with distributed locks, these systems excelled at writing multi-gigabyte files sequentially. The same system, however, reveals hidden costs when workloads shift to small, random reads or to remote, WAN-mounted access.

Classic HPC file systems split responsibility between *metadata servers* (Metadata Server (MDS)) and many *object/data servers* (Object Storage Server (OSS) in Lustre, Network Shared Disk (NSD) servers in GPFS, storage targets in BeeGFS). Files are *striped* across a few to hundreds of disks in

multi-megabyte chunks. Doing this allows a single sequential read or write to optimize the bandwidth of an entire system. **Parallel File System (PFS)** uses token-based distributed locking to keep **POSIX** consistency on a shared-disk **Storage Area Network (SAN)** [28]. **BeeGFS** achieves similar semantics over Ethernet by separating data and metadata and scaling the service.

Parallel file systems shine whenever an application issues long, contiguous transfers, checkpoints/restarts, volumes, and snapshots. NASA’s *Pleiades* cluster still sustains $\sim 1 \text{ GB s}^{-1}$ from disk for a single 512-rank IOR stream and bursts to 9 GB s^{-1} when the same data are serviced out of the page cache [30]. Modern flagships go far beyond that; Los Alamos’ *Trinity* Sonexion Lustre tops out near 1.6 TB s^{-1} for a 78 PB pool [31]. Even a single E1000 **OSS** inside NERSC’s *Perlmutter* can saturate 41 GB s^{-1} read and 27 GB s^{-1} write to one **Object Storage Target (OST)** [32]. These traits explain why parallel file systems remain the choice for bulk sequential scans and multi-terabyte tomography stacks.

Lustre separates responsibilities between dedicated *metadata servers* (**MDS**) that keep inodes and directory state on one or more *metadata targets* (MDTs) and multiple *object-storage servers* (**OSS**) that deliver the bulk data stored on *object-storage targets* (**OSTs**). Files are *striped* across a configurable number of **OSTs** (stripe count) with a tunable stripe width, enabling a single sequential transfer to engage hundreds of disks in parallel. The design of flagship systems such as Oak Ridge’s Spider-3 sustains more than 1 TB s^{-1} aggregate read bandwidth [10, 33].

IBM **GPFS** employs a distributed *token-based locking* protocol to coordinate metadata and data updates across all nodes, persisting mutations in replicated journals so that failed servers can recover without full replays. This mechanism provides stronger resilience and copes slightly better with so-called metadata “storms” than Lustre, though at the cost of a more complex recovery state [28].

Parallel file systems raise aggregate bandwidth by slicing every file across many Object Storage Targets (**OSTs/NSDs**). When workloads flip from multi-GB checkpoints to thousands of 4–64 kB reads, the same striping logic backfires. Intel’s LUG-2018 “Small File I/O” report shows that files smaller than the **Data on Metadata (DoM)** threshold generate disproportionately more RPCs per MiB, leading to significantly reduced bandwidth on multi-**OST** Lustre systems [34].

Even in a namespace-sharded file system like Lustre’s **Distributed Namespace Environment (DNE)**, a camera-style “one-file-per-frame” workflow still concentrates its create/unlink traffic in a single hot directory subtree.

Production traces from 60 HPC systems show that 64 % of files are smaller than 64 kB. When a system is flushed with thousands of tiny HDF5 or JPEG files, metadata operations increase exponentially. Based on these 60 different HPC systems, research shows that metadata operations can account for up to 80 % of total file system operations [35].

HPC administrators often impose per-user inode quotas because millions of tiny files overload Meta Target Devices [36]. Therefore, monolithic containers such as HDF5 can be preferred on-prem, whereas object stores favor chunk-per-object layouts (e.g., Zarr).

2.2.1.2 POSIX-Centric Scientific Formats

Most HPC codes write *monolithic containers*, a single POSIX-seekable file that includes raw arrays and metadata. HDF5 popularised this approach, while NeXus adopts the same file structure with beamline-specific naming conventions. The design is perfect for bulk, sequential, local parallel file system scans. However, the need to search through super-blocks and B-trees for every small slice makes it poorly suited to cloud object stores and interactive, many-small-read workflows, motivating the chunk-per-object formats.

The Hierarchical Data Format 5 (HDF5) stores every dataset, chunk, and attribute in a single byte-addressable file whose internal super-block, B-trees, and heap tables, with the help of POSIX, seek for navigation [37]. Nexus, the de facto standard at many photon and neutron sources, layers a domain system on top of the same HDF5 container, retaining identical I/O semantics while prescribing beamline-specific group names and units [38]. This “all-in-one” layout simplifies file management but couples performance to the underlying file-system block size and to how well metadata caches handle random seeks.

Out-of-the-box HDF5 access (via h5py, MATLAB, etc.) issues many small, non-contiguous POSIX reads. Each rank that samples a 4 MB to 8 MB slice generates dozens of tiny, <64 kB reads, leading to many metadata lookups. Small, non-contiguous selections generate many <64 kB reads and metadata lookups, collective I/O reduces but does not eliminate this overhead [39, 40]. Single-file heritage makes many small-read interactive workloads sensitive to metadata hotspots, motivating chunk-per-object designs.

Even when using collective metadata I/O, each HDF5 read still must fetch the superblock and descriptive metadata before reading the raw data. In the CCGrid’21 benchmarks on Summit and Cori, metadata I/O alone accounted for as much as $\approx 58\%$ of the total read time [40]. HDF5/NeXus excels for

large, sequential scans and single-node post-processing, but its single-file, **POSIX**-style heritage makes it vulnerable to metadata hotspots when handling many small reads, motivating object-store-oriented layouts such as chunk-per-object arrays (e.g., Zarr) and columnar row-group formats (e.g., Parquet).

2.2.2 Cloud Object Storage Fundamentals

Cloud object stores, exemplified by Amazon **S3**, provide a flat, immutable-object namespace, global durability, and a pay-as-you-go pricing model. They remove local capacity provisioning constraints but introduce **HTTPS**-level latency and per-request charges that do not exist in local **NVMe** or parallel file systems.

2.2.2.1 Architectural model

Amazon Simple Storage Service (**S3**) organizes data in *buckets* that share a *flat key namespace within a bucket*, and directory hierarchies are only a key-naming convention. Since December 2020, **S3** guarantees *strong read-after-write consistency* for all operations, removing the race conditions that earlier eventual-consistency semantics introduced [41].

2.2.2.2 Request classes and cost drivers

Every interaction with **S3** is an **HTTPS API** call in one of four core request classes. Each call is billed in addition to storage (GB month) and data egress charges [42].

Class	Typical purpose
PUT/POST/COPY	Create or modify objects
GET	Retrieve object data (full or ranged)
LIST	Enumerate keys/prefixes
HEAD	Fetch metadata, tags, ACLs

Table 2.3: Core **S3** request classes

2.2.2.3 Latency characteristics

Object-store access has a very different latency profile from local or shared file systems. **AWS** documentation reports that latency-sensitive **S3** applications

can achieve consistent small-object latencies, and first-byte-out latencies for larger objects, of roughly 100 ms to 200 ms [43]. In comparison, on-premises **NVMe** storage and parallel file systems are commonly designed around sub-millisecond access paths, while current **AWS Elastic File System (EFS)** documentation reports best-case first-byte latencies of approximately 1 ms for reads and 2.7 ms for writes for the Standard storage class under optimal conditions [44]. This latency gap helps explain why unmodified **HPC** workloads often perform poorly on **S3**. Figure 2.3 visualizes the comparison on a log scale, showing how the jump from sub-millisecond local or shared-file-system access to 100 ms to 200 ms **S3** requests forces cloud-native workflows to amortize the round-trip cost through chunking, caching, and request batching.

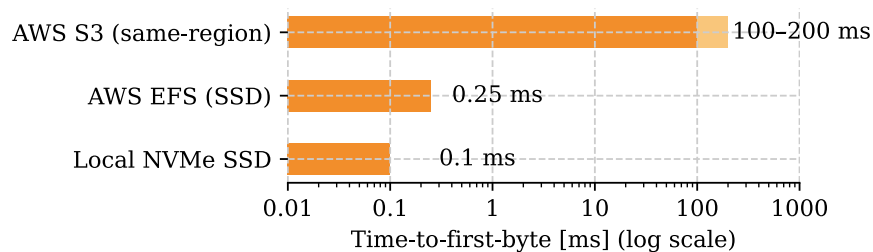


Figure 2.3: Median time-to-first-byte read across common storage tiers, plotted on a logarithmic axis. The three-order-of-magnitude gap between local **NVMe** and **S3** storage motivates the use of chunking and request strategies.

2.2.2.4 Performance-optimisation layers

- **Multipart uploads:** split objects (≥ 100 MB) into 5 MB to 5 000 MB parts that upload in parallel and resume after failure [45].
- **Range GETs:** request only required byte spans, ideally aligned with the same 8 MB to 16 MB boundaries used during uploads [46].
- **Prefix parallelism:** **S3** scales to at least 5 500 GET/HEAD and 3 500 write requests per *prefix*. Optimally sharding key lifts can aggregate throughput to multi TB/s [43].

These optimization strategies allow object storage to approach **HPC**-class throughput, but latency for small, random slice reads remains significantly higher than on local or **NVMe**-backed file systems.

2.3 Cloud-Native Models and Compression

Object stores provide scale but add request latency, immutability, and per-request billing [47]. Cloud-native models partition arrays or tables into independently addressable objects to exploit concurrency while controlling slice-read cost.

2.3.1 Cloud-Native Storage Models

Cloud object stores such as Amazon S3, Azure Blob Storage, and Google Cloud Storage provide effectively unlimited capacity and global accessibility. However, they introduce request latency, and per-request billing, which clash with the low-latency assumptions of traditional HPC formats [47]. To exploit cloud elasticity without prohibitive slice-read costs, a new generation of *cloud-native storage models* has emerged, each rethinking how scientific arrays or tables are partitioned, compressed, and addressed in an object store.

This thesis does not attempt an extensive survey of all scientific file formats. The goal is to compare a small set of *adoption-realistic* storage models that could reasonably be deployed for multi-terabyte scattering data on Amazon S3, and that span the main design choices that drive object-store performance. The candidate set of storage methods was selected to satisfy three criteria:

- **Practical deployability on S3:** an open specification and a maintained implementation with native object-store support (client-side or via a service layer).
- **Representativeness for the studied workflow:** ability to store either dense detector arrays or associated tabular metadata, and to support the dominant access patterns in this thesis (random frame/slice reads and full sequential scans).
- **Coverage of distinct layout families:** monolithic file layouts (HDF5), chunk-per-object key–value arrays (Zarr), HDF5 semantics with sharded chunks/metadata exposed via a REST service (HSDS), fragment-based arrays with metadata pruning (TileDB), and single-object page/cluster layouts accessed via range reads (ROOT/TTree).

While a custom storage layout could in principle be engineered for this dataset, it would still have to specify the same core mechanisms as

any production format: a chunking granularity, an object naming and sharding scheme, a metadata/index structure, a compression pipeline, and concurrency/consistency semantics. These choices map closely onto the layout families represented above. In particular, Zarr approximates the simplest viable custom design (chunk-per-object with lightweight metadata), while TileDB and **TTree** represent richer indexing and streaming strategies. Designing, implementing, and validating a new storage engine would shift the thesis from comparative evaluation to system construction and would reduce reproducibility for other groups. For these reasons, the study focuses on established formats and uses the experimental results to derive design implications that would also inform any future custom solution.

In addition to the conceptual layout differences, the storage candidates were chosen because they were practical to deploy and benchmark on Amazon **S3** under a single, reproducible test environment. Specifically, each option maintains a stable **S3** I/O path (either direct object access or a stateless service layer), supports partial reads without a **POSIX** gateway, and exposes tunable physical granularity (chunks/tiles/pages) and standard compression filters used in this thesis. The five candidates also represent distinct *deployment and I/O paths* that an engineering team would realistically choose between:

- **HDF5+HSDS**: preserves **HDF5**/NeXus compatibility while adding an explicit REST service that shards chunks and metadata into **S3** objects.
- **Zarr**: a serverless, direct-to-**S3** chunk-per-object store with minimal metadata, representative of the simplest cloud-native array design.
- **TileDB**: a library-based storage engine that writes immutable fragments plus fragment metadata (enabling pruning and consolidation), representing an “engine + metadata” approach rather than pure chunk key value store.
- **Parquet**: the dominant columnar format in the cloud analytics stack, included to cover tabular experiment metadata and to document the engineering mismatch between row-group I/O and frame/tile access (the detector-stack mapping is explored only via a single-object pilot; Appendix **A.1.9**).
- **ROOT/TTree**: a largely single-object layout with internal paging/clustering accessed via Range GETs, providing a counterpoint to object-sharded designs.

Table 2.4 previews the implied storage granularity, object counts, and the dominant engineering pain-point for each approach.

Table 2.4: Storage granularity, object-count, and dominant pain-point for each candidate format. Figures assume a 1 TB tomography volume stored with 16 MiB scientific chunks (except Parquet) or as a partitioned Parquet dataset with 256 MiB data files.

Format	Storage type	Objects [†]	Key pain-point
HDF5 (single file)	File-per-dataset	1	a
HDF5 + HSDS	Chunk-per-object	≈ 65,536	b
Zarr v3	Chunk-per-object	≈ 65,536	c
Parquet	Partitioned data files (256 MiB)	4,096	d
TileDB	Fragment dir with per-attribute tile files	≈ 4 [‡]	e
ROOT / TTree	Single file with clustered pages	1	f

[†] Approximate count of **POSIX** inodes or **S3** objects, computed from 1 TB and the stated granularity.

[‡] TileDB stores many tiles per file; object count scales with fragments and attributes. The figure shown assumes one write fragment and one fixed-size attribute (files: array schema, fragment metadata, one or two per-attribute data files) [48].

^a Monolithic file on **S3** yields many Range-GETs per small slice; metadata B-tree + data chunk require multiple RTTs; latency dominates [49].

^b Extra REST hop through **HSDS**; opening large hierarchies fans out **JavaScript Object Notation (JSON)** metadata reads before data access.

^c Large keyspaces cause LIST/HEAD storms on **S3** and inode pressure on **POSIX**; use consolidated metadata to cut opens [49].

^d Sub-row-group slice pulls entire row group, causing over-fetch and higher egress for small selections [50].

^e Many small fragments increase first-open cost (fragment-metadata scans) and LIST/HEAD; mitigate with fragment/fragment-metadata consolidation and vacuuming [51].

^f Single **S3** object; sparse event picks trigger many Range-GETs against small pages and index lookups, so RTT dominates unless reads are batched.

2.3.1.1 **HDF5 + HSDS**

The **HDF5** ecosystem represents the most mature and widely adopted solution for storing complex scientific data, spanning disciplines from climate modeling to synchrotron beamline acquisition [52]. However, developers designed the format for low-latency, parallel file systems. When migrated naively to cloud object stores, its monolithic layout magnifies request latency and per-operation costs. To preserve **HDF5**'s rich data model while overcoming these cloud-specific bottlenecks, the Highly Scalable Data Service (**HSDS**) was created as a stateless REST layer that shards chunks and metadata into individually addressable objects [53].

Hierarchical Data Format 5 (**HDF5**) is an open-source container, data model, and I/O library designed to store extensive, multidimensional scientific data efficiently [52]. An **HDF5** file consists of *datasets* (n-dimensional arrays) organized in a **POSIX**-style hierarchy of groups. Datasets may be stored either *contiguously* or as fixed-size *chunks*. Chunking enables transparent filter pipelines (e.g., `gzip`, **LZ4**, **ZSTD**) and independent partial reads/writes. At the same time, the library's **MPI-IO** pathway lets thousands of compute nodes access different chunks concurrently [54]. These strengths made **HDF5** the standard library/system used at X-ray, neutron, and electron facilities.

Placing a monolithic **HDF5** file on Amazon **S3** causes each **POSIX** `read()` call to issue a separate **HTTP** Range GET. Therefore, a single 1 MB slice triggers two network round-trips (metadata + data), amplifying latency and incurring request fee costs that traditional **HPC** deployments never experience [55]. Moreover, the file's internal B-tree indices are optimized for low-latency block storage, not for the 100 ms to 200 ms object-store round-trip typical of Amazon **S3**.

The *Highly Scalable Data Service* (**HSDS**) re-implements the **HDF5** abstraction as a stateless REST **API** [53]. Internally, **HSDS** maps every chunk and metadata item to separate objects (**S3**, Azure Blob, **MinIO**, or **POSIX**) and maintains small **JSON** documents that map logical dataset coordinates to object keys. This exposes **HDF5** groups, datasets, and attributes over standard **HTTP** requests.

Production **HSDS** clusters back multi-hundred-terabyte public archives, e.g., **National Renewable Energy Laboratory (NREL)**'s 500 TB Wind Toolkit dataset, enabling subsetting without full downloads [56]. **HSDS** retains **HDF5**'s rich ecosystem yet provides proper cloud-native capabilities.

Most Lustre and **GPFS** installations cap the number of inodes a single user may allocate because millions of tiny files saturate the metadata target (MDT) servers long before disk bandwidth runs out [34]. Therefore, a monolithic **HDF5** container carries a practical on-prem advantage: *one inode per file* and negligible directory pressure. By contrast, chunk-per-object formats such as Zarr can create 10^5 – 10^7 objects for a single 1 TB tomography scan, a scenario many **HPC** administrators actively discourage or block via strict inode quotas. The same characteristic becomes an asset in cloud object stores, where inodes do not exist, and latency can be hidden behind massive parallelism for GET/PUT.

HDF5's internal super-block, heap tables, and fractal B-trees replicate several file-system duties to offer **POSIX**-seekable random access [37]. On low-latency local storage, this overhead is minor; however, when a single 4

MB slice is served from [S3](#), it incurs two round-trips: first to fetch the B-tree node, then the data chunk, doubling both latency and request cost. Chunk-per-object layouts such as Zarr avoid this indirection because the chunk itself is the addressable unit. Columnar and paged formats (Parquet, [ROOT/TTree](#)) instead pay an initial footer/index load and then use Range GETs against contiguous regions within a file.

2.3.1.2 Zarr

Zarr is an open-source system for storing chunked, compressed n -dimensional arrays as small objects [\[57\]](#). It partitions each array into regular *chunks* and stores every chunk as an independent object (e.g., a file in a [POSIX](#) system or a key in Amazon [S3](#)). Zarr supports codecs such as `gzip`, [LZ4](#), and [ZSTD](#), and uses [JSON](#) metadata files. The format requires no central server, which suits cloud object stores where high-latency, high-throughput parallel access dominates.

Because each chunk maps to a [S3](#) key, multiple parallel GETs can execute simultaneously, achieving high aggregate bandwidth. Time-to-first-byte per chunk on [S3](#) is typically 100 ms to 200 ms where concurrency improves throughput rather than single-request latency. Write throughput scales similarly with thousands of concurrent PUTs (one per chunk), enabling high-rate ingestion at modern beamlines with acceptable overheads.

Researchers across various fields have adopted Zarr; [Geospatial Data Abstraction Library \(GDAL\)](#) provides a native driver for geospatial rasters [\[58\]](#) while the bioimaging community standardized on [Open Microscopy Environment \(OME\)-Zarr](#) for petabyte-scale microscopy archives [\[3\]](#). A NumFOCUS-backed governance model and a public Zarr-specs [Request for Comments \(RFC\)](#) process support long-term stability [\[57\]](#).

Zarr’s chunk-per-object design aligns well with the partial-read workloads studied in this report. However, its performance depends on chunk size, sharding, and metadata unification.

2.3.1.3 Apache Parquet

Apache Parquet is an open, columnar storage format originally developed at Twitter and Cloudera to bring Dremel-style analytics to the Hadoop ecosystem. It organizes a file into one or more *row groups*. Within each row group, every column is stored contiguously as a *column chunk*, compressed using lightweight compression and dictionary encoding. By reading only the required columns and row groups that satisfy a filter predicate, Parquet reduces

I/O volume for wide, sparse tables. Parquet supports several compression codecs, such as Snappy, `gzip`, deflate, and BZIP2 [50]. Within each row group, Parquet applies column-specific encodings—such as dictionary or run-length encoding—and independent compression, further shrinking the on-disk footprint and amplifying the format’s I/O-reduction advantage.

On Amazon S3, each row group is laid out as a large (typically 64 MB to 256 MB) contiguous byte range, allowing clients such as Apache Arrow’s vectorized reader or AWS Athena to fetch gigabyte-scale slices with a single Range GET. The AWS Athena pricing page states that converting data to compressed, columnar formats like Parquet “can save up to 90 % per query and improve performance” compared with raw Comma-Separated Values (CSV) [59].

Parquet has become the default on-disk file format throughout today’s analytics stack. Apache Spark uses Parquet as the *default data source*, and Apache Iceberg likewise keeps its immutable data files in Parquet [60]. In the in-memory layer, Pandas and Apache Arrow provide first-class readers/writers for Parquet [61]. Primary cloud analytics services natively ingest Parquet, including Google BigQuery [62], AWS Glue [63], and Amazon Athena [64].

Parquet’s columnar design is well-suited to the tabular metadata portion of scattering experiments, where scientists frequently select a handful of columns across millions of rows. However, its row-group granularity (tens of MB) and lack of native n -D chunking can inflate latency for small pixel tiles.

2.3.1.4 TileDB

TileDB is an open-source array storage engine for both dense and sparse data, organizing writes to immutable, timestamped *fragments* [65, 48]. Each fragment contains attribute data serialized into *data tiles*. For object stores, TileDB uses a virtual filesystem with native backends for S3, Azure Blob Storage, and GCS. On AWS S3, it writes using multipart uploads to create append-only fragment objects [66, 67]. The array directory contains the schema, per-fragment data, and fragment metadata [68].

Unlike chunk-per-object layouts, a single fragment typically produces a small, fixed set of objects. It generates one attribute file per attribute, one fragment-metadata file, one write-marker record, and the already-existing schema file. The total object count scales with the number of *fragments* and attributes rather than the number of tiles, which reduces LIST/HEAD amplification on S3 for large arrays [48, 68]. Excessive fragmentation, however, increases first-open overhead because readers must fetch per-

fragment metadata [51].

Upon opening, TileDB loads fragment metadata and per-tile min-max statistics. The system then queries the pruned fragments and data tiles whose minimum-bounding rectangles (MBRs) do not intersect the requested subarray [68, 69]. Reads support ordered or unordered layouts, multi-range subarrays [70].

All writes create new immutable fragments and write-marker files, enabling lock-free multiple-reader/multiple-writer behavior and consistent reads at a chosen timestamp range (“time traveling”). Fragment merging helps compact many small fragments into larger ones, allowing space to be reclaimed [71].

TileDB applies a per-attribute filter pipeline to tiles. Supported compressors include `gzip`, **ZSTD**, **LZ4**, RLE, BZIP2, and integer-oriented transforms (bit-shuffle, bit-width reduction, double-delta). Checksums (MD5/SHA-256) are available for integrity [72, 73, 74].

2.3.1.5 **ROOT/TTree**

ROOT is an object-oriented C++ framework developed at **European Organization for Nuclear Research (CERN)** to handle the petabyte-scale data streams of high-energy-physics experiments [75]. Its core storage abstraction, **TTree**, stores events as rows and detector variables as *branches*. The format splits each branch into contiguous *baskets*, allowing readers to load them independently. This column-friendly layout lets analysis jobs load only the necessary branches, minimizing I/O and deserialization overhead, a property directly relevant to slice-level scattering analysis.

This thesis benchmarks the **ROOT/TTree** path only. **ROOT Ntuple (RNTuple)** is discussed here as the newer redesign of **ROOT** I/O and as future context, it was excluded from timed measurements because the implemented **ROOT** benchmark used the established **TTree** path. Evaluating RNTuple would require a separate ingestion and tuning path and is therefore left to future work.

After two decades of incremental evolution, **TTree** showed architectural limits for modern heterogeneous hardware and cloud storage. The **RNTuple** project, therefore, re-implements **ROOT** I/O with a stricter columnar layout, page-based compression, and a self-describing, versioned metadata footer [75]. Pages (≈ 8 kB to 16 kB) are grouped into clusters so that entirely columnar reads issue a single contiguous request, while sparse event selections fetch only the referenced pages. By decoupling metadata from data pages,

RNTuple avoids rewriting the entire file when appending events, which is critical for streaming detectors and results in improved partial reads.

Through the generic **ROOT** Page Source interface, pages can reside in **POSIX** files, over **HTTP(S)**, or as individual **S3** objects [76]. Scaling studies by Blomer showed near-linear throughput up to $\mathcal{O}(10^2)$ concurrent readers on a >100-core **HPC** cluster served by a **Distributed Asynchronous Object Storage (DAOS)** object store, achieving an aggregated read rate above 35 GB s⁻¹ before the storage system became the bottleneck [77].

This newer **RNTuple** architecture provides high-throughput, scalable, and selective access to petabyte-scale datasets on **HPC** systems and in the cloud. In this thesis it serves as background and future context only; the measured **ROOT** results are for **TTree**.

2.3.2 Format design trade-offs for this thesis

From a computer-science perspective, the five formats in this study represent different points in a shared design space: how an n -dimensional or tabular dataset is mapped to physical storage, how metadata is indexed, and how many objects and round-trips each logical operation induces. This subsection summarises those design choices and how they motivate the experiments.

2.3.2.1 **HDF5/HSDS**

HDF5 defines a hierarchical container of datasets (multidimensional arrays) and groups, backed by a single byte-addressable file. **HSDS** replaces that monolithic file by sharding each **HDF5** chunk and its metadata into separate objects and exposing them via a REST **API**. Algorithmically, each logical array access becomes:

- a metadata lookup in small **JSON** documents that map indices to object IDs
- one or more chunk-object GETs.

The strength is a rich, self-describing data model and interoperability with existing **HDF5/NeXus** tooling. The weakness for this thesis is that **HSDS** introduces an extra “service layer” hop and many small metadata objects, which can increase both GET counts and per-slice latency compared with formats that go directly from index $\rightarrow$ object key. This motivates testing whether **HDF5/HSDS** can match newer array formats in terms of slice latency (RQ1) and scan throughput (RQ2) when deployed on Amazon **S3**.

2.3.2.2 Zarr

Zarr models an n -dimensional array as a pure key–value mapping: each regular chunk is stored as an independent object, and a small number of **JSON** metadata files describe dtype, chunk shape, and codecs. A read of a slice that aligns with k chunks triggers exactly k object GETs, with no intermediate service layer. Zarr’s strengths in this study are:

- direct chunk-to-object mapping that enables embarrassingly parallel GETs
- minimal per-chunk metadata overhead once the **JSON** headers are loaded.

Weaknesses include large keyspaces (many objects per dataset) and the cost of listing or opening unconsolidated stores. These properties lead directly to the hypotheses that Zarr should perform well for interactive slices (few chunks per frame) and moderately well for full scans, and that its request intensity (GETs/GiB) will be tightly tied to chunking (RQ1–RQ3).

2.3.2.3 Parquet

Parquet models data as a table of rows and typed columns. The file is organized into row groups, and within each row group, each column is stored as a contiguous column chunk with its own encoding and compression. From a layout perspective, the fundamental unit of I/O is the row group (tens to hundreds of MiB), not an n -D image tile. This is advantageous for analytics workloads that scan or filter on a subset of columns: a query engine can skip entire row groups and untouched columns. In this thesis, the same design is a weakness for detector frames: loading a single 2-D slice within a row group pulls the entire row group and incurs a high metadata-open cost. These semantics motivated the pilot-only Parquet experiment and its exclusion from timed n -D slice/scan benchmarks (documented in Chapters 4–5).

2.3.2.4 TileDB

TileDB generalises dense and sparse arrays as a set of immutable fragments. Within each fragment, attribute values are stored in tiles; the fragment directory holds array schema, fragment metadata (including tile ranges and min–max), and one or more attribute files. On object stores, reads proceed in two stages:

- load and prune fragment metadata using tile bounds and statistics
- issue a small number of large GETs for the relevant attribute tiles.

The strengths of this study are:

- a low object count that scales with fragment count rather than tile count
- efficient full scans where long, contiguous tile ranges can be streamed

The drawbacks are higher first-open costs when fragment counts are high and slightly higher per-fragment complexity than in a pure chunk-per-object layout. These design choices make TileDB a strong candidate for high-throughput full scans (RQ2) and for moderate-latency slices that benefit from pruned, long-range reads (RQ1, RQ3).

2.3.2.5 **ROOT/TTree**

The timed **ROOT** condition in this thesis is specifically **ROOT/TTree**; **RNTuple** is discussed only as the newer redesign and as future context. In **TTree**, each branch behaves like a column and data are organized into baskets that can be fetched independently. A read of a contiguous range of events on a few branches can be served by a single or a small number of long-range GETs; a sparse selection of events or an awkward “frame-shaped” slice may touch many small pages. The format’s strengths are:

- fine-grained, columnar access for event-style analyses
- efficient sequential scans when reading along the event axis

The weaknesses of this thesis are that detector frames are not the natural access unit and that sparse, frame-random reads can devolve into many small-range GETs against a single large object. This leads directly to the experimental question of whether **TTree** can still offer competitive full-scan throughput and what latency penalty it incurs for frame-based slices (RQ1–RQ2).

2.3.3 **Compression Algorithms Reviewed**

Lossless compression is a first-order lever for improving performance and reducing costs in cloud object-store workflows. Smaller chunks reduce **S3**

egress and request fees, while faster codecs shorten the latency that dominates interactive partial reads in synchrotron experiments.

The focus is on three open-source codecs that form a spectrum of trade-offs: *gzip/DEFLATE* (high ratio, high CPU), *LZ4* (low ratio, ultra-low latency), and *ZSTD* (balanced). All four backends under test (*HDF5/HSDS*, Zarr, TileDB, *ROOT/TTree*) support *gzip*, *lz4*, and *zstd* (built-in or via standard filters/plugins).

2.3.3.1 *gzip*

GNU *gzip* has been the default lossless compression tool on Unix-like systems since 1992. RFC 1952 formalizes *gzip*'s file format and header/trailer structure, and those specifications have remained patent-free and backward-compatible across three decades of scientific workflows [78].

The compressor implements the *DEFLATE* scheme, a cascade of Lempel–Ziv 1977 (*LZ77*) sliding-window substitution, followed by static/dynamic Huffman coding, tunable via nine quality levels that trade compression time for compression ratio. Level 6 is the GNU default, while level 1 sacrifices a $\approx 30\%$ ratio to gain $3\text{--}4\times$ speed [78].

Benchmarks on floating-point climate and remote-sensing datasets stored in Network Common Data Form (*netCDF*)-4/*HDF5* show typical space savings of $1.6\text{--}2.2\times$ at levels 4–6. In these studies, *gzip* incurred $2\text{--}5\times$ higher CPU costs than *LZ4*, and decompression throughput saturated around 500 MB s^{-1} per core [79, 80].

Native support for *gzip* exists in all five tested backends. Because *gzip* compresses each chunk independently, its compression ratio varies with chunk size and should be taken into account in these experiments. In the experiments, *gzip* is used as a “high-ratio, high-CPU” reference, and its mature ecosystem guarantees consistent behavior across all five storage backends. Its well-characterized latency overhead provides a baseline against which to measure the partial-read performance of newer codecs like *ZSTD* and *LZ4*.

2.3.3.2 *LZ4*

Introduced in 2011, *lz4* emerged as a lightweight derivative of the Lempel–Ziv (*LZ77*) family, designed to maximize streaming throughput while keeping memory overhead to a minimum. Its open-source framing format defines a compact header, provides optional block and content checksums, and employs a skippable frame mechanism, allowing new features to be added without breaking older decoders. These attributes have led to

widespread adoption in operating systems, high-performance storage engines, and scientific data formats where sub-millisecond decode latency is critical.

The codec emphasizes extremely fast byte-wise parsing and a greedy match-finder that limits back-references to 64 kB. Because of this, the compression runs at $>500 \text{ MB s}^{-1}$ per core while decompression routinely exceeds 2 GB s^{-1} to 3 GB s^{-1} , though with modest ratios ($1.2\text{--}1.7\times$ on floating-point data) [81, 82]. A high-compression variant `lz4hc` improves the ratio by $\sim 15\%$ at $3\text{--}5\times$ the CPU cost.

Almeida *et al.* compared LZ4 with zlib compression library (zlib) on ROOT HEP and climate grids, reporting 1.0 GB s^{-1} to 1.2 GB s^{-1} decompression (versus 120 MB s^{-1} for zlib) at the expense of only a 5–10 % ratio loss [83]. The Very Large Data Bases (VLDB)’24 “Floating-Point Compression Benchmark (FCBench)” study recently measured the CPU variant bitshuffle with LZ4 across 33 floating-point datasets. The study showed that it sustained 0.9 GB s^{-1} compression and 1.2 GB s^{-1} decompression on 64 kB blocks while achieving a median compression ratio of 1.43 [80].

As a codec, lz4 filters are first-class options in HDF5 (H5Z_LZ4), Zarr v3, Parquet, and TileDB. Because the lz4 64 kB match window limits cross-chunk redundancy, chunk sizes above 1 MB to 2 MB yield diminishing returns, which for this study aligns well with S3 request-cost sweet spots.

In our benchmarks, LZ4 represents the “low-latency, low-CPU” option, serving as the low-latency reference alongside gzip and ZSTD. The benchmarks examine how far compression ratio can be traded for lower partial-read latency without materially increasing monthly S3 storage costs.

2.3.3.3 ZSTD

ZSTD (zstd) was released in 2016 and standardized by the IETF as RFC 8878 [84]. It combines an LZ77 dictionary back-reference engine with Finite-State Entropy (Finite State Entropy (FSE)) coding. It supports window sizes up to 128 MiB and offers 23 quality levels (-7 to 22), optional long-range mode, and trained dictionaries (negative values activate ultra-fast modes that trade ratio for speed). Decompression speed is deliberately kept almost constant across levels, whereas compression trades CPU for ratio.

According to the zstd manual, the fastest compression mode exceeds 200 MB s^{-1} per core, while the decoder sustains $>500 \text{ MB s}^{-1}$ per core on commodity x86 CPUs [85].

Promberger *et al.* measured zstd level 2 on a 4.9 GB Large Hadron Collider beauty (LHCb) collision dataset and found that its compression

throughput was roughly $2\text{--}3 \times$ higher than that of *zlib* level 2 while achieving an almost identical compression ratio [86].

Delaunay et al. found that “Shuffle + Zstandard-2” compresses *China–France Oceanography Satellite (CFOSAT)* and *Surface Water and Ocean Topography (SWOT)* remote-sensing files about $1.4 \times$ to $7 \times$ faster than the mission’s baseline “Shuffle + Deflate-3/4”. It delivers virtually identical space savings, accelerating decompression by up to seven-fold [79]. Across the full 33-dataset *FCBench* suite, the *VLDB’24* study reports that a *bitshuffle+zstd* pipeline achieves a harmonic-mean compression ratio of 1.47, while sustaining compression and decompression throughputs of $\sim 1.4 \text{ GB s}^{-1}$ and $\sim 1.3 \text{ GB s}^{-1}$, respectively [80].

As a codec, *ZSTD* is a first-class option in Parquet and TileDB, and is available as an extension for *HDF5* and Zarr v3. With *FSE* contexts resetting per block, 1–4 MiB chunks maximize the ratio without inflating *S3* request counts. *ZSTD* occupies the “balanced ratio/latency” middle ground between *gzip* (high ratio, high *CPU*) and *LZ4* (low ratio, low *CPU*) in the study.

2.3.4 Positioning Relative to Previous Work

The closest related work falls into three streams rather than one directly comparable benchmark. First, object-store studies for scientific arrays and file formats show that the physical layout of chunks, metadata, and range reads strongly affects latency and request count on cloud storage [47, 49, 3, 53]. Second, compression studies for scientific and high-performance data show that codec choice changes both stored bytes and decode cost, with *gzip*, *LZ4*, and *ZSTD* representing different points in the ratio–throughput trade-off [83, 79, 80]. Third, domain-specific systems such as Parquet in cloud analytics and *ROOT/TTree* in high-energy physics demonstrate that columnar or paged layouts can be effective when the access pattern matches the data model [4, 5].

This thesis differs from those streams in goal and methodology. Rather than optimizing one format or one codec in isolation, it holds one detector stack, one *AWS* region, one client class, and two scattering-style workloads fixed while varying storage layout and codec. This provides a controlled, end-to-end comparison of beamline-style frame reads and full scans on *S3*, rather than a general survey of all cloud storage formats. The comparison is therefore narrower than cloud analytics or video/geospatial delivery, but closer to the operational decision faced by synchrotron and neutron facilities that need both interactive inspection and full-data reprocessing.

2.4 Summary and Link to Problem Statement

The background chapter shows how accelerating detector output and evolving collaboration models outpace traditional **HPC** storage, while object storage introduces latency–cost trade-offs. The key insights, mapped directly to the thesis problem, are:

- **Data rates:** Multi-GB s^{-1} streams and multi-TB experiments are routine.
- **Mismatch:** Parallel file systems deliver bulk throughput but suffer under many small reads; **S3** scales throughput via concurrency but has 100 ms to 200 ms TTFB per request.
- **Rise of object-store-oriented formats:** Chunked arrays (Zarr, **HDF5/HSDS**), fragment engines (TileDB), and columnar/paged files (Parquet, **ROOT/TTree**) trade object count, metadata strategy, and I/O granularity to adapt scientific data access to **S3**.
- **Thesis contribution and practical significance:** Prototype ingestion for five formats (Parquet as a pilot), quantitative benchmarking of four backends across small-slice and full-scan patterns, and a decision matrix that informs interactive analysis, cloud spend, and next-generation scientific data platforms.

Taken together, the format descriptions in Section 2.3.1 and the trade-offs in Section 2.3.2 suggest that chunk-per-object layouts (Zarr, **HDF5/HSDS**) should favor interactive slices, fragment-based layouts (TileDB) should favor full scans, and single-file range-read layouts (**ROOT/TTree**) may struggle for frame-granularity reads on **S3**. For Parquet, we report only a single-object pilot mapping of the detector stack (Appendix A.1.9); partitioned Parquet variants were not evaluated. Chapters 3 and 5 formalize these expectations as RQ1–RQ3 and test them empirically on a controlled detector dataset across the four timed backends.

Chapter 3

Methods

We use an experimental benchmarking design within a design–science framework. Provisioned **AWS** infrastructure and a single ~ 20 GiB synthetic detector stack derived from real beamline frames exercise small-slice and full-scan access patterns to stress latency and throughput. A data-collection pipeline captures wall-clock time and per-session **S3** request counts, bytes, and service latencies (Appendix **A.1.4**). Replication and QC settings are in Appendix **A.1.2**. A multi-criteria evaluation then converts these measurements into a decision matrix for selecting cloud-native storage.

3.1 Research Design and Process

We follow a design-science path from idea to evidence: build artifacts, test them, and refine the design based on results.

3.1.1 Research Paradigm

This thesis uses an experimental approach within design-science research (**Design Science Research (DSR)**). We benchmark four cloud-native storage formats (**HDF5/HSDS**, Zarr, TileDB, and **ROOT/TTree**) and three compression algorithms (gzip, lz4, zstd) under controlled workloads and derive guidelines for optimizing cloud-native workflows for data-intensive synchrotron and neutron-scattering experiments.

Within this **DSR** framing, the primary artifacts are (i) the storage and compression configurations that implement alternative cloud-native layouts and (ii) the decision matrix that encodes their performance and cost trade-offs for beamline scientists and engineers working on **AWS S3**.

On the problem-driven side, the study targets three challenges: the massive data volumes generated by modern synchrotron and neutron facilities, the need for rapid, selective retrieval, and the need for cost-efficient cloud storage. On the solution-driven side, it implements and benchmarks cloud-native storage models and compression algorithms to address these challenges.

The research uses an iterative experimentation cycle in which preliminary findings can adjust subsequent tests (Figure 3.1, decision point “Results consistent?”).

3.1.2 Research Process

Figure 3.1 summarizes four stages: planning the experiment, conducting the experiment, analyzing the data, and discussing the results. Together they define the benchmarking framework for the storage formats and codecs on AWS S3.

3.1.2.1 Step 1: Planning the Experiment

The planning stage involved several crucial tasks. First, the objectives of the experiment were defined, focusing on the comparison of four timed storage formats (HDF5/HSDS, Zarr, TileDB, ROOT/TTree) with Parquet as a pilot-only artifact (no timed runs) and three compression algorithms (gzip, lz4, zstd) concerning performance and compression efficiency for partial retrieval in scattering-style object-store workflows.

Key benchmarking variables are read latency, derived throughput, and request- and byte-based outputs. The four timed backends were chosen to span the main S3 layout families available to scientific users: HDF5 semantics exposed through a service layer (HSDS), direct chunk-per-object arrays (Zarr), fragment-based dense arrays (TileDB), and a single-object tree/column layout (ROOT/TTree). Parquet was retained as a pilot to represent the cloud analytics/data-lake family, but it was not timed once the single-object image-stack mapping proved operationally unstable. The three codecs were selected because they are supported across the timed backends and cover a practical spectrum: gzip as a high-ratio legacy baseline, lz4 as a low-CPU/low-latency option, and zstd as a balanced ratio/latency option.

3.1.2.2 Step 2: Conducting the Experiment

For each format, one dataset for each codec (gzip, lz4, and zstd) was created. Chunking was fixed per format and held constant across runs. Two

access patterns were executed per dataset: full scan of the 20 GiB object and random single-frame reads (~ 15.82 MiB per frame). Logged metrics: per-run wall-clock time and per-session **S3** request counts, bytes, and latency percentiles (Appendix **A.1.4**). QC tests and rerun rules are listed in Appendix **A.1.2**.

The primary observations in this thesis are run-level wall-clock times for storage *operations* (one random frame read or one full scan). Such timings are strictly positive, typically right-skewed, and often dominated by multiplicative factors (network RTT, request queuing, decompression cost, and library-level overhead). In addition, consecutive runs are not perfectly independent in a cloud setting: **HTTP**/TLS connections are reused, client-side and library metadata caches warm up, and object-store throughput can vary over time due to shared infrastructure. For these reasons, we analyze $\ln(\text{time})$ and quantify uncertainty with resampling methods (Section **3.1.5** and Appendix **A.1.3**) rather than relying on normal-error, independent-sample assumptions.

An ideal benchmark design would obtain *independent* replications by repeating each cell (format $\times$ codec $\times$ pattern) across many separate, isolated sessions (e.g., different time windows, fresh processes/containers, or even fresh instances) and by interleaving conditions to average out time-varying cloud effects. This would strengthen generalization beyond a single session and would simplify inference.

In practice, fully independent replications are expensive because the cost scales roughly linearly with the amount of data read and the required setup overhead. In this study, there are 24 cells, each with 5 warm-ups and 50 timed runs; the full-scan workload alone reads approximately $20 \text{ GiB} \times 55 \times 12 = 13,200 \text{ GiB}$ ($\approx 12.9 \text{ TiB}$) from **S3**. Replicating each cell across, for example, 5 independent sessions to reduce session-specific effects would increase this to $\sim 66 \text{ TiB}$ and multiply **EC2** runtime and **S3** request charges by a similar factor (plus additional restart/cool-down overhead). Given these constraints, we instead run repeated measurements within a controlled session, randomize run order within the session, apply warm-ups to reduce cold-start effects and approach a near-steady state (residual drift may remain), and treat remaining serial dependence explicitly in the uncertainty procedure (moving-block bootstrap). The resulting cross-format contrasts are therefore reported as *descriptive and session-conditional*.

A practical precision target for the primary estimand (**GM** time/throughput) is relative 95% **Confidence Interval (CI)** half-widths of a few percent for full scans and on the order of 10% for slice reads, while keeping runtime and **S3** traffic tractable. Because interval width typically shrinks approximately

as $1/\sqrt{n}$, increasing beyond $n = 50$ yields diminishing precision gains relative to linear cost. These half-widths were computed after the experiment from the **moving-block bootstrap (MBB)** confidence intervals as the larger relative distance from the **GM** point estimate to the lower or upper **CI** bound, summarized across all cells.

3.1.2.3 Step 3: Analyzing the Data

This step summarizes run-level timings and **S3** aggregates using the estimands and bootstrap procedures in Section 3.3 and Appendices A.1.2–A.1.4.

3.1.2.4 Step 4: Discussing the Results

The last stage interprets the findings for beamline scientists and data engineers by comparing performance and compression trade-offs across the storage configurations. It derives configuration recommendations for high-throughput, low-latency use cases and summarizes them in a decision matrix.

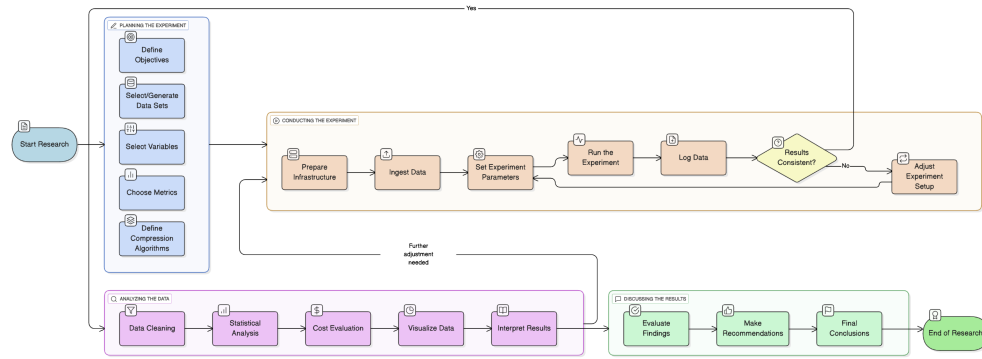


Figure 3.1: Research process flowchart with four stages: planning, conducting, analyzing, and discussing the experiment.

3.1.3 RQ map and estimands

This section maps the three research questions to their experimental units and estimands. For each RQ, we state the workload, the primary and secondary estimands, and the interpretation scope. The shared testbed and run protocol are fixed in sections 3.2 and 4.4.

3.1.3.1 RQ1 — Interactive slice latency

- **Workload.** Random single-frame reads of ~ 15.82 MiB per repetition, as defined in section 3.2. Replication and randomization policy: Appendix A.1.2.
- **Estimands.** GM latency with 95% intervals, 95th percentile latency. Statistical settings: Appendix A.1.3.
- **Scope.** Cross-format contrasts are *descriptive, session-conditional*.

3.1.3.2 RQ2 — Full-dataset throughput

- **Workload.** Full sequential scans over the ~ 20 GiB array. Replication and isolation: Appendix A.1.2.
- **Estimands.** GM scan time as the clearest same-logical-dataset completion metric, GM throughput (GiB/s) as transferred bytes divided by time, and 95th percentile scan time. Statistical settings: Appendix A.1.3.
- **Scope.** Descriptive, session-conditional.

3.1.3.3 RQ3 — Cost posture and mechanisms

- **Service/request metrics.** Per-session S3 GET counts, downloaded bytes, and latency percentiles at 1-minute granularity (Appendix A.1.4). Derive GETs per repetition and GETs per GiB (descriptive only).
- **Storage metrics.** Stored bytes on S3 per {format, codec}, counting data+metadata objects; compression ratios relative to raw baseline; object counts to characterise layout/objectization.
- **Linkage.** Relate GM latency (RQ1), GM scan time, and transferred-byte GM throughput (RQ2) to GETs/GiB and objectization. Cloud-Watch latencies are contextual, not causal; no inference can be made from service/request metrics.

3.1.4 Hypotheses

Based on the storage models in Section 2.3.1, the format trade-offs in Section 2.3.2, and the access patterns in Section 2.1.3, we formulate the following working hypotheses. Each hypothesis is evaluated descriptively using the estimands in Section 3.3, where we quantify uncertainty with dependence-robust confidence intervals but do not perform formal null-hypothesis testing or make causal claims.

3.1.4.1 RQ1 — Interactive slice latency.

H1.1 (layout vs single-object). For a fixed codec, formats with per-frame chunks or fragments (**HDF5/HSDS**, Zarr, TileDB) have lower geometric-mean slice latency than the single-object layout (**ROOT/TTree**), because they can overlap multiple smaller GETs and reduce metadata seek overheads.

H1.2 (layout–codec interaction). For lightweight codecs (`lz4`, `zstd`), the per-frame chunk layout with direct object mapping (Zarr) achieves the lowest slice latencies, whereas under `gzip` the fragment-based layout (TileDB) can outperform Zarr by trading a smaller number of larger GETs for higher decode cost.

H1.3 (request intensity vs latency). Within chunked/fragmented stores (**HDF5/HSDS**, Zarr, TileDB), lower GETs per GiB is associated with lower slice **GM** latency, whereas **ROOT** may violate this pattern because its single-object layout incurs paging and decode overhead even at low GETs/GiB.

3.1.4.2 RQ2 — Full-dataset throughput.

H2.1 (throughput vs layout). For any given codec, TileDB’s fragment layout achieves the highest full-scan throughput, followed by Zarr, with **HDF5/HSDS** and **ROOT/TTree** substantially slower because of higher request fan-out (**HSDS**) or single-object paging overhead (**ROOT/TTree**).

H2.2 (codec effect on scans). For each format, `lz4` and `zstd` achieve equal or higher full-scan throughput than `gzip`, because they lower per-chunk decode cost at similar or only moderately larger stored size.

H2.3 (request intensity vs throughput). Among chunked/fragmented formats, lower GETs per GiB is associated with higher **GM** throughput; **ROOT/TTree** is expected to be an outlier, with low GETs/GiB but relatively low throughput due to constrained parallelism.

3.1.4.3 RQ3 — Cost posture and mechanisms.

H3.1 (codec vs storage). For a fixed format, `zstd` and `gzip` achieve similar compression ratios and both store fewer bytes than `lz4`.

H3.2 (layout vs request cost). For a given codec, Zarr and TileDB achieve lower GETs per GiB than **HDF5/HSDS** at comparable or lower latencies, reflecting their more favorable objectization for this workload.

3.1.5 Statistical formulation and evaluation

The verbal hypotheses in H1.1–H3.2 do not all reduce to one common pairwise contrast. Instead, they concern four estimand classes: geometric-mean (**GM**) time for slice latency, **GM** scan time together with transferred-byte **GM** throughput for full scans, compression ratio, and request intensity (GETs/GiB). We therefore define the relevant estimands separately and evaluate them descriptively using the procedures in Section 3.3 and Appendix A.1.3.

For each cell defined by *format* f , *codec* c , and *pattern* $p \in \{\text{slice}, \text{full}\}$, let $T_{fcp,1}, \dots, T_{fcp,n}$ denote the repeated wall-clock times and

$$Y_{fcp,r} := \ln T_{fcp,r}.$$

The expectation defines the session-conditional target estimand; in practice, it is estimated by the sample mean of the observed log-times within that session. The session-conditional log-scale mean

$$\mu_{fcp}^{\text{time}} := \mathbb{E}[Y_{fcp,r} \mid \text{session}]$$

defines the **GM** time

$$\theta_{fcp}^{\text{time}} := \exp(\mu_{fcp}^{\text{time}}).$$

For slice latency, the primary estimand is $\theta_{f,c,\text{slice}}^{\text{time}}$. For any two formats f and g at fixed codec c , we use the descriptive **GM**-time ratio

$$\rho_{f,g;c,\text{slice}}^{\text{time}} := \frac{\theta_{fc,\text{slice}}^{\text{time}}}{\theta_{gc,\text{slice}}^{\text{time}}}.$$

Here, $\rho_{f,g;c,\text{slice}}^{\text{time}} < 1$ favors f because the numerator has lower **GM** latency.

For full scans, let B_{fc}^{dl} denote the session-conditional downloaded GiB per timed repetition, obtained from `BytesDownloadedSum/n` (Appendix A.1.4). The **GM** throughput estimand is

$$\theta_{fc}^{\text{thr}} := \frac{B_{fc}^{\text{dl}}}{\theta_{fc,\text{full}}^{\text{time}}},$$

and for two formats f and g at fixed codec c the corresponding throughput ratio is

$$\rho_{f,g;c}^{\text{thr}} := \frac{\theta_{fc}^{\text{thr}}}{\theta_{gc}^{\text{thr}}}.$$

Here, $\rho_{f,g;c}^{\text{thr}} > 1$ favors f because the numerator has higher throughput. Because B_{fc}^{dl} can differ by format and codec, throughput ratios are not, in general, the inverse of **GM**-time ratios.

For RQ3, compression and request intensity are summarized by

$$R_{fc} := \frac{B_{\text{raw}}}{B_{fc}^{\text{stored}}}, \quad \lambda_{fcp} := \frac{G_{fcp} 2^{30}}{B_{fcp}^{\text{dl}}},$$

where higher R_{fc} means better compression and lower λ_{fcp} means fewer GETs per GiB. When useful, we form descriptive codec-wise or format-wise ratios from these quantities. H1.3, H2.3, and H3.2 are association/mechanism statements rather than single pairwise contrasts. H3.1 also contains a “similar compression” claim. These hypotheses are therefore interpreted descriptively rather than as formal equivalence or null-hypothesis tests.

The clearest pairwise translations are therefore:

- H1.1 (*per-frame layouts faster than **ROOT***): for fixed codec c ,

$$\rho_{f,\text{ROOT};c,\text{slice}}^{\text{time}} < 1,$$

where $f \in \{\text{HDF5/HSDS}, \text{Zarr}, \text{TileDB}\}$.

- H1.2 (*Zarr fastest for light codecs*): for $c \in \{\text{lz4}, \text{zstd}\}$,

$$\rho_{\text{Zarr},f;c,\text{slice}}^{\text{time}} < 1,$$

where $f \in \{\text{HDF5/HSDS}, \text{TileDB}, \text{ROOT}\}$.

- H2.1 (*TileDB highest throughput*): for fixed codec c ,

$$\rho_{\text{TileDB},f;c}^{\text{thr}} > 1,$$

where $f \in \{\text{Zarr}, \text{HDF5/HSDS}, \text{ROOT}\}$.

- H2.2 (*LZ4 and ZSTD outperform gzip on scans*): for fixed format f and $c \in \{\text{lz4}, \text{zstd}\}$,

$$\theta_{fc}^{\text{thr}} \geq \theta_{f,\text{gzip}}^{\text{thr}}.$$

H1.3, H2.3, H3.1, and H3.2 are evaluated from the joint descriptive patterns in the timing, request-intensity, and storage summaries rather than from a single null/alternative test.

3.1.5.1 Evaluation procedure (*estimation*)

Rather than perform classical null-hypothesis significance tests for many contrasts, we adopt an *estimation* approach:

1. Work on the log scale $Y_{fcp,r} = \ln T_{fcp,r}$ and estimate μ_{fcp}^{time} and its uncertainty using a moving-block bootstrap (MBB) that preserves within-session serial dependence (Appendix A.1.3).
2. Back-transform to obtain $\theta_{fcp}^{\text{time}}$ and dependence-robust 95% confidence intervals.
3. For full scans, transform GM time to GM throughput using the session-specific B_{fc}^{dl} and propagate the same bootstrap uncertainty through this monotone transformation.
4. Form descriptive time- or throughput-ratio summaries from the corresponding bootstrap draws; request intensity and compression are summarized from session aggregates and inventory totals, with exact count-based intervals reported for GET rates where shown.

We interpret H1.1–H3.2 by asking whether the observed point estimates and intervals fall on the expected side of the relevant reference value or preserve the expected ordering. We do not aggregate these comparisons into a single global decision rule, and we do not control a family-wise or false-discovery rate across all contrasts.

3.1.5.2 Why bootstrap, and why not *t*-tests or **ANOVA**?

As discussed in Step 2, run-level timings are right-skewed and can exhibit within-session serial dependence due to cache and cloud-variability effects, making independent and identically distributed assumptions fragile. Serial dependence and non-normality in the run-level timing data (Section 5.1 and Appendix Table A.2) violate the independence and homoscedastic normal-error assumptions underlying standard *t*-tests and **ANOVA**. A parametric linear model on $\ln(\text{time})$ could, in principle, be equipped with a dependence structure, but this would require additional modelling choices and diagnostics that are not central to the research questions. The moving-block bootstrap on $\ln(\text{time})$ was chosen because:

- it respects the observed lag-1 autocorrelation and within-session serial dependence by resampling blocks rather than individual runs;
- it makes no parametric assumption on the marginal distribution beyond approximate stationarity within the timed window; we diagnose run-order drift separately and treat intervals as descriptive when drift is present;
- it yields confidence intervals for both cell-wise time estimands ($\theta_{fcp}^{\text{time}}$, p_{95}) and derived quantities such as time ratios and transformed throughput summaries in a uniform way.

Alternative procedures considered but not used include:

- independent-sample *t*-tests or **ANOVA** on raw or log times (discarded due to serial dependence and skew);
- per-cell normal-theory intervals on $\ln(\text{time})$ (discarded because they rely on approximate normality of the sample mean under dependence);
- global regression models with format and codec as factors (discarded because formats are executed in separate sessions, so cross-format coefficients would still be session-conditional and require strong assumptions about between-session comparability).

In summary, the formal contrast language in this thesis applies to **GM** time and **GM** throughput comparisons, while the mechanism-oriented hypotheses involving GETs/GiB and compression are evaluated from descriptive joint patterns. No formal multiple-comparison testing or causal inference is

attempted, and all cross-format statements remain descriptive and session-conditional.

3.2 Experimental Testbed and Protocols

We evaluate four timed cloud-native storage formats (**HDF5/HSDS**, Zarr, TileDB, and **ROOT/TTree**) on **AWS S3** using a single ~ 20 GiB detector dataset; Parquet is included only as a single-object pilot (ingest only, no timed runs; Appendix A.1.9). The hardware, region, libraries, ingestion, and run protocol are identical across RQ1–RQ3, with only the workload unit (slice vs full scan) differing, as defined in 3.1.3.1–3.1.3.2.

3.2.1 Test Environment and Infrastructure Setup

The experiment runs on Amazon Web Services (**AWS**). Storage backends use Amazon **S3**, and Amazon **EC2** instances perform data processing and benchmarking (Figure 3.2).

AWS benchmarking Topology

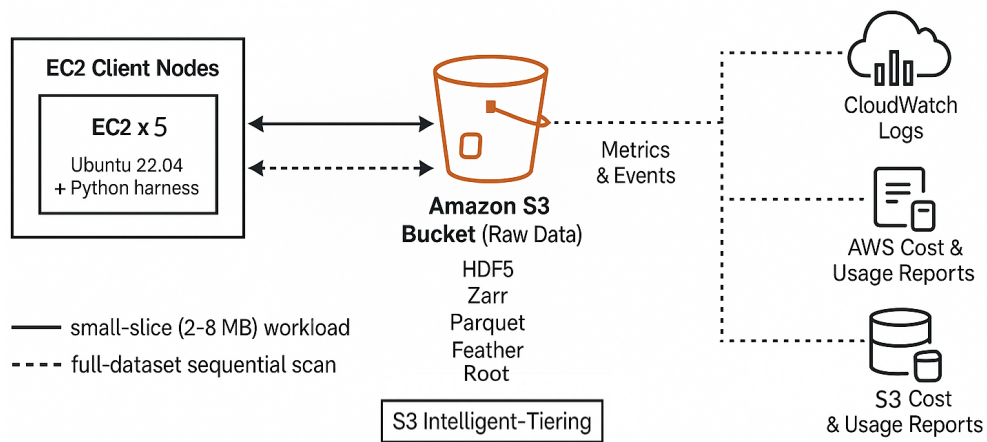


Figure 3.2: **AWS** benchmarking topology. A single **EC2** client issues small-slice requests (~ 15.82 MiB) and full-dataset scans to one Amazon **S3** bucket that stores four format prefixes (**HDF5/HSDS**, Zarr, TileDB, **ROOT/TTree**). CloudWatch metrics collect request counts, bytes, and service latencies.

All runs are executed in a single **AWS** region with the **S3** bucket co-located. No CloudFront, **S3** Transfer Acceleration, or cross-region traffic was used. Full region and network settings are listed in Appendix A.1.1.

3.2.2 Data Ingestion and Configuration

A single detector dataset of ~ 20 GiB ($N=1296 \approx 1,300$ frames) was used, derived from 80 real **Crystallographic Binary File (CBF)** seed frames and synthetically expanded (Appendix A.1.8). The 20 GiB figure refers to the uncompressed pixel payload of the logical array, while stored bytes on **S3** (data+metadata) are reported separately (Appendix Table A.5, Appendix Table A.1). For each timed storage format, three datasets were created, one for each codec (`gzip`, `lz4`, and `zstd`); the Parquet pilot used `gzip` and `lz4` only (Appendix A.1.9). Format-specific schemas are in Appendix A.1.7.

3.2.3 Experimental Parameters and Workloads

The experiment compares the four storage formats under two access patterns: small-slice reads and full-dataset sequential scans, reflecting typical beamline workflows. Runs were executed in sessions, one cell (format $\times$ codec $\times$ pattern) at a time, with within-session run order randomized using a fixed seed.

Small-slice reads. In this pattern, each run performs a random single-frame read; each frame averages ~ 15.82 MiB. This pattern represents interactive retrieval for exploratory inspection and later analysis.

Full-dataset sequential scans. In this pattern, each run performs a sequential read of the entire N -frame dataset for each format-codec combination. This pattern stresses storage bandwidth and tests high-throughput processing of full-dataset reads.

3.3 Metrics, Statistics, and Scoring

This section defines how measurements are summarized, how uncertainty is reported, and how the decision-matrix scores are computed. The log-time and bootstrap machinery applies directly to the timing estimands in RQ1–RQ2, while RQ3 combines those timing summaries with descriptive request and storage metrics as specified in 3.1.3.

3.3.1 Performance Metrics and Evaluation Criteria

We evaluate each format along four dimensions. **Latency** is reported for both slice and full-scan workloads as geometric-mean time with dependence-robust 95% **CI**s on the log scale, together with the 95th percentile and its interval (Appendix A.1.3). For full scans, **GM** scan time is the clearest

same-logical-dataset completion metric across formats. **Throughput** for full scans is additionally reported as geometric-mean GiB/s derived from transferred/downloaded bytes divided by time and analyzed on the log scale with **MBB** 95% **CI**s. **Request intensity** is measured as GETs per repetition and GETs per GiB using per-session **S3** aggregates (Appendix **A.1.4**). **Compression** is summarized through total stored bytes on **S3** and compression ratio relative to the raw baseline.

Cross-format contrast views are reported descriptively. For slice latency, the primary effect-size view is the **GM** time ratio within each codec family. For full scans, **GM** scan time is the primary same-logical-dataset comparison, while **GM** throughput is reported as an operational transferred-byte throughput summary; **GM** time ratios are retained as a supporting appendix view within each codec family.

3.3.2 Decision-matrix scoring (normalization and weights)

For downstream selection, we compute normalized 0–100 scores per codec×pattern using robust min–max scaling within that family (lower bound = 5th percentile, upper bound = 95th percentile, clipped).

For slice workloads, the score weights **GM** latency at 0.60, p_{95} at 0.20, GETs/GiB at 0.10, and stored GiB at 0.10, where lower values are preferred throughout. For full scans, throughput carries weight 0.60, GETs/GiB and stored GiB each carry 0.15, and p_{95} carries 0.10, with higher throughput preferred and lower values preferred for the remaining criteria.

The weights are heuristic and reflect the intended decision priority of each workload: user-facing latency dominates slice reads, while transferred-byte throughput dominates full scans. Request intensity and stored size are retained as secondary cost proxies rather than primary performance objectives.

League tables are ranked by the primary objective (slice: **GM** latency; full: throughput), with tie-breakers matching those in the Results section.

3.3.3 Statistical methods

Timings are positive and multiplicative, so we analyze $\ln(\text{time})$ per cell (format×codec×pattern). The primary estimands are geometric-mean (**GM**) time, reported with 95% intervals on the log scale and back-transformed; tail latency (p_{95}), reported with 95% intervals on the original scale; and full-scan throughput in GiB/s, computed from transferred/downloaded bytes divided by time and analyzed on the log scale with the same interval procedure. For same-

logical-dataset full-scan comparisons, **GM** scan time is the clearest completion metric. Bootstrap and sensitivity settings are given in Appendix A.1.3.

Here, *dependence-robust* means that intervals are computed by resampling contiguous blocks of runs rather than individual runs, so the calculation preserves short-range within-session serial dependence. The diagnostic labels used later have the following meanings. *Shapiro p* is the Shapiro–Wilk normality-test *p*-value on $\ln(\text{time})$ and is used only as a shape diagnostic. *Lag-1 autocorrelation* measures similarity between adjacent timed repetitions. *Spearman drift* is the Spearman rank association between run order and $\ln(\text{time})$, used as a heuristic check for monotonic warm-up or drift within a session. The *Harrell–Davis* p_{95} is a smooth quantile estimator used only in the tail-sensitivity analysis, not as the primary main-table percentile.

Within each $\text{codec} \times \text{pattern}$, the appendix ratio plots are reported as **GM** time *ratios* versus Zarr and are **descriptive, session-conditional**. For full scans, the main-text performance comparisons are based on **GM** scan time, with **GM** throughput reported alongside it as an operational transferred-byte summary and the **GM**-time ratio plots used as a supporting view. Diagnostics include Q–Q plots on $\ln(\text{time})$, lag-1 autocorrelation, and a run-order trend test (Appendix A.1.2).

Per-session **S3** aggregates are used descriptively only; identifiers and statistics are listed in Appendix A.1.4. We do not average percentiles across sessions or attach inferential intervals to service metrics. Warm-up and replication policy are defined in Appendix A.1.2.

For all between-format comparisons, we treat *ratios of geometric means* as the primary effect size for time-based outcomes. For latency, this is the ratio of **GM** times between two formats on the original scale (for example, $\text{GM}_{\text{ROOT}}/\text{GM}_{\text{Zarr}}$), where values below 1 indicate that the numerator is faster. Because we analyse $\ln(\text{time})$, these ratios correspond to differences of means on the log scale. Empirically, response-time distributions in computer systems are right-skewed and often close to lognormal, so working on the log scale and reporting geometric means is standard in performance evaluation. For full scans, **GM** scan time is the clearest same-logical-dataset completion estimand. We additionally report **GM** throughput in GiB/s, obtained by dividing session-specific downloaded GiB by **GM** time; this should be read as a transferred-byte throughput summary of the stored representation. Because downloaded bytes can differ across formats, descriptive **GM**-time ratios and **GM**-throughput contrasts need not coincide; the appendix ratio plots therefore show **GM** time ratios versus Zarr, while throughput values are reported directly in the tables. Nonparametric effect sizes are reported as *median ratios* in the robustness

tables.

For pairwise time-ratio contrasts within a codec $\times$ pattern family, the descriptive reference value is 1. Rather than performing formal tests, we present 95% confidence intervals for the ratio and interpret them descriptively.

3.3.4 Uncertainty in AWS

AWS introduces time-varying latency and variance that cannot be fully controlled. We mitigated this issue by using a single region with the **S3** bucket co-located, a Gateway VPC endpoint, and fixed client settings. Operational controls and randomization are documented in Appendix A.1.2. Formats were executed in *separate sessions* with no cross-format overlap. Consequently, cross-format contrasts are **descriptive, session-conditional**. Inference is limited to within-cell estimands (**GM**, **95th percentile (P95)**, throughput) with dependence-robust bootstrap **CI**s. CloudWatch aggregates are per-session and reported descriptively (no inference, no cross-session averaging of percentiles).

Parquet was piloted in a single-object layout but excluded from the timed results due to high metadata open time and rapid object growth observed in the pilot runs. This pilot documents one concrete mapping of an n -D detector stack into a single Parquet object and should not be interpreted as a general performance statement about Parquet as a partitioned format family (Appendix A.1.9). The pilot and rationale are documented in Chapter 4, where Parquet is presented only qualitatively. Timed comparisons cover four formats: **HDF5/HSDS**, Zarr, TileDB, and **ROOT/TTree**.

3.4 Data Collection and Reproducibility

The data-collection plan for this thesis centers on a single dataset built from 80 real seed frames and synthetically expanded to ~ 20 GiB ($\sim 1,300$ frames). The resulting stack is therefore a controlled benchmark rather than a fully independent: it preserves the detector geometry, `uint16` encoding, per-frame size, and local intensity structure needed for I/O testing, but most frames are deterministic perturbations of the seed set. Generating the benchmark dataset from a small seed set reduces data sharing burden, and the synthesis procedure is deterministic given the same seeds and software versions (Appendix A.1.8). We also release the raw timing logs and CloudWatch exports used for all reported statistics (Appendix A.1.6).

Dataset facts and size definitions are summarized in Appendix Table A.1.

3.4.1 Data Sources

Detector stack (raw pixel payload ~ 20 GiB): $N=1296$ CBF frames at 16-bit depth, consisting of 80 real seed frames from one experiment and 1,216 synthetic frames. One frame constitutes one sample and contains 2880×2880 pixel features (~ 15.82 MiB uncompressed). A jitter-noise routine perturbs intensity, offset, spatial shifts, and noise to synthesize the remaining frames (Appendix A.1.8, Appendix Table A.1).

3.4.2 Sampling Strategy

Two patterns were executed on the same dataset. For the slice workload, each run reads one randomly selected frame (~ 15.82 MiB) and each backend uses its native physical granularity, so a frame maps to different numbers of objects across formats. For the scanning workload, the entire N -frame dataset is read sequentially. Within-session run order was randomized with a fixed seed, and no cross-format concurrency occurred.

3.4.3 Instrumentation and Metric Logging

Full-scan time is measured from entering the scan read routine to completion of the scan with all data materialized/decoded by the client library; slice-read time is measured from entering the single-frame read routine until it returns a fully materialized frame. These timers include network transfer, decompression, and client decode, while log serialization is outside the timer. Throughput equals bytes divided by time. Accordingly, we record client start and stop timestamps together with bytes per run, per-session S3 aggregates such as request counts, transferred bytes, and latency percentiles (identifiers and statistics in Appendix A.1.4), and S3 sizes per format-codec combination for the compression calculations.

3.4.4 Validity and Reliability Measures

No explicit cache-clearing was performed between repetitions, so the reported latency and throughput results should be interpreted as *warm, within-session* behavior rather than cold-open first-touch performance. Cache regime and replication policy are specified in Appendix A.1.2.

3.4.5 Reproducibility and availability

Random seeds are fixed and logged with per-run timers, and CloudWatch aggregates are saved as **JSON** with run IDs. The code and notebooks are released on GitHub at <https://github.com/CasperKristiansson/KTH-MSc-Thesis-Computer-Science>. The repository includes (i) seed frames under `Experiment/Base Data/` and the synthesis procedure (Appendix A.1.8), and (ii) the raw measurement logs used to generate all reported tables and figures (`Experiment/Experiment Result/`). The full ingested **S3** dataset prefixes and the **AWS** execution environment are not publicly hosted; rerunning timed experiments therefore requires re-provisioning comparable cloud resources and will reproduce the *workflow* and analysis, not the exact wall-clock timings. Exact package versions and access paths are listed in Appendix A.1.5.

Chapter 4

What you did

This chapter documents the concrete experimental artifacts and execution environment used to implement the methodology defined in Chapter 3. It specifies the dataset and augmentation strategy, the **AWS** environment and client software, how each backend (**HDF5/HSDS**, Zarr, TileDB, **ROOT/TTree**, Parquet) was ingested on **S3**, the monitoring and logging setup, and deviations from the plan, including the pilot-only evaluation of Parquet. The emphasis is on reproducibility: enough implementation detail is provided to rebuild the benchmark pipeline and interpret the reported results.

4.1 Dataset and preprocessing

All experiments used a single detector stack derived from 80 real CBF frames and expanded to a raw pixel payload of ~ 20 GiB ($N=1296 \approx 1,300$ frames; Appendix Table A.1).

4.1.1 Raw structure

Each detector exposure is a dense two-dimensional frame. The specifications below fix the frame geometry, axis conventions, numeric encoding, byte size, and the theoretical versus empirically observed value range used for all subsequent ingestion and synthetic data generation.

- Array shape per frame: (2880, 2880).
- Axes: slow = rows, fast = columns (units: pixels).
- dtype: uint16 (bit depth 16).

- Endianness: native little-endian.
- Per-frame size: 16 588 800 B (15.82 MiB).
- Features per frame: $2880 \times 2880 = 8,294,400$ pixel intensities.
- Seed container: **CBF** 1.5 with `x-CBF_BYTE_OFFSET` compression (header convention `PILATUS_1.2`); decoded into `uint16` arrays before synthesis and ingestion.
- Value range (theoretical): $[0, 65535]$.
- Value range (observed, raw): $[0, 1199]$.

4.1.2 Preprocessing and synthetic augmentation

We started from 80 raw CBF detector frames. Frames were enumerated in zero-based order; the naming convention `slice_{index}.cbf` fixes the frame-index $\rightarrow$ key mapping used later during ingestion. No resampling, normalization, or pixel edits were applied before augmentation.

To scale the corpus for I/O benchmarking while preserving frame geometry and `uint16` encoding, additional frames were synthesized from the real frames using simple, distribution-preserving perturbations:

- Intensity gain: multiply by $g \in [0.85, 1.15]$.
- Local contrast scale around the frame mean: factor $c \in [0.9, 1.1]$.
- Spatial jitter: with probability 0.8, apply integer circular shifts up to ± 16 px on each axis.
- Additive noise: with probability 0.9, add zero-mean Gaussian noise with $\sigma \approx 20$ counts.
- Clip to $[0, 65535]$ and cast back to `uint16`.

Random choices used fixed Python and NumPy seeds (Appendix A.1.8). We targeted $N=1296$ frames (raw pixel payload ~ 20.02 GiB; Appendix Table A.1) by estimating average slice size and computing the required frame count. Outputs were written as **CBF** frames and then uploaded to **AWS S3** (implementation details in Appendix A.1.8).

4.2 Infrastructure and software

All experiments ran in a single **AWS** region (`eu-north-1`) on one **EC2** node, with all artifacts stored in one **S3** bucket. **HDF5** access in the timed benchmark was provided via a local **HSDS** service; Zarr, TileDB, **ROOT/TTree**, and the Parquet pilot wrote directly to **S3** via client libraries. **S3** bucket analytics and CloudWatch request metrics captured request counts, bytes, and service latencies per session.

Region and bucket. The benchmark used a single **S3** bucket in `eu-north-1`, with distinct prefixes for inputs, intermediate artifacts, and final format stores (**HDF5/HSDS**, Zarr, TileDB, **ROOT/TTree**, Parquet). The exact bucket name and prefixes are listed in Appendix [A.1.1](#).

Monitoring and metrics. Bucket analytics were enabled to track object counts and byte growth over time. In addition, CloudWatch **S3** request metrics were enabled at 1-minute resolution and exported per session (request counts, bytes, and service-side latency summaries). Metric identifiers and statistics are listed in Appendix [A.1.4](#).

HSDS service. **HDF5** access in the timed benchmark was provided by **HSDS** running via Docker on the **EC2** node, following the vendor's **AWS** instructions [87]. The service wrote to the benchmark **S3** bucket and exposed a local **HTTP** endpoint where environment variables and configuration are listed in Appendix [A.1.1](#).

Other backends. Zarr, TileDB, **ROOT/TTree**, and Parquet required no separate service. They used client libraries to write directly to **S3** with the instance role.

Client software stack. The client environment used standard Python scientific and storage libraries where package versions are listed in Appendix [A.1.5](#).

4.3 Storage layouts, codecs, and objectization

This section summarizes how the array was mapped to **S3** for each format.

4.3.1 Common design

- Logical array for all stores: $(N, 2880, 2880)$, $N=1296$, `uint16`.
- One frame per chunk/tile: $(1, 2880, 2880)$ to keep read units comparable.

- Codecs targeted: `gzip`, `lz4`, `zstd`. Parquet used only `gzip` and `lz4`.
- Frame enumeration: zero-based; source files named `slice_{index}.cbf`.
- Concurrency: threaded frame loading; batch writes by contiguous index slices.

4.3.2 HDF5 via HSDS

HSDS serves **HDF5** on the **EC2** node, where we record domain paths, dataset geometry, chunking, and codec settings. **HSDS** manages **S3** keys and metadata with one frame per chunk, preserving parity with other stores.

Layout:

- Dataset: shape $(N, 2880, 2880)$, chunks $(1, 2880, 2880)$, `uint16`.
- Compression: `gzip`, `lz4`, or `zstd` at library default levels.
- Objectization: approximately one chunk object per frame plus **HSDS** metadata/index objects.

Ingestion:

- Frames were written in batches of 16 into contiguous slices of the dataset using the **HSDS** client.
- Host-level threading parallelized frame decoding and I/O (implementation details in Appendix [A.1.7](#)).

4.3.3 Zarr on S3

Zarr uses one chunk per frame where we record store paths, array geometry, chunking parity, codecs, and the write procedure.

Layout:

- Array: shape $(N, 2880, 2880)$, chunks $(1, 2880, 2880)$, `uint16`.
- Compression: `gzip`, `zstd`, or `lz4` with moderate compression level (≈ 3).
- Metadata: default Zarr metadata files.
- Objectization: approximately one chunk object per frame plus metadata files.

Ingestion:

- Frames were written in batches of 16 as contiguous slices, with threaded frame loading.

4.3.4 Apache Parquet on S3 (single-object pilot)

Frames were tabularized as rows with 2-D rows as fixed-size lists. *Note:* Parquet was piloted only in a single-object layout and is excluded from timed results due to an operational stop; see Appendix A.1.9.

Layout:

- Files: one Parquet file per codec under a dedicated prefix.
- Schema: 2880 columns `row_0...row_2879`, each `list<fixed_size=2880, uint16>`.
- Row groups: one appended row per frame (writer defaults).
- Compression: `gzip` or `lz4`; dictionaries disabled.
- Objectization: single Parquet object per codec.

Ingestion:

- For each frame, per-row arrays were assembled into a one-row table and appended to the Parquet file (Appendix A.1.7).

Parquet was implemented as a single file with 2880 list-of-2880 `uint16` columns (`row_0...row_2879`), one appended row per frame, using `gzip` and `lz4` compression with dictionaries disabled. In the pilot, this layout produced a single Parquet object of approximately 34 GiB for a raw stack of approximately 20 GiB. Opening the file on the EC2 test node required approximately 3 min to 4 min to load the schema, footer, and column indexes, and repeated attempts to open the `gzip` file led to instance crashes. Although one frame could be decoded after the metadata load in an initial smoke test, these behaviors made it impractical to proceed with timed runs. The benchmark was therefore stopped after the initial write and smoke read. The rationale was that the metadata open time and object size made a full benchmark infeasible within the experiment budget. Consequently, Parquet was excluded from the timed latency and throughput results, while the pilot artifacts were retained for transparency and are discussed qualitatively in Chapter 6.

4.3.5 TileDB dense array

TileDB stores a dense array with frame-sized tiles and immutable fragments; this records the domain, filters, and fragment layout.

- URIs: one TileDB array per codec under a dedicated prefix.
- Domain dims: `file: [0, N-1], y: [0, 2879], x: [0, 2879]`.
- Tile extents: (1, 2880, 2880) (full-frame tiles).

Filters:

- `gzip (level=3)`, `lz4 (level=3)`, `zstd (level=3)` as attribute filter lists.

Fragments and objectization:

- Writes executed in batches of 16 frames per task; each task produced a fragment.
- Expected fragments: $\lceil N/16 \rceil$.
- Each fragment contributes internal files (data, metadata, footers) under the array URI.

4.3.6 ROOT/TTree

ROOT/TTree encodes all frames into a single file per codec; this notes the tree layout, compression, and access pattern.

Tree layout:

- Files: one **ROOT/TTree** file per codec under a dedicated prefix.
- TTree `img_tree` with one branch data of length 2880×2880 `uint16`; one entry per frame.

Compression:

- `gzip`: algorithm `kZLIB`, level 3; `lz4`: `kLZ4` (default level); `zstd`: `kZSTD` (default level).

Objectization:

- Single **ROOT/TTree** object per codec. Selective reads use **HTTP/S3** range-GETs into baskets/pages.

4.3.7 RQ links from the layout choices

- **RQ1 (interactive slices):** Frame-aligned chunks/tiles (1, 2880, 2880) to match the slice unit.
- **RQ2 (full scans):** Fragmenting and page/cluster settings that enable long sequential reads.
- **RQ3 (cost posture):** Object counts per backend, metadata consolidation status (e.g., Zarr consolidated metadata; TileDB fragment consolidation), and stored bytes per codec.

4.4 Execution, logging, and quality control

This section summarizes the run protocol for each {format×codec×pattern} cell, along with the associated logging and quality-control checks.

4.4.1 Execution protocol

All reads ran on a single EC2 node in eu-north-1. For each *format* × *codec* × *pattern*, we used a fixed warm-up followed by timed runs, with no cross-format concurrency (Appendix A.1.2).

Each *session* targeted one cell (format, codec, pattern) and consisted of a small warm-up phase, an idle gap, and a block of timed repetitions where exact counts and timings are detailed in Appendix A.1.2. Between sessions, formats did not overlap; HSDS, Zarr, TileDB, and ROOT/TTree were run in separate sessions. A fixed seed ensured reproducibility: slice workloads used precomputed random frame indices, whereas full scans iterated over all frames (sampling details in Appendix A.1.2).

Per run, we measured wall-clock time using a high-resolution timer around a single read call. The timer includes network transfer, decompression, and client decode, while log serialization is outside the timer. S3 bucket request metrics were enabled and exported per session for request counts, bytes, and service-side latency summaries. Metric identifiers/statistics and the export schema are listed in Appendix A.1.4; timing windows, separation from warm-ups, inclusion and retry rules, cache-control assumptions, and repetition policy are specified in Appendix A.1.2. Each session emitted a monotonic `run_id`, and timed samples and CloudWatch aggregates were stored as JSON with format/codec/pattern identifiers (log schema in Appendix A.1.4).

4.4.2 Quality control, exclusions, and deviations

Runs followed the fixed warm-up and timing protocol where integrity checks verified shapes and dtypes, and only successful timed reads were included. Exclusion and retry rules are specified in Appendix A.1.2. The following quality control checks were performed:

- Protocol details (warm-ups, repetitions, timing windows, isolation) are specified in Appendix A.1.2.
- Integrity checks: spot decode of returned arrays; shape = (2880, 2880), dtype `uint16`.
- Logs: per-cell client timing logs and CloudWatch exports saved as **JSON** (Appendix A.1.4).

Deviations from plan: Parquet was pilot-only because writing and the smoke-test read revealed impractical metadata open times for the single-file layout and a single huge object; Parquet was omitted from timed results and kept for qualitative discussion (Appendix A.1.9).

Chapter 5

Results and Analysis

This chapter reports the empirical results of benchmarking four storage backends on Amazon **S3**, **HDFS** via **HSDS**, Zarr, TileDB, and **ROOT/TTree** under two workload patterns on a single ~ 20 GiB detector stack ($\sim 1,300$ frames, `uint16`). Parquet is included as a pilot-only storage engine (no timed results). Each timed backend is evaluated with three codecs (`gzip`, `lz4`, `zstd`). We present outcomes for (i) partial retrieval of a single frame (~ 15.82 MiB), and (ii) full sequential scans of the dataset.

Metrics include client-observed timing, service-side behavior, and storage footprint, with timings analyzed on a log scale. For each `format $\times$ codec $\times$ pattern` cell, we report geometric-mean (**GM**) time with dependence-robust 95% **CIs** via a moving-block bootstrap (**MBB**; settings in Appendix **A.1.3**). Tail behavior is summarized by p_{95} with the same procedure. For full scans, **GM** scan time is the primary completion metric and transferred-byte throughput (GiB/s) is additionally reported as bytes/time. Storage footprint is reported as stored bytes and compression ratio relative to the raw array. Request intensity (GETs/rep, GETs/GiB) is derived from CloudWatch session aggregates.

Cross-format contrasts are reported as **descriptive, session-conditional** because formats run in separate sessions. Warm-ups are excluded; $n = 50$ timed repetitions per cell are analyzed. This constraint limits formal causal generalization across arbitrary clouds, datasets, and implementations; however, the large effect sizes in several comparisons and the agreement between **GM**, median, and tail-sensitivity checks make the main within-testbed rankings practically informative for similar single-client **AWS S3** deployments.

This chapter is organized by *RQI* (interactive slice latency, section **5.3.1**),

RQ2 (full-dataset scan performance, section 5.3.2), and *RQ3* (cost posture and mechanisms, section 5.2).

5.1 Inventory and data hygiene

This section verifies run coverage and summarizes timing diagnostics that motivate the log-scale, dependence-robust analysis used in the main results.

All planned timed runs completed successfully: $n = 50$ timed runs per {format $\times$ codec $\times$ pattern} cell (24 cells; 1,200 total), with $n = 0$ retries and $n = 0$ failures; warm-ups (5 per cell) were excluded by design. No **S3**, **HSDS**, or client errors were observed during execution.

Run-level times are right-skewed and exhibit within-session serial dependence, so we analyze $\ln(\text{time})$ and report dependence-robust intervals via the **MBB**. Appendix Table A.2 summarizes per-cell diagnostic flags (Shapiro p on $\ln(\text{time})$, lag-1 autocorrelation, p95/GM, Spearman drift), and Appendix Figures A.1–A.4 provide the Q–Q and drift plots. Because runs are serially dependent, these p -values are used only as heuristics rather than formally calibrated tests. Overall, the diagnostics support the log-scale, dependence-robust analysis used in the main results.

5.2 Footprint and service behavior — RQ3 context

We summarize storage footprint, objectization, and **S3** service behavior to provide context for RQ1–RQ3. We report stored bytes and compression ratios, object counts and average bytes per object, GETs per repetition and per GiB, and per-session FirstByte and TotalRequest latencies.

5.2.1 Compression and objectization outcomes

We quantify storage footprint and objectization for each format–codec combination on **S3**, reporting metadata-inclusive stored bytes, compression ratios relative to the raw array, object counts, and derived bytes per object.

5.2.1.1 **S3** object sizes and compression ratios

This section enumerates the total stored bytes on **S3** for each format, codec and reports compression ratios relative to the uncompressed array baseline. Stored

bytes are computed by summing the sizes of all objects under the format–codec prefix (data and metadata files included). The compression baseline is the raw pixel payload of the logical array, not container files. Let N be the number of frames. With frame shape (2880, 2880) and 16-bit depth, the baseline size is

$$R_{f,c} := \frac{B_{\text{raw}}}{B_{f,c}^{\text{stored}}}, \quad B_{\text{raw}} := N \times 2880 \times 2880 \times 2\text{bytes}. \quad (5.1)$$

For $N = 1,296$, $B_{\text{raw}} = 21,499,084,800$ bytes (≈ 20.02 GiB). Figure 5.1 reports absolute stored size and Figure 5.2 shows the corresponding compression ratios.

Figure 5.1 quantifies absolute stored size per format and codec. For this detector stack, **ZSTD** and **gzip** achieve similar total bytes across most backends, while **LZ4** tends to store more data, particularly for **HDF5/HSDS**, **TileDB**, and **ROOT**; Zarr is the main exception, where **LZ4** is slightly smaller than **gzip**. For the Zarr condition, however, **lz4** was implemented through **Blosc(lz4)** in the evaluated Python stack rather than as a fully harmonized **raw-lz4** kernel matched across all backends. The Zarr exception should therefore be read as a benchmark-specific outcome of the evaluated codec pipeline, not as a general claim that **lz4** is more compact than **gzip**. This pattern is otherwise consistent with codec choice, rather than format, being the dominant determinant of space usage for this workload.

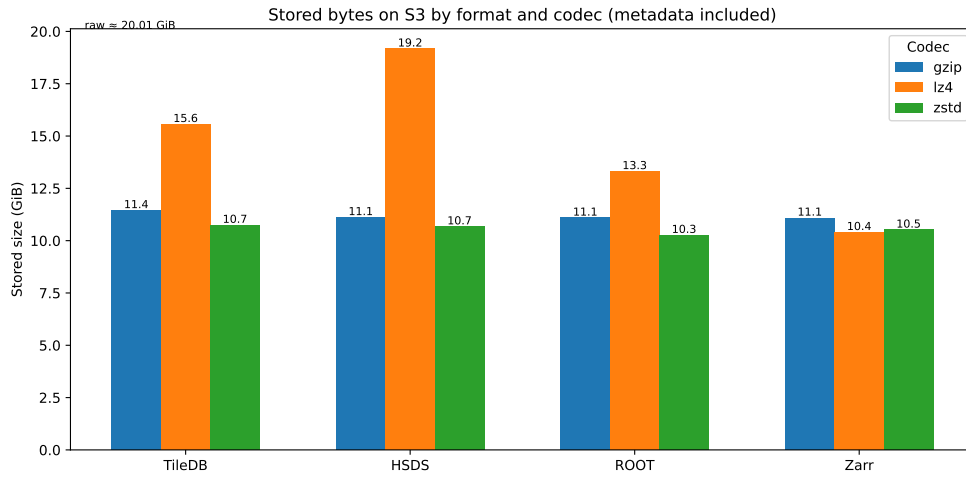


Figure 5.1: Total stored size on **S3** by format and codec. Bars sum all objects (data + metadata) under each prefix. The raw baseline is $B_{\text{raw}} \approx 20.02$ GiB ($1296 \times 2880 \times 2880 \times 2$ bytes).

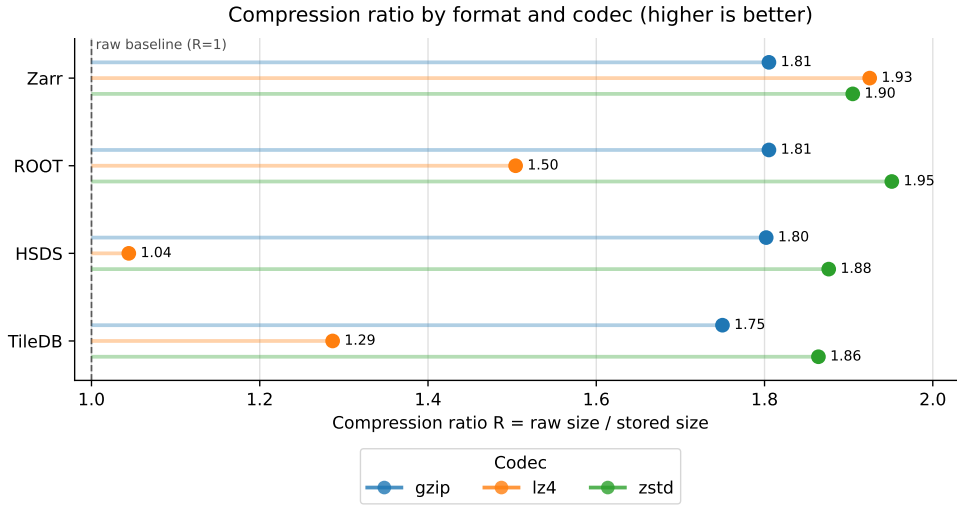


Figure 5.2: Compression ratios $R_{f,c} = B_{\text{raw}}/B_{f,c}^{\text{stored}}$ for the same format–codec totals. Higher is better. Values are grouped by format and codec; read this together with Figure 5.1, since compression ratios summarize relative compactness but do not show absolute stored GiB. Minor between-format differences at fixed codec are consistent with additional layout metadata.

Figure 5.2 shows the corresponding compression ratios relative to the raw array. Ratios cluster tightly by codec: $\text{LZ4} < \text{gzip} \lesssim \text{ZSTD}$, with only minor between-format differences at fixed codec that are consistent with additional layout metadata rather than payload changes. The takeaway is that codec selection dominates compression for this workload, and no format is significantly disadvantaged in terms of space efficiency. Apart from the **HSDS+LZ4** outlier, codec selection dominates compression for this workload; **HSDS+LZ4** should be treated as a storage-heavy implementation-specific result rather than as representative of all **LZ4** layouts.

The weak compression of **HSDS+LZ4** likely reflects the evaluated **HSDS/HDF5** filter path and metadata/layout overhead rather than a raw **LZ4** property alone. In contrast, the Zarr **LZ4** case uses the Python `Blosc(lz4)` path, so the **LZ4** results are not fully harmonized across backends.

5.2.1.2 Object counts and layout footprint

Figure 5.3 summarizes how each backend maps the logical array into **S3** objects. **HDF5/HSDS** and Zarr use chunk-per-frame layouts, yielding $O(N)$ objects plus metadata, whereas TileDB produces $\lceil N/16 \rceil$ fragments with several internal files each, and **ROOT/TTree** stores a single large object per

codec. These object counts determine whether the system favors many small requests or a few large ones, which in turn shapes request intensity and potential concurrency. Appendix Table A.5 lists the exact counts and average KiB per object.

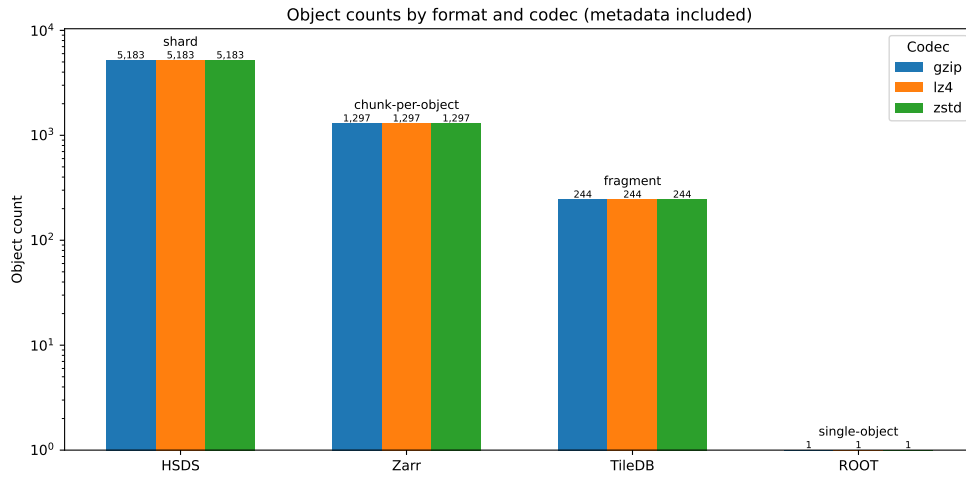


Figure 5.3: Object counts by format and codec with annotated layout type. Counts include data and metadata objects. The y-axis is logarithmic only to keep the one-object **ROOT/TTree** case and the thousand-object Zarr/HSDS cases visible in the same plot; exact counts are listed in Appendix Table A.5.

5.2.2 Request intensity and **S3** service latencies

We next summarize **S3** request behavior and service-side latencies to provide context for client timings. We report the number of GETs per session, per repetition, and per GiB. CloudWatch FirstByte/TotalRequest averages and p_{95} are considered descriptive because they are aggregated, session-conditional metrics and are reported as point estimates only. Request-count summaries are also session aggregates; where shown in Table A.3, we accompany them with exact Poisson intervals as descriptive uncertainty summaries.

5.2.2.1 GETs per repetition and per GiB

We summarize request intensity from CloudWatch *session* aggregates. All n timed runs in a cell (format×codec×pattern) contribute to one set of counts. Let G be GetRequestsSum and B be BytesDownloadedSum. Counts include **S3** GET only where HEAD/LIST/PUT are excluded. Warm-ups are excluded and separated temporally from the timed window.

Per-repetition intensity:

$$\hat{\lambda}_{\text{rep}} = \frac{G}{n}. \quad (5.2)$$

Volume-normalized intensity:

$$\hat{\lambda}_{\text{GiB}} = \frac{G 2^{30}}{B}. \quad (5.3)$$

Here n is the number of timed repetitions in the cell, and $\hat{\lambda}_{\text{GiB}}$ is measured in GETs per GiB.

Rates are descriptive and session-conditional. Table A.3 lists $\hat{\lambda}_{\text{rep}}$ and $\hat{\lambda}_{\text{GiB}}$ for all {format, codec, pattern} cells, together with exact Poisson intervals for the count-based rates. Figure 5.4 summarizes S3 request intensity by format, showing the distributions of GETs per repetition and GETs per GiB aggregated across codecs and access patterns. For this workload, Zarr and TileDB cluster toward lower GETs/GiB than HDF5/HSDS, indicating fewer requests for the same volume of data, while ROOT consistently has very low GETs/GiB because it serves from a single object. These differences are used as descriptive context for later latency and throughput patterns, together with layout and codec effects. Detailed per-cell point estimates by codec and pattern are provided in Appendix Figures A.7 and A.8.

S3 request intensity by format (aggregated across codecs and patterns)

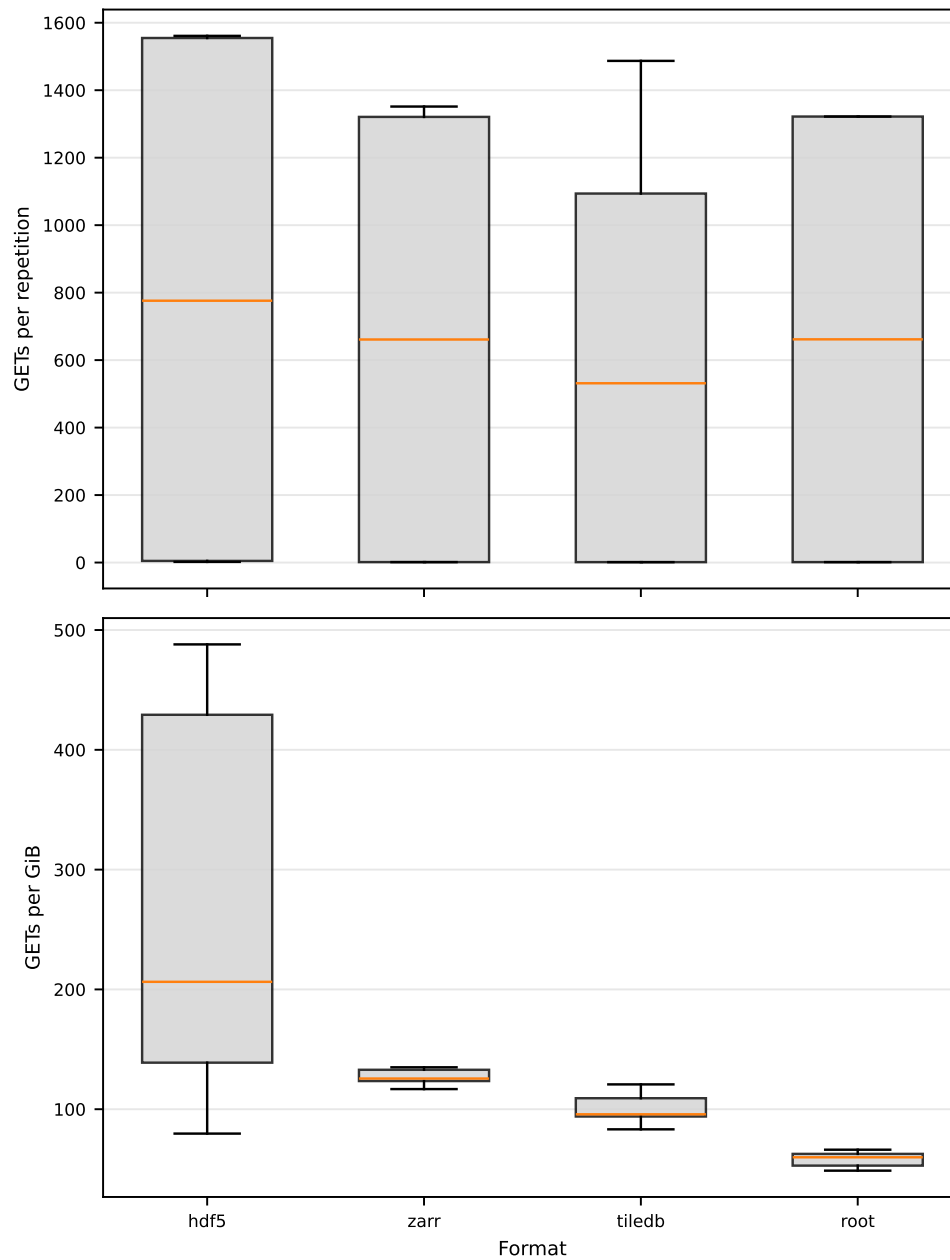


Figure 5.4: **S3** request intensity by format. Top: GETs per repetition; bottom: GETs per GiB. Each box shows the distribution across codecs and access patterns for a given format.

5.2.2.2 CloudWatch FirstByte and TotalRequest latencies

We report **S3** service-side latency summaries *per session* (format×codec×pattern): *FirstByteLatency* and *TotalRequestLatency* (average and **P95**, ms). These provider-defined aggregates are used solely for descriptive purposes, with no intervals or cross-session averages computed. Values reflect **S3** service timing, not end-to-end client timers. Exact per-session values appear in Appendix **A.4**.

CloudWatch *FirstByteLatency* and *TotalRequestLatency* are stable within sessions and vary only within a narrow band across formats and codecs (Appendix Table **A.4** and Figures **A.9**, **A.10**). *FirstByteLatency* varies on the order of a few tens of milliseconds across cells, while *TotalRequestLatency* is larger (often hundreds of milliseconds for these GETs) but remains relatively stable across cells; neither tracks the much larger cross-format gaps in client timings.

5.2.3 Mechanistic links (RQ3)

This subsection links request intensity and objectization to client timings to provide a mechanism context for RQ1 (latency) and RQ2 (throughput).

5.3 Core performance results

This section reports client-observed timings for partial (single-frame) retrievals and full-dataset scans across formats and codecs. We present geometric-mean time with dependence-robust 95% confidence intervals, p_{95} latency, and for scans also throughput, with all cross-format contrasts described as session-conditional. Main result tables report the empirical p_{95} ; Harrell–Davis p_{95} is used only in the tail-sensitivity subsection as a robustness check.

5.3.1 RQ1 — Interactive slice latency

This section quantifies interactive single-frame latency for each {format, codec} cell. We focus on geometric-mean (**GM**) latency and p_{95} with dependence-robust 95% **CI**s (**MBB** settings in Appendix **A.1.3**). All contrasts are descriptive and session-conditional. Table **5.1** gives a reader-oriented overview before the codec-specific breakdown. It uses rounded values to show the main ranking and interpretation, while Tables **5.2–5.4** retain the full per-codec estimates and confidence intervals. A codec-level summary across

formats is provided later in Figure 5.5. Detailed dependence-robust CI and ratio plots for each codec and format appear in Appendix Figures A.11–A.19.

Table 5.1: Overview of RQ1 slice-latency results by codec. Values are rounded and intended as a reader-oriented summary; detailed GM estimates, p_{95} values, and dependence-robust confidence intervals are reported in Tables 5.2–5.4.

Codec	Fastest format	GM latency	Second format	Main gap
gzip	TileDB	195 ms	Zarr	Zarr follows at about 229 ms; ROOT/TTree is clearly slower.
lz4	Zarr	176 ms	TileDB	TileDB and HDF5/HSDS are close behind, but Zarr has heavier tails.
zstd	Zarr	155 ms	TileDB	TileDB follows at about 177 ms; ROOT/TTree is slowest.

For interactive single-frame reads, the winner depends on the codec. Zarr is fastest for lz4 and zstd, while TileDB is fastest for gzip. The differences between the leading formats are smaller than in the full-scan workload, so tail behavior and codec requirements matter more for the practical choice. Overall, Zarr+zstd remains the strongest latency-first option in this testbed.

5.3.1.1 gzip

Table 5.2 reports GM slice latency and p_{95} for each format under gzip. TileDB is fastest, with a GM latency of about 195 ms, followed by Zarr (~ 229 ms) and HDF5/HSDS (~ 241 ms), while ROOT/TTree is roughly twice as slow at ~ 406 ms. In effect-size terms (GM ratios vs. Zarr), TileDB’s GM is $\approx 0.85\times$ Zarr (about 15% lower latency), HDF5/HSDS is $\approx 1.05\times$ Zarr (about 5% higher latency), and ROOT/TTree is $\approx 1.77\times$ Zarr (about 77% higher latency). The p_{95} point estimates follow the same ordering (TileDB < Zarr < HDF5/HSDS < ROOT/TTree); for TileDB and Zarr, the p_{95} CIs partially overlap, so the tail difference between those two should be read as descriptive, but both lie well below HDF5/HSDS and ROOT/TTree. Overall, under gzip, chunked (Zarr) and fragment-based (TileDB) layouts yield substantially lower slice latency and lower tail latency than the single-object ROOT/TTree layout in this dataset. The full dependence-robust CI

curves and **GM** ratio plots for this codec are shown in Appendix Figures A.11–A.13.

Table 5.2: Slice latency, `gzip`: **GM** and p_{95} (ms) with dependence-robust 95% CIs (**MBB**; $\ell \in \{4, 6, 8\}$; $B=10,000$).

Format	n	GM (ms) [95% CI]	p_{95} (ms) [95% CI]
TileDB	50	195.33 [181.42, 209.74]	262.24 [231.98, 318.13]
Zarr	50	229.30 [222.72, 236.39]	282.96 [262.02, 313.81]
HDF5/HSDS	50	241.18 [231.63, 251.94]	341.74 [295.08, 364.32]
ROOT/TTree	50	406.29 [396.64, 416.04]	471.31 [450.79, 501.10]

5.3.1.2 LZ4

For `lz4/slice`, **GM** latencies and p_{95} are given in Table 5.3. Zarr is fastest at about 176 ms, with TileDB (~ 214 ms) and **HDF5/HSDS** (~ 222 ms) following closely behind, and **ROOT/TTree** again the slowest at ~ 328 ms. Relative to Zarr, TileDB’s **GM** latency is $\approx 1.22\times$ (about 22% slower), **HDF5/HSDS** is $\approx 1.26\times$ (about 26% slower), and **ROOT/TTree** is $\approx 1.87\times$ (about 87% slower). A notable difference from `gzip` is that Zarr’s p_{95} is higher and its p_{95} /**GM** ratio is larger, reflecting heavier tails under `lz4`; the p_{95} CIs for Zarr and TileDB are both wide and overlap. The relatively wide p_{95} CIs mean that tail contrasts between Zarr and TileDB/**HDF5/HSDS** should be interpreted cautiously. The more stable signal is the **GM** ranking, which remains $\text{Zarr} < \text{TileDB} \approx \text{HDF5/HSDS} \ll \text{ROOT/TTree}$. Appendix Figures A.14–A.16 show the full **CI** and ratio behavior.

Table 5.3: Slice latency, `lz4`: **GM** and p_{95} (ms) with dependence-robust 95% CIs (**MBB**; $\ell \in \{4, 6, 8\}$; $B=10,000$).

Format	n	GM (ms) [95% CI]	p_{95} (ms) [95% CI]
Zarr	50	175.84 [158.86, 194.77]	349.19 [280.68, 414.36]
TileDB	50	214.27 [203.42, 227.45]	274.37 [260.99, 402.94]
HDF5/HSDS	50	222.35 [214.94, 229.53]	273.11 [250.50, 280.97]
ROOT/TTree	50	328.31 [314.05, 343.65]	380.41 [368.60, 429.22]

5.3.1.3 ZSTD

For `ZSTD/slice`, Table 5.4 shows that Zarr is fastest, with a **GM** latency of about 155 ms. TileDB (~ 177 ms) and **HDF5/HSDS** (~ 210 ms) are slower

but still clearly separated from **ROOT/TTree** (~ 269 ms). The corresponding effect sizes vs. Zarr are $\approx 1.14\times$ for TileDB (about 14% slower), $\approx 1.35\times$ for **HDF5/HSDS** (about 35% slower), and $\approx 1.74\times$ for **ROOT/TTree** (about 74% slower). The p_{95} point estimates display the same ordering, and differences are large in magnitude, especially between **ROOT/TTree** and the other formats. CIs for Zarr, TileDB, and **HDF5/HSDS** overlap to some extent, so tail contrasts among those three are descriptive rather than sharp, but all three have markedly lower tails than **ROOT/TTree**. Detailed **MBB CI** curves and **GM** ratio plots for **ZSTD/slice** are provided in Appendix Figures A.17–A.19.

Table 5.4: Slice latency, **zstd**: **GM** and p_{95} (ms) with dependence-robust 95% CIs (**MBB**; $\ell \in \{4, 6, 8\}$; $B=10,000$).

Format	n	GM (ms) [95% CI]	p_{95} (ms) [95% CI]
Zarr	50	155.09 [149.19, 161.91]	194.77 [176.71, 242.63]
TileDB	50	176.83 [164.98, 189.88]	249.79 [205.50, 312.41]
HDF5/HSDS	50	210.00 [205.12, 215.20]	254.79 [235.60, 286.18]
ROOT/TTree	50	269.12 [263.83, 274.91]	311.12 [291.81, 340.58]

5.3.2 RQ2 — Full-dataset scan performance

This section reports full-scan performance primarily through **GM** scan time, with transferred-byte throughput (GiB/s) additionally computed as session-specific downloaded GiB per timed repetition divided by **GM** time and propagated through the same dependence-robust bootstrap transformation on the log scale (Appendix A.1.3); summary throughput values appear alongside request intensity in Table 5.10. Table 5.5 gives a reader-oriented overview before the codec-specific breakdown. It uses rounded values to show the main ranking and magnitude of the result, while Tables 5.6–5.8 retain the full per-codec estimates and confidence intervals.

Table 5.5: Overview of RQ2 full-scan results by codec. Values are rounded and intended as a reader-oriented summary; detailed **GM** scan times, p_{95} values, and dependence-robust confidence intervals are reported in Tables 5.6–5.8.

Codec	Fastest format	GM time	Second format	Main gap
gzip	TileDB	11.3 s	Zarr	Zarr: 35.6 s; HDF5/HSDS and ROOT/TTree are much slower.
lz4	TileDB	13.6 s	Zarr	Zarr is about $2.2\times$ slower by GM scan time.
zstd	TileDB	10.4 s	Zarr	Zarr is about $2.9\times$ slower by GM scan time.

Across all three codecs, TileDB has the lowest **GM** full-scan time, and Zarr is consistently second. The gap is much larger than in the slice-latency workload: Zarr is about $2.2\text{--}3.2\times$ slower than TileDB, depending on codec, while **HDF5/HSDS** and **ROOT/TTree** are substantially slower in every codec family. For throughput-dominated workloads, this makes TileDB the clear default among the tested backends, with `zstd` giving the fastest TileDB full-scan result in this experiment.

5.3.2.1 gzip

Table 5.6 reports **GM** full-scan latencies and p_{95} for each format under `gzip`. TileDB has both the lowest **GM** time and the highest reported **GM** throughput, at about 11.3 s and ≈ 1.04 GiB/s, followed by Zarr at ≈ 35.6 s (0.32 GiB/s). **HDF5/HSDS** and **ROOT/TTree** are substantially slower. In the supporting **GM**-time ratio view versus Zarr, TileDB’s **GM** time is $\approx 0.32\times$ Zarr, **HDF5/HSDS** is $\approx 4.4\times$ Zarr, and **ROOT/TTree** is $\approx 12.4\times$ Zarr. The p_{95} point estimates follow the same ordering. The CIs are narrow relative to the gap between TileDB and **ROOT/TTree**, suggesting that a few extreme runs do not drive the substantial throughput differences; however, all tail summaries are interpreted descriptively. These patterns are consistent with the throughput-versus-GETs/GiB view in Table 5.10 and Figure 5.7. Detailed dependence-robust **CI** and **GM**-time-ratio curves for `gzip`/full scans are provided in Appendix Figures A.30–A.32.

Table 5.6: Full-scan latency, `gzip`: **GM** and p_{95} (ms) with dependence-robust 95% CIs (**MBB**; $\ell \in \{4, 6, 8\}$; $B=10,000$).

Format	n	GM (ms) [95% CI]	p_{95} (ms) [95% CI]
TileDB	50	11 254 [10 994, 11 529]	12 419 [11 728, 12 846]
Zarr	50	35 598 [35 280, 35 926]	36 415 [35 871, 36 447]
HDF5/HSDS	50	156 966 [156 394, 157 761]	159 600 [157 119, 162 739]
ROOT/TTree	50	442 484 [434 012, 451 398]	464 301 [451 985, 468 891]

5.3.2.2 LZ4

For `lz4`/full scans, Table 5.7 shows that TileDB again achieves the highest reported **GM** throughput, with a **GM** time of about 13.6 s (1.17 GiB/s). Zarr follows at ≈ 29.5 s (0.37 GiB/s). In the supporting **GM**-time ratio view versus Zarr, TileDB’s **GM** time is $\approx 0.46\times$ Zarr, while **HDF5/HSDS** and **ROOT/TTree** are much slower at $\approx 5.1\times$ and $\approx 12.3\times$ Zarr, respectively. The p_{95} point estimates mirror this ranking, and CIs are comparatively tight for TileDB and Zarr, which supports the qualitative picture of TileDB as the highest-throughput format for this workload. These supporting **GM**-time ratios agree with the codec-level summary in Section 6.1.2. The full set of **CI** curves and **GM**-time-ratio plots for `lz4`/full scans is shown in Appendix Figures A.33–A.35.

Table 5.7: Full-scan latency, `lz4`: **GM** and p_{95} (ms) with dependence-robust 95% CIs (**MBB**; $\ell \in \{4, 6, 8\}$; $B=10,000$).

Format	n	GM (ms) [95% CI]	p_{95} (ms) [95% CI]
TileDB	50	13 602 [13 265, 13 871]	14 387 [13 977, 14 568]
Zarr	50	29 483 [29 255, 29 741]	30 280 [29 707, 30 395]
HDF5/HSDS	50	151 427 [151 201, 151 666]	152 780 [152 179, 153 033]
ROOT/TTree	50	363 490 [358 672, 367 871]	375 031 [369 041, 376 388]

5.3.2.3 ZSTD

Under **ZSTD**, Table 5.8 shows that TileDB has the lowest **GM** full-scan time and the highest reported **GM** throughput, at about 10.4 s (≈ 1.06 GiB/s). Zarr follows at ≈ 30.4 s (0.35 GiB/s), while **HDF5/HSDS** and **ROOT/TTree** have much longer scan times and much lower throughput. In the supporting **GM**-time ratio view versus Zarr, TileDB’s **GM** time is $\approx 0.34\times$, **HDF5/HSDS**

is $\approx 5.0\times$, and **ROOT/TTree** is $\approx 12.3\times$. The p_{95} statistics follow the same ordering and are numerically far apart, particularly between TileDB/Zarr and **ROOT/TTree**. For Zarr versus TileDB/**HDF5/HSDS**, the CIs overlap, so tail contrasts among those three remain descriptive, but all three exhibit substantially lower tails than **ROOT/TTree**. These results are consistent with the cross-metric view in Table 5.10 and Figure 5.7. Appendix Figures A.36–A.38 and Figures A.23 and Table 5.28 provide the corresponding **CI** and tail-sensitivity plots.

Table 5.8: Full-scan latency, **ZSTD: GM** and p_{95} (ms) with dependence-robust 95% CIs (**MBB**; $\ell \in \{4, 6, 8\}$; $B=10,000$).

Format	n	GM (ms) [95% CI]	p_{95} (ms) [95% CI]
TileDB	50	10 384 [10 123, 10 687]	11 506 [10 696, 11 771]
Zarr	50	30 365 [30 167, 30 555]	31 093 [30 714, 31 406]
HDF5/HSDS	50	151 497 [151 071, 151 996]	153 273 [152 072, 155 554]
ROOT/TTree	50	373 589 [357 497, 390 216]	437 730 [419 904, 452 208]

5.3.3 Codec comparison (summary)

To summarize the codec trade-offs for this dataset, Figure 5.5 compares gzip, **LZ4**, and **ZSTD** side-by-side for each format. The top panel reports **GM** slice latency, and the bottom panel reports transferred-byte **GM** full-scan throughput, both with dependence-robust 95% confidence intervals.

Codec comparison by format (slice latency and full-scan throughput)

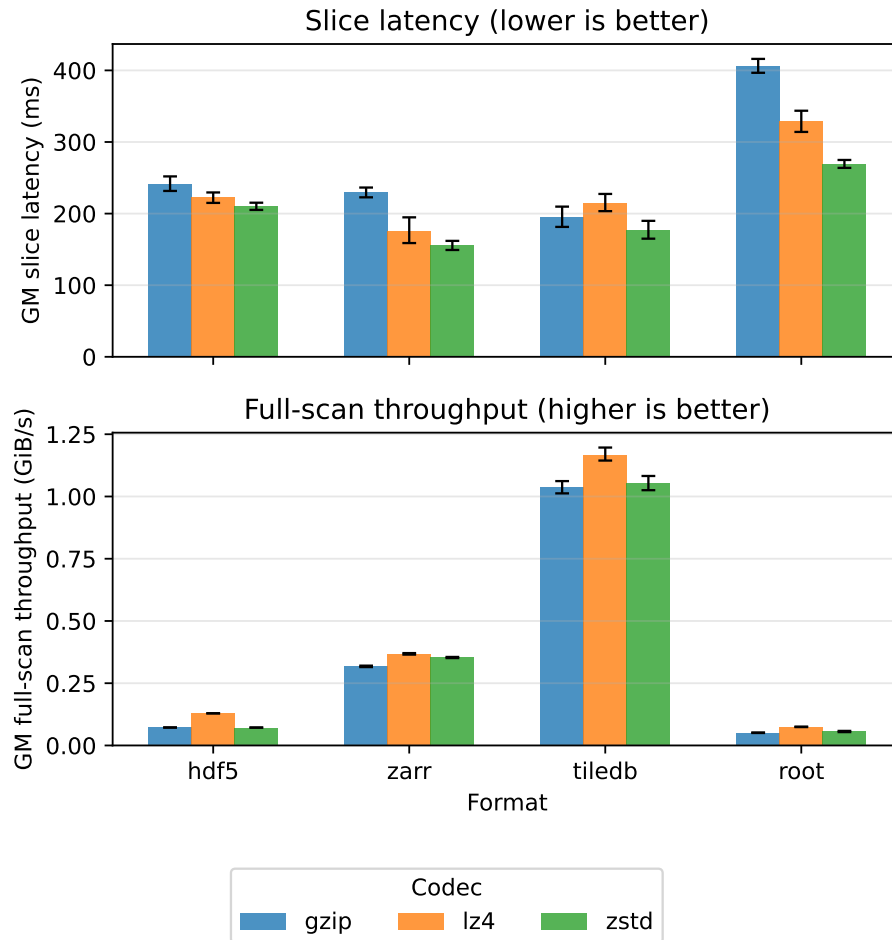


Figure 5.5: Codec comparison by format. Top: **GM** slice latency (ms), where lower is better; bottom: transferred-byte **GM** full-scan throughput (GiB/s), where higher is better. Bars are grouped by format with one bar per codec (gzip, **LZ4**, **ZSTD**); error bars show dependence-robust 95% CIs.

5.4 Synthesis, sensitivity, and decision support

This section links performance to request intensity and service latencies, then tests robustness using nonparametric centers and tail sensitivity. We finish with per-codec league tables and normalized score matrices to support

configuration choices.

5.4.1 Cross-metric views

This section links client performance to request intensity and service latencies to clarify mechanisms behind observed differences. All relationships are descriptive and session-conditional. Figures show per-session points with dependence-robust CIs.

5.4.1.1 Slice: latency vs GETs per GiB

Figure 5.6 links slice latency to request intensity on S3 by plotting GM latency against GETs/GiB for each format and codec. Table 5.9 lists the underlying values. For Zarr and TileDB, lower GETs/GiB coincide with lower GM latency, while ROOT/TTree shows low GETs/GiB but high latency due to its single-object layout and paging overhead. In this dataset, lower GETs/GiB is associated with lower GM latency for chunked stores (Zarr, TileDB), but not for ROOT/TTree, which combines low GETs/GiB with high latency. Request intensity appears to account for part of the observed differences, but layout, object size, and decode paths also vary, and the experiment is not designed to attribute causal contributions between these factors.

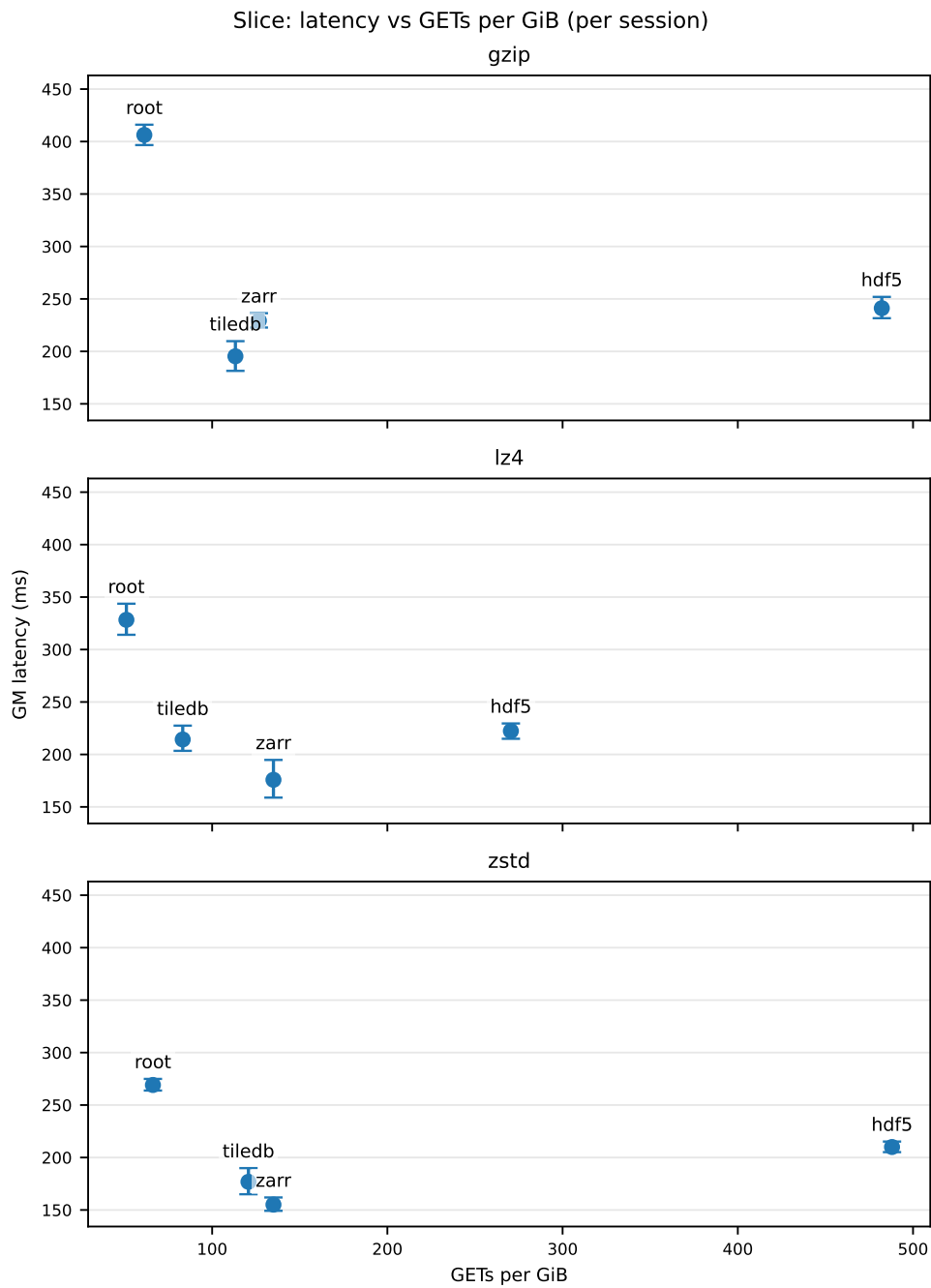


Figure 5.6: Slice: **GM** latency (ms, dependence-robust **CI**) versus GETs per GiB by format and codec. Points are per session and are used as descriptive mechanism evidence rather than as a fitted causal relationship.

Table 5.9: Slice: request intensity and latency by format and codec (descriptive).

Format	Codec	GETs/GiB	GM latency (ms)	p_{95} (ms)
HDF5/HSDS	gzip	482.2	241	342
Zarr	gzip	126.7	229	283
TileDB	gzip	113.3	195	262
ROOT/TTTree	gzip	61.3	406	471
HDF5/HSDS	LZ4	270.5	222	273
Zarr	LZ4	135.0	176	349
TileDB	LZ4	83.2	214	274
ROOT/TTTree	LZ4	51.1	328	380
HDF5/HSDS	ZSTD	488.0	210	255
Zarr	ZSTD	136.6	155	195
TileDB	ZSTD	120.7	177	250
ROOT/TTTree	ZSTD	66.2	269	311

5.4.1.2 Scan: transferred-byte throughput vs GETs per GiB

This measure relates full-scan transferred-byte throughput to request intensity. Throughput is derived from session-specific downloaded bytes and GM time; GETs/GiB are session aggregates.

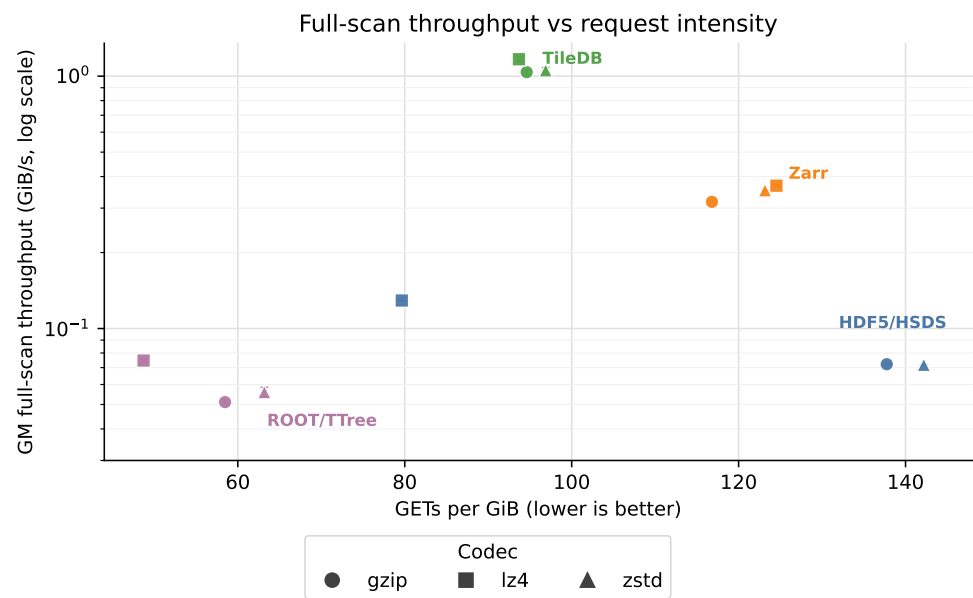


Figure 5.7: Full-scan transferred-byte **GM** throughput versus GETs per GiB. Higher throughput and lower GET intensity are preferable, so the upper-left region is best. The y-axis is logarithmic to show both high-throughput and low-throughput formats. Points are per format–codec session; vertical bars show dependence-robust 95% CIs for throughput. Lines connect codecs within the same format.

Table 5.10: Full scan: request intensity and throughput by format and codec (descriptive).

Format	Codec	GETs/GiB	GM time (s)	GM throughput (GiB/s)
TileDB	gzip	94.6	11.254	1.037
Zarr	gzip	116.8	35.598	0.318
HDF5/HSDS	gzip	137.7	156.966	0.072
ROOT/TTTree	gzip	58.5	442.484	0.051
TileDB	LZ4	93.7	13.602	1.167
Zarr	LZ4	124.5	29.483	0.368
HDF5/HSDS	LZ4	79.6	151.427	0.129
ROOT/TTTree	LZ4	48.7	363.490	0.075
TileDB	ZSTD	96.9	10.384	1.055
Zarr	ZSTD	123.2	30.365	0.353
HDF5/HSDS	ZSTD	142.2	151.497	0.072
ROOT/TTTree	ZSTD	63.2	373.589	0.056

For full scans, lower GETs/GiB is associated with higher throughput for TileDB and Zarr, while **ROOT** remains slow despite low GETs/GiB because its single-object layout and decode overhead dominate. This pattern is consistent with H2.3: request intensity likely contributes to throughput differences for chunked/fragmented stores, but is not sufficient to account for **ROOT**'s behavior, where single-object layout and decode overhead dominate in this setup.

5.4.2 Sensitivity analyses and robustness checks

We apply the robustness procedures to all $4 \times 3 \times 2 = 24$ cells (format $\times$ codec $\times$ pattern) using the moving-block bootstrap settings in Appendix A.1.3. The goal is to check whether the descriptive orderings are stable under alternative centers and tail estimators, not to perform formal hypothesis testing across formats.

- **Cells to include:** all $4 \times 3 \times 2 = 24$ cells (format $\times$ codec $\times$ pattern);
 formats = {**HDF5/HSDS**, Zarr, TileDB, **ROOT/TTree**},
 codecs = {gzip, **LZ4**, **ZSTD**},
 patterns = {slice, full}.

5.4.2.1 Nonparametric confirmation

We cross-check central tendency using the sample median with moving-block bootstrap (**MBB**) 95% **CI**s ($\ell \in \{4, 6, 8, 10\}$, $B=10,000$). Appendix Figures A.5, A.6, and Tables 5.11–5.22 show that median-based rankings match **GM**-based rankings for all codecs and patterns. Descriptive median ratios reproduce the slice and full-scan orderings, and rank-agreement tables confirm that no format changes rank under median centers. This agreement concerns the ordering of central tendencies (**GM** vs. median); it does not imply that individual **GM** or median estimates are high-precision in absolute terms, especially for more variable slice workloads such as **lz4**/Zarr and **lz4**/TileDB.

5.4.2.1.1 Slice We confirm central tendency using the sample median with moving-block bootstrap (**MBB**) 95% **CI**s ($\ell \in \{4, 6, 8, 10\}$, $B=10,000$). Rankings align with geometric means where ratios are descriptive and session-conditional (reference = Zarr).

Table 5.11: Slice / gzip: median latency with **MBB** 95% CIs and ratios vs Zarr.

Format	n	Median (ms)	95% CI (ms)	Ratio vs Zarr
TileDB	50	201.65	[179.32, 218.75]	0.880
Zarr	50	229.18	[219.95, 236.79]	1.000
HDF5/HSDS	50	234.12	[217.50, 241.29]	1.022
ROOT/TTree	50	407.67	[391.61, 418.37]	1.779

Table 5.12: Slice / gzip: rank agreement (median vs **GM**).

Format	rank_median	rank_gm	rank_diff
TileDB	1.0	1.0	0.0
Zarr	2.0	2.0	0.0
HDF5/HSDS	3.0	3.0	0.0
ROOT/TTree	4.0	4.0	0.0

Table 5.13: Slice / lz4: median latency with **MBB** 95% CIs and ratios vs Zarr.

Format	n	Median (ms)	95% CI (ms)	Ratio vs Zarr
Zarr	50	160.60	[143.33, 197.33]	1.000
TileDB	50	202.33	[193.75, 223.94]	1.260
HDF5/HSDS	50	218.70	[209.11, 229.81]	1.362
ROOT/TTree	50	332.89	[316.05, 349.75]	2.073

Table 5.14: Slice / lz4: rank agreement (median vs **GM**).

Format	rank_median	rank_gm	rank_diff
Zarr	1.0	1.0	0.0
TileDB	2.0	2.0	0.0
HDF5/HSDS	3.0	3.0	0.0
ROOT/TTree	4.0	4.0	0.0

Table 5.15: Slice / zstd: median latency with **MBB** 95% CIs and ratios vs Zarr.

Format	n	Median (ms)	95% CI (ms)	Ratio vs Zarr
Zarr	50	154.12	[143.12, 161.81]	1.000
TileDB	50	177.97	[165.49, 197.45]	1.155
HDF5/HSDS	50	205.19	[200.07, 211.67]	1.331
ROOT/TTree	50	263.98	[260.80, 271.68]	1.713

Table 5.16: Slice / zstd: rank agreement (median vs **GM**).

Format	rank_median	rank_gm	rank_diff
Zarr	1.0	1.0	0.0
TileDB	2.0	2.0	0.0
HDF5/HSDS	3.0	3.0	0.0
ROOT/TTree	4.0	4.0	0.0

5.4.2.1.2 Full Scan We confirm central tendency using the sample median with moving-block bootstrap (**MBB**) 95% **CI**s ($\ell \in \{4, 6, 8, 10\}$, $B=10,000$). Cross-format differences are descriptive and session-conditional (reference = Zarr), and rankings match geometric-mean results for all codecs.

Table 5.17: Full / gzip: median latency with **MBB** 95% CIs and ratios vs Zarr.

Format	n	Median (ms)	95% CI (ms)	Ratio vs Zarr
TileDB	50	11 187	[10 869, 11 486]	0.315
Zarr	50	35 555	[35 151, 36 057]	1.000
HDF5/HSDS	50	156 805	[156 246, 157 025]	4.410
ROOT/TTree	50	442 069	[432 257, 455 224]	12.433

Table 5.18: Full / `gzip`: rank agreement (median vs **GM**).

Format	rank_median	rank_gm	rank_diff
TileDB	1.0	1.0	0.0
Zarr	2.0	2.0	0.0
HDF5/HSDS	3.0	3.0	0.0
ROOT/TTree	4.0	4.0	0.0

Table 5.19: Full / `lz4`: median latency with **MBB** 95% CIs and ratios vs Zarr.

Format	<i>n</i>	Median (ms)	95% CI (ms)	Ratio vs Zarr
TileDB	50	13 702	[13 151, 13 894]	0.465
Zarr	50	29 461	[29 286, 29 668]	1.000
HDF5/HSDS	50	151 378	[151 023, 151 529]	5.138
ROOT/TTree	50	364 053	[359 639, 369 897]	12.357

Table 5.20: Full / `lz4`: rank agreement (median vs **GM**).

Format	rank_median	rank_gm	rank_diff
TileDB	1.0	1.0	0.0
Zarr	2.0	2.0	0.0
HDF5/HSDS	3.0	3.0	0.0
ROOT/TTree	4.0	4.0	0.0

Table 5.21: Full / `zstd`: median latency with **MBB** 95% CIs and ratios vs Zarr.

Format	<i>n</i>	Median (ms)	95% CI (ms)	Ratio vs Zarr
TileDB	50	10 283	[9 931, 10 620]	0.340
Zarr	50	30 253	[30 150, 30 461]	1.000
HDF5/HSDS	50	151 476	[151 087, 151 658]	5.007
ROOT/TTree	50	367 066	[354 750, 402 529]	12.133

Table 5.22: Full / zstd: rank agreement (median vs **GM**).

Format	rank_median	rank_gm	rank_diff
TileDB	1.0	1.0	0.0
Zarr	2.0	2.0	0.0
HDF5/HSDS	3.0	3.0	0.0
ROOT/TTree	4.0	4.0	0.0

5.4.2.2 Tail sensitivity

We assess tail sensitivity by estimating p_{95} with the Harrell–Davis quantile and **MBB** CIs across block lengths $\ell \in \{4, 6, 8, 10\}$. Appendix Figures [A.22](#), [A.23](#) and Tables [5.23–5.28](#) show that the ordering of formats is stable across ℓ for all codecs. The main exception is Zarr+**LZ4**, which exhibits heavier tails, but this does not overturn the winner hierarchy established by **GM** and median centers.

5.4.2.2.1 Slice We estimate p_{95} with the Harrell–Davis quantile and dependence-robust 95% **CI**s via a moving-block bootstrap ($\ell \in \{4, 6, 8, 10\}$, $B=10,000$). Ordering is stable across ℓ , and we report the widest **CI** per format and show ℓ –sensitivity curves.

Table 5.23: Slice / gzip: **Harrell–Davis (HD)** p_{95} with widest **MBB** 95% **CI** and selected ℓ^* .

Format	p_{95} (ms)	95% CI (ms)	CI width (ms)	ℓ^*
HDF5/HSDS	344.70	[302.25, 366.59]	64.34	8
ROOT/TTree	479.58	[453.33, 503.94]	50.61	10
TileDB	274.99	[239.68, 330.97]	91.30	6
Zarr	293.99	[266.30, 314.06]	47.76	10

Table 5.24: Slice / lz4: **HD** p_{95} with widest **MBB** 95% **CI** and selected ℓ^* .

Format	p_{95} (ms)	95% CI (ms)	CI width (ms)	ℓ^*
HDF5/HSDS	274.94	[256.95, 280.35]	23.40	4
ROOT/TTree	393.55	[373.21, 439.37]	66.16	4
TileDB	321.57	[261.70, 407.99]	146.29	10
Zarr	370.84	[298.23, 404.56]	106.33	10

Table 5.25: Slice / zstd: HD p_{95} with widest MBB 95% CI and selected ℓ^* .

Format	p_{95} (ms)	95% CI (ms)	CI width (ms)	ℓ^*
HDF5/HSDS	264.69	[242.89, 288.24]	45.35	4
ROOT/TTree	321.45	[298.81, 339.12]	40.31	4
TileDB	274.79	[227.79, 307.66]	79.87	4
Zarr	206.34	[178.68, 253.86]	75.18	8

5.4.2.2.2 Full Scan We estimate p_{95} with the Harrell–Davis quantile and dependence-robust 95% CIs via a moving-block bootstrap ($\ell \in \{4, 6, 8, 10\}$, $B=10,000$). Ordering is stable across ℓ and we report the widest CI per format and show ℓ -sensitivity curves.

Table 5.26: Full / gzip: HD p_{95} with widest MBB 95% CI and selected ℓ^* .

Format	p_{95} (ms)	95% CI (ms)	CI width (ms)	ℓ^*
HDF5/HSDS	160 558	[157 142, 162 905]	5 763	10
ROOT/TTree	465 371	[451 293, 469 620]	18 327	10
TileDB	12 528	[11 830, 12 834]	1 004	6
Zarr	36 417	[35 871, 36 445]	574	10

Table 5.27: Full / lz4: HD p_{95} with widest MBB 95% CI and selected ℓ^* .

Format	p_{95} (ms)	95% CI (ms)	CI width (ms)	ℓ^*
HDF5/HSDS	152 821	[152 328, 153 067]	738	4
ROOT/TTree	375 321	[369 349, 376 319]	6 970	10
TileDB	14 427	[14 068, 14 582]	513	8
Zarr	30 297	[29 738, 30 409]	670	10

Table 5.28: Full / zstd: HD p_{95} with widest MBB 95% CI and selected ℓ^* .

Format	p_{95} (ms)	95% CI (ms)	CI width (ms)	ℓ^*
HDF5/HSDS	154 112	[152 178, 155 549]	3 371	8
ROOT/TTree	441 405	[423 524, 453 744]	30 219	4
TileDB	11 567	[10 790, 11 809]	1 019	10
Zarr	31 207	[30 815, 31 389]	573	4

5.4.3 Summary tables for downstream decision matrix

This section reports representative summary tables used in the downstream decision matrix; the complete set is provided in Appendix Section A.2. Entries are *session-conditional* and descriptive, where we report geometric mean (**GM**) latency with dependence-robust 95% **CI**s and p_{95} (**CI**). For full scans, we also include transferred-byte **GM** throughput (GiB/s, **CI**), and each table lists **S3** GETs/GiB and stored size (GiB).

5.4.3.1 Per-codec league tables

This subsection presents representative per-codec league tables to summarize practical rankings; remaining league tables are reported in Appendix Section A.2. Scores use the stated primary objective and tie-breakers and totals are descriptive and session-conditional.

5.4.3.1.1 Slice Rankings use lower **GM** latency (tie-break: $p_{95} \rightarrow$ GETs/GiB $\rightarrow$ stored GiB), which are descriptive and session-conditional. We show representative league tables for `gzip` and `zstd` in the main text; the `lz4` slice league table is reported in Appendix Table A.6.

Table 5.29: Slice — `gzip`: league table.

Rank	Format	GM (ms) [CI]	p_{95} (ms) [CI]	GETs/GiB	Stored GiB
1	TileDB	195 [181, 210]	262 [232, 318]	113.3	11.44
2	Zarr	229 [223, 236]	283 [262, 314]	126.7	11.09
3	HDF5/HSDS	241 [232, 252]	342 [295, 364]	482.2	11.11
4	ROOT/TTree	406 [397, 416]	471 [451, 501]	61.3	11.09

Table 5.30: Slice — `zstd`: league table.

Rank	Format	GM (ms) [CI]	p_{95} (ms) [CI]	GETs/GiB	Stored GiB
1	Zarr	155 [149, 162]	195 [177, 243]	136.6	10.51
2	TileDB	177 [165, 190]	250 [205, 312]	120.7	10.74
3	HDF5/HSDS	210 [205, 215]	255 [236, 286]	488.0	10.67
4	ROOT/TTree	269 [264, 275]	311 [292, 341]	66.2	10.26

5.4.3.1.2 Full Scan Rankings use higher **GM** throughput (tie-break **GM** time $\rightarrow$ GETs/GiB $\rightarrow$ stored GiB), which are descriptive and session-conditional. We show the `zstd` full-scan league table in the main text; the corresponding `gzip` and `lz4` tables are reported in Appendix Tables [A.7](#) and [A.8](#).

Table 5.31: Full — `zstd`: league table.

Rank	Format	GM thr (GiB/s) [CI]	GM time (ms) [CI]	GETs/GiB	Stored GiB
1	TileDB	1.055 ([1.025, 1.082])	10 384 ([10 123, 10 687])	96.9	10.74
2	Zarr	0.353 ([0.351, 0.356])	30 365 ([30 167, 30 555])	123.2	10.51
3	HDF5/HSDS	0.072 ([0.072, 0.072])	151 497 ([151 071, 151 996])	142.2	10.67
4	ROOT/TTree	0.056 ([0.054, 0.059])	373 589 ([357 497, 390 216])	63.2	10.26

5.4.3.2 Normalized scores matrix

This thesis uses normalized score matrices as a compact multi-criteria view for configuration selection. The full score matrices are provided in Appendix Tables [A.9–A.14](#) with corresponding heatmaps in Appendix Figures [A.24–A.29](#). We use these scores as descriptive decision support; the practitioner-facing guidance is summarized in Chapter [7](#).

Chapter 6

Discussion

This chapter interprets the results, links them to layout and request behavior, and states the limits where all cross-format contrasts remain descriptive and session-conditional. It then answers *RQ1* (slice latency in Section 5.3.1), *RQ2* (full-scan performance in Section 5.3.2), and *RQ3* (cost posture in Section 5.2), followed by the decision matrix and threats to validity.

6.1 Answers to the research questions

This section reports winners by workload and codec, using **GM** latency for slices and **GM** scan time for full scans, together with transferred-byte throughput summaries, dependence-robust 95% **CI**s, and p_{95} for tail risk. All contrasts are *descriptive* and *session-conditional*.

6.1.1 Answer to RQ1 — Interactive slice latency

Single-frame reads have two stress areas: (i) layout/objectization, which sets how many objects and round-trips each request touches, and (ii) decode cost, which depends on the codec and the library path. RQ1 and hypotheses H1.1–H1.3 (Section 3.1.4) ask how these factors shape slice-level **GM** latency and p_{95} . We judge the winners by geometric mean (**GM**) latency with dependence-robust 95% **CI**s and use p_{95} to flag tail risk. The references below point to the slice tables and figures for each codec.

Zarr is the fastest in terms of **GM slice latency for **ZSTD** in this dataset.** Its **GM CI** lies fully below TileDB's, which is second, followed by **HDF5/HSDS** and then **ROOT/TTTree**. The p_{95} ordering mirrors the center, indicating Zarr combines a fast typical latency with controlled tails.

Mechanistically, this matches a chunk-per-frame layout that amortizes **S3** RTT via client concurrency, while **ZSTD** keeps the decode cost moderate. See Table 5.4 and Figures A.17, A.18, A.19. **GM** deltas show that Zarr is $\approx 12\%$ faster than TileDB (Table 5.4). Together, the three codecs confirm H1.1–H1.2: chunked/fragmented formats consistently beat **ROOT/TTree** on **GM** latency, Zarr leads for **ZSTD** and **LZ4**, and TileDB leads for **gzip**.

Zarr remains fastest on GM latency under LZ4. TileDB and HDF5/HSDS are close behind it, with overlapping **CIs** between those two, and **ROOT/TTree** is slowest. A trade-off appears in the tails: Zarr’s p_{95} is higher and has wider **CIs** than TileDB and **HDF5/HSDS**, consistent with **LZ4**’s bias toward decode speed over compactness. If an application is strongly tail-sensitive, TileDB’s lower and more stable p_{95} may offset its slightly higher **GM**. See Table 5.3 and Figures A.14, A.15, A.16. **GM** deltas show that Zarr is $\approx 18\%$ faster than TileDB and $\approx 21\%$ faster than **HDF5/HSDS** (Table 5.3). This supports H1.1 and H1.2 for **LZ4**: Zarr is the fastest layout on **GM** latency, while TileDB and **HDF5/HSDS** sit close behind, and **ROOT/TTree** remains clearly slower.

TileDB wins gzip slices with non-overlapping **GM CIs** versus Zarr. Zarr is next and outperforms **HDF5/HSDS**, where **ROOT/TTree** again is slowest by a wide margin. This pattern aligns with **gzip**’s higher **CPU** cost, which leads to TileDB’s fewer, larger fragments reducing request overhead enough to compensate for **gzip** decode, while Zarr’s per-frame objects incur more round-trips. Ordering is consistent in p_{95} and in the Zarr-referenced ratio view. See Table 5.2 and Figures A.11, A.12, A.13. **GM** deltas show that TileDB is $\approx 17\%$ faster than Zarr (Table 5.2). These **gzip/slice** results are consistent with H1.1 (all chunked/fragmented layouts are faster than **ROOT/TTree**) and with H1.2’s expectation that TileDB’s fragment layout can win under **gzip** by reducing request fan-out.

Across codecs in this dataset, the winner patterns align with evidence of request intensity: lower GETs/GiB contributes, but does not predict slice latency across formats; object size, parallelism, and decode paths matter. At the same time, **ROOT/TTree**’s single-object layout shows that “few GETs” does not guarantee speed, because paging and decode path overhead dominate. See Figure 5.6 and Table 5.9.

Nonparametric summaries reproduce the same slice rankings by codec, and Harrell–Davis p_{95} with an ℓ -grid preserves ordering, including the heavier tails for **LZ4/Zarr**. See Figure A.20 and Tables 5.11, 5.13, 5.15, and the tail-sensitivity artifacts in Figure A.22 and Tables 5.23–5.25. Throughout this chapter, such “consistency” language refers to agreement in the *ordering*

of *central tendencies* (**GM** and median) across formats. It does not mean that every **GM** estimate is exact, especially in more variable cells such as **LZ4** slices for Zarr and TileDB, where the CIs for means and tails are comparatively wide.

6.1.2 Answer to RQ2 — Full-dataset scan performance

Full-dataset reads test sustained sequential delivery and decode under continuous concurrency. RQ2 and hypotheses H2.1–H2.3 (Section 3.1.4) use **GM** scan time as the clearest same-logical-dataset completion metric and also report **GM** throughput (GiB/s), computed from session-specific downloaded GiB per repetition and **GM** time with dependence-robust 95% **CIs**. The latter should be read as transferred-byte throughput of the stored representation rather than as a pure logical-payload throughput. p_{95} times are used to check stability. All cross-format contrasts are *descriptive* and *session-conditional*. Across **ZSTD**, **LZ4**, and **gzip**, **TileDB leads by a wide margin, Zarr is consistently second, and HDF5/HSDS and ROOT/TTree come last**.

Supporting **GM**-time ratios versus Zarr show that TileDB’s full-scan **GM** time is $\approx 0.32\times$ under **gzip**, $\approx 0.46\times$ under **LZ4**, and $\approx 0.34\times$ under **ZSTD**; see Tables 5.6, 5.7, and 5.8.

Under **ZSTD**, **TileDB is the clear throughput leader**. Zarr is a distant second, and **HDF5/HSDS** and **ROOT/TTree** are far behind. **GM** time and its **CI** show clean separation, and the ratio view reinforces TileDB’s advantage. p_{95} times follow the same ordering and are far apart in magnitude, particularly between TileDB/Zarr and **ROOT/TTree**, which supports the same qualitative throughput ordering for both center and tail in this dataset. See Table 5.8 and Figures A.36, A.37, A.38. These full-scan results across codecs support H2.1 (TileDB > Zarr > **HDF5/HSDS** > **ROOT/TTree** on **GM** throughput) and H2.2 (both **LZ4** and **ZSTD** improve throughput relative to **gzip** for our throughput metric (downloaded bytes/time)).

Under **LZ4**, **TileDB remains first and Zarr second**. Faster decode narrows absolute times but does not change ranks. **HDF5/HSDS** and **ROOT/TTree** stay well behind. p_{95} CIs are relatively tight for TileDB and Zarr compared with the gap to **ROOT/TTree**, which supports the qualitative picture of TileDB as the highest-throughput format and Zarr second for this workload. See Table 5.7 and Figures A.33, A.34, A.35.

Under **gzip**, **TileDB still leads and Zarr is second**. **gzip**’s higher **CPU** cost lifts absolute times for all formats, but ordering is unchanged. **HDF5/HSDS** and **ROOT/TTree** remain the slowest. p_{95} point estimates are ordered in the same way as the **GMs**, so tail summaries do not contradict the

GM-based throughput ordering. See Table 5.6 and Figures A.30, A.31, A.32.

In these experiments, throughput patterns align with request intensity and layout: TileDB attains high throughput with moderate GETs/GiB, thanks to fragment-backed long reads. Zarr has higher GETs/GiB yet parallelizes effectively, whereas **HDF5** via **HSDS** shows higher GETs/GiB but poorer streaming performance. **ROOT/TTree** has few GETs/GiB but still has low throughput because its single-object layout and paging/decode overhead dominate. See Figure 5.7 and Table 5.10.

CloudWatch FirstByte and TotalRequest latencies are stable within sessions and do not reflect the large cross-format gaps in client timings, where the dominant drivers are layout and decode rather than differences in **S3** service timing. See Figures A.9, A.10.

Nonparametric centers yield the same ranks across codecs, and the **HD** p_{95} sensitivity maintains the ordering. See Figure A.21 and Tables 5.17, 5.19, 5.21, see also Figure A.23 and Tables 5.26–5.28.

6.1.3 RQ3 — Cost posture and mechanisms (summary)

Chunk/fragment layouts (Zarr, TileDB) produce low-to-moderate GETs/GiB and are associated with lower **GM** slice latency and higher scan throughput in this dataset. In contrast, **HSDS** yields higher GETs/GiB, and **ROOT/TTree** is the counterexample: few GETs yet slow due to single-object paging and decode (Section 5.2; Figures 5.6, 5.7). Storage posture is codec-driven overall (**ZSTD** $\approx$ **gzip** $<$ **LZ4**), with a significant inflation for **LZ4+HSDS** compared with Zarr/TileDB at the same payload (Table A.5, Figure 5.1, Figure 5.2). **S3** FirstByte/TotalRequest latencies are stable across sessions and do not explain cross-format gaps; they serve as context only (Figure A.9, Figure A.10). Mechanistically, objectization determines request patterns, and the codec defines the bytes transferred together; these factors are consistent with the client timings observed in 5.3.1–5.3.2.

These findings address H3.1–H3.2: codec choice (especially **LZ4** vs **ZSTD/gzip**) dominates stored GiB, Zarr and TileDB offer a more favorable request+storage posture than **HSDS+LZ4** or **ROOT**, and the relatively flat CloudWatch service latencies confirm that **S3** service timing is a background factor rather than a primary driver of client-observed performance.

6.2 RQ3 interpretation: mechanisms and cost posture

We explain the observed rankings by linking objectization and GETs per GiB to client timings. CloudWatch service latencies are treated as context, where codec trade-offs and request and storage posture complete the picture.

6.2.1 Mechanisms explaining outcomes

This section explains *why* the observed orderings occur. We link layout-driven objectization and request intensity (GETs/GiB) to client timings, show that **S3** service latencies are stable and not causal, and summarize codec **CPU**–bytes trade-offs. In this experiment, request intensity contributes to performance within chunked and fragmented stores, but it does not fully predict outcomes: lower GETs/GiB often coincides with higher full-scan throughput and can coincide with lower slice **GM** latency, yet cases like TileDB vs Zarr show that object size, parallelism, and decode paths can dominate even when GETs/GiB is lower (Figures 5.6, 5.7; Tables 5.9, 5.10).

6.2.1.1 Objectization and layout

Objectization for each storage backend determines the request pattern. Zarr/**HSDS** materializes roughly one data object per frame (plus metadata), TileDB writes a small number of immutable fragments (each with internal files), and **ROOT/TTree** stores a single large object. Per-frame layouts expose more opportunities for client concurrency, fragment layouts trade fewer objects for long sequential reads, and single-object layouts minimize GET count but constrain parallelism to range requests within one key. This matches the data in Table A.5 and Figure 5.3. In practice, concurrency benefits Zarr and TileDB, while **ROOT/TTree** shows few GETs but slow performance because paging and decoding dominate once request latency is amortized.

6.2.1.2 Request intensity vs client timing

In this experiment, lower request intensity is associated with better client outcomes in both chunked and fragmented stores. Fewer GETs/GiB are associated with lower slice **GM** and higher full-scan throughput for Zarr and TileDB (Figure 5.6, Figure 5.7, Tables 5.9, 5.10). The pattern is not universal: **ROOT/TTree** has very low GETs/GiB yet poor latency and

throughput because the single-object layout, small pages, and decode costs limit effective parallelism. GETs/GiB is one factor, but does not predict latency across formats; object size, parallelism, and decode path matter, with layout and decode paths accounting for the rest. These observations are consistent with H1.3: lower GETs/GiB aligns with lower **GM** slice latency for Zarr and TileDB, while **ROOT/TTree** violates the monotonic trend by combining few GETs with poor latency due to paging and decode overhead.

6.2.1.3 Service latencies are not the driver

CloudWatch *FirstByteLatency* and *TotalRequestLatency* are stable within sessions (both averages and p_{95}) and do not track the large cross-format gaps in client timings. The dominant factors are request pattern and decode path rather than “faster **S3**.” These metrics provide context, not causal evidence (Figures A.9, A.10).

6.2.1.4 Codec effects

Codecs trade **CPU** time for stored size and transferred bytes. **LZ4** minimizes decode time and helps latency where per-chunk **CPU** dominates. **ZSTD** strikes a balance between compression and decoding, while `gzip` yields the smallest stored size but is slower to decode. Stored size is primarily codec-driven (typically $\text{ZSTD} \approx \text{gzip} < \text{LZ4}$), but format deltas are large under **LZ4**: in our data, **HDF5** via **HSDS** stored 19.17 GiB vs Zarr 10.40 GiB and TileDB 15.56 GiB (Table A.5, Figures 5.1, 5.2). **LZ4+HSDS** should therefore be treated as storage-heavy. These trade-offs align with the winners in Section 6.1 where Zarr leads **ZSTD/LZ4** slices while TileDB leads `gzip` slices (Tables 5.4, 5.3, 5.2). TileDB dominates full-scan throughput across codecs (Tables 5.8, 5.7, 5.6).

As summarized in Figure 5.5, **ZSTD** consistently achieves lower or comparable latency and higher throughput than `gzip` at similar compression ratios. At the same time, **LZ4** reduces latency further at the cost of substantially higher storage, especially for **HDF5/HSDS**. This matches the codec trade-offs described above.

Across formats, **ZSTD** consistently outperforms `gzip` on both slice latency and full-scan throughput, while achieving similar compression ratios (Figure 5.2). **LZ4** reduces slice latency further but provides a slight throughput advantage over **ZSTD** and incurs noticeably larger stored size, especially for **HDF5/HSDS** (Figure 5.1). Compared with the related work summarized in Section 2.3.4, these results are consistent with the general mechanism

that object-store performance depends on physical granularity and metadata strategy rather than on nominal file format alone. The contribution here is not a new codec benchmark in isolation, but the combined observation that frame-aligned chunking favors Zarr for interactive slices, fragment-backed long reads favor TileDB for full scans, and single-object layouts can underperform even when GET counts are low. This extends prior storage and compression observations to a controlled scattering-style **S3** benchmark where the same detector stack, codecs, and workloads are held fixed.

6.2.2 Cost and storage posture

Stored size is mostly codec-driven, but **LZ4** shows substantial format variation (**HSDS** 19.17 GiB vs Zarr 10.40 GiB vs TileDB 15.56 GiB, see Table **A.5**). For compact storage, prefer **ZSTD** or `gzip` and avoid **LZ4+HSDS** unless storage is unconstrained.

Three practical cost observations follow. First, for slice-heavy workloads, latency is the user-facing metric, and GETs/GiB is the request-cost proxy. Second, stored size is mainly codec-driven. Third, **ROOT/TTree** shows that low GETs/GiB alone does not imply good performance.

But as a counterexample, **ROOT/TTree** shows that a few requests do not imply speed or efficiency. It has the lowest *GETs/GiB* yet poor slice latency and scan throughput due to single-object paging and decode overhead. See Tables **5.9**, **5.10** and the per-codec **MBB** results (slices: Tables **5.2**, **5.3**, **5.4** and full scans tables **5.6**, **5.7**, **5.8**).

6.3 Threats to validity and limits

This section checks whether the rankings are robust to the main statistical complications in the timing data. Skew motivates the log transform, serial dependence motivates the moving-block bootstrap, and observed drift motivates conservative, session-conditional interpretation.

6.3.1 Distribution shape and robustness

We verify that the modeling choices are appropriate and yield results similar to those of other estimators. We assess shape and serial dependence to justify log-time analysis with the moving-block bootstrap (**MBB**), then confirm rankings with median centers, and finally test tail stability with Harrell–Davis

6.3.1.1 Shape and dependence

The timing data support log-scale summaries, but not strict independent-sample inference. Run-level times are positive and right-skewed, and the diagnostics show frequent positive lag-1 autocorrelation across cells. The Q–Q plots for $\ln(\text{time})$ indicate that the log transform gives a more stable basis for estimating central tendency, while the observed autocorrelation motivates **MBB CIs** rather than intervals based on independent runs (Appendix Table A.2; Appendix Figures A.1, A.2).

Some cells also show monotonic run-order trends in the trace plots and the Spearman drift diagnostic. The **MBB** intervals are therefore interpreted as dependence-robust, session-conditional summaries of the timed window, not as formal inference under strict stationarity (Appendix Figures A.3, A.4). Together, these diagnostics support reporting **GM** times from log-time with dependence-robust **CIs**.

6.3.1.2 Center robustness

Median-based rankings match **GM** across codecs and patterns. Descriptive median ratios reproduce the slice and full-scan orderings, and rank-agreement tables show identical ranks to **GM** within each codec (Figures A.20, A.21 and Tables 5.11, 5.13, 5.15, 5.17, 5.19, 5.21, and 5.12, 5.14, 5.16, 5.18, 5.20, 5.22). Thus, when we describe rankings as “consistent” across estimators, this refers to the relative ordering of formats, not to low variance or narrow CIs for every **GM** estimate.

6.3.1.3 Tail robustness

Orderings are stable under Harrell–Davis p_{95} with an $\ell \in \{4, 6, 8, 10\}$ block-length grid, where we report the widest dependence-robust **CI** per cell. The notable exception is **LZ4/Zarr**, which shows heavier tails relative to its peers, but this does not overturn the winner hierarchy established by **GM** and median centers (Figures A.22, A.23 and Tables 5.23, 5.24, 5.25, 5.26, 5.27, 5.28).

6.3.2 Validity and limits

We state what the measurements support and where they do not generalize. Internal checks and external scope are made explicit so readers interpret results correctly.

6.3.2.1 Internal checks

All planned runs were completed without failure, and warm-ups were excluded by design. Run order was randomized with a fixed seed. Serial dependence is handled via the moving-block bootstrap, and the log-time choice is supported by shape diagnostics (Section 5.1 and Appendix Table A.2).

6.3.2.2 External validity and synthetic-data limits

Findings reflect an **AWS**-only setup (**S3** + **EC2** + CloudWatch) using one **S3** bucket in a single region (`eu-north-1`) and a single **EC2** client, with two access patterns and three codecs. No multi-client contention, cross-region traffic, or non-**AWS** object stores were evaluated; performance and cost posture may differ on Azure Blob Storage, Google Cloud Storage, or on-premises **S3**-compatible systems. We do not include an on-premises baseline (e.g., Lustre/**GPFS** or **POSIX** file I/O), so results should be interpreted as *within-S3* cross-format comparisons rather than cloud-versus-**HPC** claims. The detector stack is also synthetic beyond the 80 real seed frames. This preserves geometry and I/O scale, but may alter redundancy and therefore compression ratios compared with a larger fully authentic datastack; stored-size results should therefore be interpreted as codec/layout behavior for this controlled setup, not as universal compression ratios for all beamlines. The dataset size is sufficient to expose request/metadata overheads for the tested workloads, but it does not establish asymptotic behavior at multi-terabyte or petabyte scale. Cross-format contrasts are *descriptive* and *session-conditional*. CloudWatch service metrics are used solely for descriptive context, not as causal evidence (Figures A.9, A.10).

6.4 Deviations and exclusions

Parquet was piloted only and excluded from timed comparisons. When we scaled to complete ingestion and attempted full-dataset scans, a single large object (~34 GiB for a ~20 GiB raw stack) incurred 3–4 min metadata open time and intermittent client instability on the `gzip` file, making it unsuitable for frame-granularity reads and unattractive even for bulk scans in this setup. The root cause is objectization and file semantics: a single-object, columnar Parquet file offers few opportunities for request-level parallelization and concentrates footer/metadata work at open. More fundamentally, Parquet is a tabular, columnar format; mapping dense image frames into thousands

of list-valued row columns is an unnatural representation for frame-random image access. We therefore retain Parquet as a qualitative caution for n -D image stacks on [S3](#) single-object layouts and exclude it from quantitative timing (Chapter [5](#)). This evidence is limited to the tested single-object layout and should not be generalized to partitioned Parquet datasets with tuned file and row-group sizing (data-lake style); evaluating such variants was out of scope.

Chapter 7

Conclusions and Future Work

Zarr yields the lowest slice latency under **ZSTD** and **LZ4**, while TileDB wins slices under `gzip`. TileDB completes full sequential scans $2.2\text{--}3.2\times$ faster (lower **GM** scan time) than Zarr across codecs, with stable tails. **ROOT/TTree** remains slow for frame-random reads despite a few GETs.

For RQ1, Zarr+**ZSTD** is the strongest latency-first choice, with TileDB+`gzip` best when `gzip` is required. For RQ2, TileDB dominates full sequential scans across all codecs. For RQ3, objectization and codec choice explain the cost posture: Zarr and TileDB offer better request/performance balance than **HSDS**, while **ROOT/TTree** shows that low GET counts alone do not guarantee speed.

Layout and request behavior explain the backend differences. Per-frame objectization (Zarr, **HDF5** via **HSDS**) and fragment layouts (TileDB) expose client concurrency, whereas a single-object layout (**ROOT/TTree**) minimizes GETs but constrains parallelism and incurs paging/decode overhead. Codec choice trades **CPU** for bytes, with **ZSTD** and `gzip` more compact than **LZ4**. In our data **HDF5** via **HSDS** with **LZ4** stored 19.17 GiB versus Zarr 10.40 GiB and TileDB 15.56 GiB at the same payload size, indicating a storage-heavy configuration.

Winners are judged by geometric-mean (**GM**) latency for slices and by **GM** scan time for full scans, with **GM** throughput retained as an operational transferred-byte summary, together with dependence-robust 95% **CI**s and Harrell–Davis p_{95} for tails. Findings reflect an **AWS**-only **S3** testbed on one region (eu-north-1) and one client, with no on-prem baseline; claims should be read as within-**S3** cross-format contrasts. CloudWatch service latencies are used as context, not causal drivers.

The remainder of the chapter provides a closing synthesis, decision-

oriented recommendations, backend-specific operational implications, and a prioritized agenda for future work. Conclusions map to RQ1–RQ3 and reference Chapter 5 (sections 5.3.1, 5.3.2, 5.2).

7.1 Closing synthesis

Across single-frame slices and full scans, summaries of central tendency (**GM** and median) show a consistent, session-conditional ordering. For slices, **Zarr** is fastest under **ZSTD** and **LZ4**, while **TileDB** wins under **gzip**. **HDF5/HSDS** is slower than Zarr and TileDB but substantially faster than **ROOT/TTree**, which is slowest. For full scans, **TileDB** delivers the highest throughput across all codecs by a wide margin, with **Zarr** second and **HDF5/HSDS** and **ROOT/TTree** far behind. Mechanisms include objectization/request patterns and decode paths: per-frame objects (Zarr/**HSDS**) and fragments (TileDB) enable client concurrency. In contrast, a single object (**ROOT/TTree**) minimizes GETs but limits parallelism and pays paging/decode overhead. In this dataset, *few GETs* therefore does not imply speed, which can be seen in the cross-metric views (5.9, 5.10). Cost properties follow: request cost tracks GETs/GiB (important for slice-heavy use), while stored GiB is codec-driven in our measurements ($\text{ZSTD} \approx \text{gzip} < \text{LZ4}$). For workloads similar to this detector stack, these results suggest choosing **ZSTD** or **gzip** when storage dominates and **LZ4** when **CPU**/latency dominates.

If one backend+codec must be selected for both workloads with no site-specific weighting, **TileDB+ZSTD** is the most defensible general-purpose default in this testbed: it is only about 14% slower than Zarr on **ZSTD** slice **GM** latency, but about $2.9\times$ faster than Zarr on **ZSTD** full-scan **GM** time. The exception is a latency-first interactive UI, where **Zarr+ZSTD** remains the safer default for the fastest single-frame reads.

7.2 Practitioner choices (decision matrix hook)

This section turns the empirical rankings into concrete decisions. Choices are related to the primary objective of each workload. All evidence is descriptive and session-conditional, and ties back to the league tables and per-codec slice/full results.

7.2.1 RQ1 — Interactive slice latency (latency–first)

For random single-frame reads, the default recommendation is *Zarr* + *ZSTD*, which delivers the lowest slice *GM* with controlled tails in this setting and scales well with client concurrency (Tables 5.30, 5.4). If *gzip* is mandated, *TileDB* + *gzip* is the better choice because *TileDB*’s fragment layout offsets *gzip*’s *CPU* cost and wins on *GM* with non-overlapping *CIs* versus *Zarr* (Tables 5.29, 5.2). One caution is that *HSDS*+*LZ4* becomes unattractive when storage cost matters, since it stored 19.17 GiB compared with *Zarr*’s 10.40 GiB at the same codec (Table A.5). By contrast, *ROOT/TTree* should be avoided for frame-random reads: few GETs do not translate into low latency when paging and decode dominate (Tables 5.29, A.6, 5.30 and Tables 5.2, 5.3, 5.4).

7.2.2 RQ2 — Full-dataset scan performance (scan–time–first)

For full-dataset scans, the default recommendation is *TileDB* + *ZSTD*. *TileDB* dominates full-scan performance across codecs in this testbed, and *ZSTD* provides the best overall scan time while keeping the stored footprint compact. *TileDB* + *LZ4* is worth considering only if the scan becomes *CPU*-bound in a deployment where the larger stored size and downloaded bytes are acceptable (Tables 5.31, A.8 and Tables 5.8, 5.7). When ecosystem fit matters more than maximizing throughput, *Zarr* remains the second choice across *ZSTD*, *LZ4*, and *gzip*, trading some scan speed for broader tooling while preserving stable streaming behavior (Tables A.7, A.8, 5.31 and Tables 5.6, 5.7, 5.8).

7.2.3 Weight sensitivity

Top selections remain stable under the stated weights, as the selected weights do not alter the winners in our data. Where near-ties appear within a codec×pattern bucket, we call them out in the league tables and recommend using the tie-breakers (slice: $p_{95} \rightarrow \text{GETs/GiB} \rightarrow \text{stored GiB}$ and for full scan *GM* time $\rightarrow \text{GETs/GiB} \rightarrow \text{stored GiB}$). See the score tables (A.9, A.10, A.11, A.12, A.13, A.14).

7.3 Operational implications

This section converts the Results into backend-specific operational actions. Recommendations cover metadata consolidation, prefix sharding, chunk/tile

sizing, fragment consolidation, cache tuning, and read-page batching. Measured object counts justify each item, GETs/GiB, and p_{95} tails.

7.3.1 Zarr

Consolidate metadata to cut LIST/HEAD fan-out and reduce cold-open overhead; unconsolidated layouts inflate object count and amplify per-request latency (Figure 5.3). Use stable, logical key naming; **S3** no longer requires randomizing prefixes for performance. Only introduce additional prefix sharding if you need to scale very high request rates through parallelism and to keep operational grouping manageable as datasets grow. **Keep frame-sized chunks** (1, ·, ·) for frame-granularity reads and adjust only if the access unit changes. **Monitor tails on LZ4**: slice p_{95} is heavier for Zarr+LZ4; prefer **ZSTD** for tighter latency envelopes or mitigate with concurrency and consolidated metadata (Figure A.15, Figure 5.3).

7.3.2 TileDB

Manage fragment counts: each write produces an immutable fragment, where many small fragments raise metadata work. Schedule **fragment consolidation** when counts grow to keep open/scan overhead low (Table A.5). **Align tile extents** to the access unit (frame-aligned tiles for these arrays) to preserve long sequential reads. These practices sustain **scan dominance** observed across codecs (Figures A.30, A.33, A.36) and the favorable throughput vs GETs/GiB posture (Figure 5.7).

7.3.3 HDF5/HSDS

Account for REST fan-out and **many small objects**: chunk and metadata requests add round-trips (Table A.5). Provision and tune **server-side caches** (dataset/chunk caches) and concurrency to hide latency, and right-size chunks to reduce request pressure where feasible. CloudWatch service latencies are stable across sessions, so speedups must come from reducing fan-out or improving caching, not from “faster **S3**” (Figures A.9, A.10).

7.3.4 ROOT/TTree

Batch page reads and **avoid per-frame random access**: issue ranged reads and increase read windows so many small pages are fetched together. Prefer workflows that **scan clusters** over frame-random access for images.

Remember that **few GETs do not imply speed**: single-object layout plus paging/decode keeps latency high despite low GETs/GiB (Tables 5.9, 5.2, 5.3, 5.4).

7.3.5 Sustainability, ethics, and societal relevance

Although this thesis is primarily a systems benchmark, its findings have implications for sustainable development because storage layout and codec choice materially change stored GiB, downloaded bytes, request intensity, and end-to-end processing time. From an *economic sustainability* perspective, these quantities map directly to cloud request and storage costs and therefore to the long-term operating cost of data-intensive scientific platforms. From an *ecological sustainability* perspective, lower stored volume, fewer transferred bytes, and shorter scan times are plausible indicators of lower infrastructure resource use. However, this thesis does not measure electricity consumption, hardware life-cycle impact, or carbon emissions, so any ecological implication should be read as directional rather than quantified. From a *social sustainability* perspective, lower latency for slice access and higher scan throughput can reduce waiting time for researchers and make remote analysis workflows more usable for geographically distributed users of shared facilities.

The ethical aspects are limited but still relevant. The work does not involve human subjects, personal data, or safety-critical decision making; the benchmark concerns synthetic detector data and storage-system behavior. The main ethical requirement is therefore responsible reporting and interpretation. Because the results could influence infrastructure choices at publicly funded laboratories, the thesis makes its scope explicit, keeps all cross-format contrasts descriptive and session-conditional, and avoids claiming measurements that were not performed. It is also important not to overstate generality across clouds, datasets, or implementations, since doing so could lead to costly or misleading engineering decisions.

In a broader societal perspective, the thesis is relevant for beamline scientists, data engineers, facility operators, and research organizations that must manage rapidly growing experimental datasets under budget and infrastructure constraints. Better storage choices can shorten the time from experiment to analysis, improve the usability of shared research infrastructure, and support more efficient use of public research funding. The contribution is therefore not only technical performance ranking, but also decision support for how cloud-based scientific data platforms can be designed more transparently and efficiently within the limits of the tested **AWS S3** setting.

7.4 Future work

This section highlights the highest-leverage uncertainties revealed by the results and frames them as follow-up studies that either de-risk tail behavior or generalize the findings beyond a single dataset, region, or client.

Several follow-up directions emerge directly from the mechanisms observed here. One is to study multi-client scaling and cache effects for **HSDS** and Zarr, so their impact on **GM** and p_{95} under contention can be measured explicitly. Another is to evaluate consolidation strategies, specifically Zarr metadata consolidation and TileDB fragment consolidation, and relate them to GETs/GiB, open time, and tail behavior. A particularly important follow-up is to rerun the codec study with stricter cross-backend harmonization of codec wrappers and filters, especially by comparing the evaluated Zarr Blosc(1 z 4) path against a direct/plain 1 z 4 path where supported.

The experimental matrix should also be widened to include additional codecs and filters such as Snappy, bitshuffle+**ZSTD** levels, and ZFP, together with chunk and tile sweeps aligned to the intended access unit. Beyond that, the thesis should be extended to sparse or multi-detector stacks, batched and random windows, mixed read/write workloads, and comparisons across regions, clouds, and on-prem **PFS** deployments while modeling request, storage, and egress cost surfaces for representative usage patterns. A direct follow-up should repeat the benchmark on a larger, fully authentic beamline dataset, ideally at multi-terabyte scale, to test whether the rankings and compression behavior observed on the synthetic 20 GiB stack persist as dataset size and experimental diversity increase.

A particularly valuable next step would be a dedicated comparison between conventional **HDF5** on high-IOPS **Elastic Block Store (EBS)** or **NVMe-backed GPFS/Lustre** and **HDF5** via **HSDS** on **S3**, using identical datasets and access patterns. That experiment would separate *format* behavior from *infrastructure* effects and directly test whether block storage can deliver substantially lower slice latency than **S3-backed HSDS** for the same **HDF5** layout. A related study should isolate **HSDS** spin-up and control-plane overhead by separating container start-up, metadata warm-up, and steady-state behavior. This thesis focuses on warm, steady-state measurements after warm-ups, so that decomposition would clarify the file sizes, session counts, and query rates at which **HSDS** amortizes its overheads and becomes competitive with alternative layouts.

The decision matrix itself could also be extended with weight-sensitivity visualizations, per-site presets, and a one-page selection checklist. Finally,

it would be useful to quantify cold-open behavior for Zarr `consolidate_metadata()` versus unconsolidated stores and for TileDB fragment consolidation, and to benchmark **ROOT**'s newer **RNTuple** backend under the same **S3** testbed so the effect of its redesigned page and cluster layout on frame-random latency, full-scan throughput, and request intensity can be measured directly.

References

- [1] T. Celcer *et al.*, “Gigafrost (gigabyte fast read-out system for tomography): Control and daq system design,” in *16th Int. Conf. on Accelerator and Large Experimental Control Systems (ICALEPCS’17)*, Barcelona, Spain, 8-13 October 2017. JACOW, Geneva, Switzerland, 2018, pp. 1240–1244. [Pages 2, 11, 12, and 15.]
- [2] S. Narayan and J. A. Chandy, “I/o characterization on a parallel file system,” in *Proceedings of the 2010 International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS’10)*. IEEE, 2010, pp. 133–140. [Online]. Available: <https://snsl.engr.uconn.edu/wp-content/uploads/sites/2477/2018/06/spects10-submission.pdf> [Page 2.]
- [3] J. Moore, D. Basurto-Lozada, S. Besson, J. Bogovic, J. Bragantini, E. M. Brown, J.-M. Burel, X. Casas Moreno, G. de Medeiros, E. E. Diel *et al.*, “Ome-zarr: a cloud-optimized bioimaging file format with international community support,” *Histochemistry and Cell Biology*, vol. 160, no. 3, pp. 223–251, 2023. [Pages 3, 26, and 34.]
- [4] T. Ivanov and M. Pergolesi, “The impact of columnar file formats on sql-on-hadoop engine performance: A study on orc and parquet,” *Concurrency and Computation: Practice and Experience*, vol. 32, no. 5, p. e5523, 2020. [Pages 3 and 34.]
- [5] J. López-Gómez and J. Blomer, “Exploring object stores for high-energy physics data storage,” *CoRR*, vol. abs/2107.07304, 2021. [Online]. Available: <https://arxiv.org/abs/2107.07304> [Pages 3 and 34.]
- [6] A. R. Hevner, S. T. March, J. Park, and S. Ram, “Design science in information systems research,” *MIS quarterly*, pp. 75–105, 2004. [Page 4.]

- [7] M. Eriksson, J. F. Van der Veen, and C. Quitmann, “Diffraction-limited storage rings—a window to the science of tomorrow,” *Synchrotron Radiation*, vol. 21, no. 5, pp. 837–842, 2014. [Pages 10 and 12.]
- [8] C. R. Carnegie, R. A. Beaumont, R. Dashwood, and D. J. Hughes, “Uninterrupted in-situ compression of automotive expanded polypropylene, imaged via synchrotron x-ray computed tomography,” in *MATEC Web of Conferences*, vol. 401. EDP Sciences, 2024. [Page 10.]
- [9] W. Yoshimune, Y. Higuchi, F. Song, S. Hibi, Y. Matsumoto, H. Hayashida, H. Nozaki, T. Shinohara, and S. Kato, “Neutron imaging for automotive polymer electrolyte fuel cells during rapid cold starts,” *Physical Chemistry Chemical Physics*, vol. 26, no. 47, pp. 29 466–29 474, 2024. [Page 10.]
- [10] P. Braam, “The lustre storage architecture,” *arXiv preprint arXiv:1903.01955*, 2019. [Online]. Available: <https://arxiv.org/abs/1903.01955> [Pages xi, 11, and 18.]
- [11] M. Galchenkova, A. Tolstikova, B. Klopprogge, J. Sprenger, D. Oberthuer, W. Brehm, T. A. White, A. Barty, H. N. Chapman, and O. Yefanov, “Data reduction in protein serial crystallography,” *IUCrJ*, vol. 11, no. 2, pp. 190–201, Mar 2024. doi: 10.1107/S205225252400054X. [Online]. Available: <https://doi.org/10.1107/S205225252400054X> [Page 11.]
- [12] M. Yusuf, J. M. LaManna, P. P. Paul, D. N. Agyeman-Budu, C. Cao, A. R. Dunlop, A. N. Jansen, B. J. Polzin, S. E. Trask, T. R. Tanim, E. J. Dufek, V. Thampy, H.-G. Steinrück, M. F. Toney, and J. Nelson Weker, “Simultaneous neutron and x-ray tomography for ex-situ 3d visualization of graphite anode degradation in extremely fast-charged lithium-ion batteries,” *ChemRxiv preprint*, Version 1, jun 2022. [Page 11.]
- [13] P. Shrestha, J. M. LaManna, K. F. Fahy, P. Kim, C. Lee, J. K. Lee, E. Baltic, D. L. Jacobson, D. S. Hussey, and A. Bazylak, “Simultaneous multimaterial operando tomography of electrochemical devices,” *Science Advances*, vol. 9, no. 45, p. eadg8634, 2023. [Page 11.]
- [14] D. Ltd., “EIGER2 XE 16M x-ray detector: Technical specifications,” https://media.dectris.com/230216-Technical_Specifications-DECTRIS_EIGER2_XE_16M.pdf, 2025, accessed 7 June 2025. [Page 12.]

- [15] F. Leonarski, J. Nan, Z. Matej, Q. Bertrand, A. Furrer, I. Gorgisyan, M. Bjelčić, M. Kepa, H. Glover, V. Hinger, T. Eriksson, A. Cehovin, M. Eguiraun, P. Gasparotto, A. Mozzanica, T. Weinert, A. Gonzalez, J. Standfuss, M. Wang, T. Ursby, and F. Dworkowski, “Kilohertz serial crystallography with the JUNGFR AU detector at a fourth-generation synchrotron source,” *IUCrJ*, vol. 10, no. 6, pp. 729–737, Nov 2023. doi: 10.1107/S2052252523008618. [Online]. Available: <https://doi.org/10.1107/S2052252523008618> [Page 12.]
- [16] J. Cooper, I. Juvan-Beaulieu, and A. Wise, “Scientific cmos optical detectors for orbital debris observations,” in *First international orbital debris conference*, 2019. [Page 12.]
- [17] S. Veseli, N. Arnold, T. Berenc, J. Carwardine, G. Decker, T. Fors, T. Madden, and G. Shen, “Data acquisition system for the aps upgrade,” *ICALEPCS 2019*, 2019. [Page 12.]
- [18] W. Yashiro, W. Voegeli, and H. Kudo, “Exploring frontiers of 4d x-ray tomography,” *Applied Sciences*, vol. 11, no. 19, 2021. [Online]. Available: <https://www.mdpi.com/2076-3417/11/19/8868> [Page 12.]
- [19] V. Briois, J. P. Itié, A. Polian, A. King, A. S. Traore, E. Marceau, O. Ersen, C. La Fontaine, L. Barthe, A. Beauvois, O. Roudenko, and S. Belin, “Hyperspectral full-field quick-EXAFS imaging at the ROCK beamline for monitoring micrometre-sized heterogeneity of functional materials under process conditions,” *Journal of Synchrotron Radiation*, vol. 31, no. 5, pp. 1084–1104, Sep 2024. doi: 10.1107/S1600577524006581. [Online]. Available: <https://doi.org/10.1107/S1600577524006581> [Page 12.]
- [20] T. Bicer, D. Gursoy, R. Kettimuthu, I. T. Foster, B. Ren, V. De Andrede, and F. De Carlo, “Real-time data analysis and autonomous steering of synchrotron light source experiments,” in *2017 IEEE 13th International Conference on e-Science (e-Science)*, 2017. doi: 10.1109/eScience.2017.53 pp. 59–68. [Page 14.]
- [21] C. Wang, X. Li, R. Wan, J. Chen, J. Ye, K. Li, A. Li, R. Tai, and A. Sepe, “Accelerating imaging research at large-scale scientific facilities through scientific computing,” *Synchrotron Radiation*, vol. 31, no. 5, 2024. [Page 14.]

- [22] A. V. Babu, T. Zhou, S. Kandel, T. Bicer, Z. Liu, W. Judge, D. J. Ching, Y. Jiang, S. Veseli, S. Henke *et al.*, “Deep learning at the edge enables real-time streaming ptychographic imaging,” *Nature Communications*, vol. 14, no. 1, p. 7059, 2023. [Page 15.]
- [23] U. Lienert, S. F. Li, C. M. Hefferan, J. Lind, R. M. Suter, J. V. Bernier, N. R. Barton, M. C. Brandes, M. J. Mills, M. P. Miller *et al.*, “High-energy diffraction microscopy at the advanced photon source,” *Jom*, vol. 63, pp. 70–77, 2011. [Page 16.]
- [24] A. M. Maiden and J. M. Rodenburg, “An improved ptychographical phase retrieval algorithm for diffractive imaging,” *Ultramicroscopy*, vol. 109, no. 10, pp. 1256–1262, 2009. [Page 16.]
- [25] E. Lancon *et al.*, “A coordinated ecosystem for hl-lhc computing r&d — object storage at bnl,” Presentation, IRIS-HEP R&D Workshop, Nov. 2022, 2022. [Online]. Available: <https://indico.cern.ch/event/1203733/contributions/5110087/attachments/2543000/4378629/BNL-IRIS-HEP-Nov-2022.pdf> [Page 16.]
- [26] STFC Scientific Computing Department. (2025) ECHO: Ceph-backed object storage—100 pb capacity. [Online]. Available: <https://www.sc.stfc.ac.uk/platforms-and-services/echo/> [Page 16.]
- [27] P. Schwan *et al.*, “Lustre: Building a file system for 1000-node clusters,” in *Proceedings of the 2003 Linux symposium*, vol. 2003, 2003, pp. 380–386. [Page 17.]
- [28] F. Schmuck and R. Haskin, “{GPFS}: A {Shared-Disk} file system for large computing clusters,” in *Conference on file and storage technologies (FAST 02)*, 2002. [Pages 17 and 18.]
- [29] J. L. Bez, A. M. Karimi, A. K. Paul, B. Xie, S. Byna, P. Carns, S. Oral, F. Wang, and J. Hanley, “Access patterns and performance behaviors of multi-layer supercomputer i/o subsystems under production load,” in *Proceedings of the 31st International Symposium on High-Performance Parallel and Distributed Computing*, 2022, pp. 43–55. [Page 17.]
- [30] S. Saini, J. Rappleye, J. Chang, D. Barker, P. Mehrotra, and R. Biswas, “I/o performance characterization of lustre and nasa applications on pleiades,” in *2012 19th International Conference on High Performance Computing*, 2012. doi: 10.1109/HiPC.2012.6507507 pp. 1–10. [Page 18.]

- [31] K. S. Hemmert, M. Rajan, R. J. Hoekstra, S. Dawson, M. Vigil, D. Grunau, J. Lujan, D. Morton, H. A. Nam, P. J. Peltz *et al.*, “Trinity: Architecture and early experience.” in *SAND2016-4315C 639554*. Sandia National Lab. (SNL-NM), Albuquerque, NM (United States), 05 2016. [Online]. Available: <https://www.osti.gov/biblio/1367056> [Page 18.]
- [32] G. K. Lockwood, A. Chiusole, L. Gerhardt, K. Lozinskiy, D. Paul, and N. J. Wright, “Architecture and performance of perlmutter’s 35 pb clusterstor e1000 all-flash file system,” *Concurrency and Computation: Practice and Experience*, vol. 36, no. 23, p. e8143, 2024. [Page 18.]
- [33] S. Oral, D. A. Dillow, D. Fuller, J. Hill, D. Leverman, S. S. Vazhkudai, F. Wang, Y. Kim, J. Rogers, J. Simmons *et al.*, “Olcfs 1 tb/s, next-generation lustre file system,” in *Proceedings of Cray User Group Conference (CUG 2013)*, 2013, pp. 1–12. [Page 18.]
- [34] J. G. Mikhail Pershin, “Small File I/O Performance in Lustre,” Intel HPDD LUG Presentation, 2018, intel slides. [Online]. Available: https://wiki.lustre.org/images/c/c5/LUG2018-Small_File_IO_Perf_DataOnMDT-Gmitter.pdf [Pages 18 and 25.]
- [35] H. Dai, Y. Wang, K. B. Kent, L. Zeng, and C. Xu, “The state of the art of metadata managements in large-scale distributed file systems — scalability, performance and availability,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 3850–3869, 2022. doi: 10.1109/TPDS.2022.3170574 [Page 19.]
- [36] P. Carns, S. Lang, R. Ross, M. Vilayannur, J. Kunkel, and T. Ludwig, “Small-file access in parallel file systems,” in *2009 IEEE International Symposium on Parallel & Distributed Processing*. IEEE, 2009, pp. 1–11. [Page 19.]
- [37] M. Howison, “Tuning hdf5 for lustre file systems,” 2010. [Pages 19 and 25.]
- [38] M. Könnecke, F. A. Akeroyd, H. J. Bernstein, A. S. Brewster, S. I. Campbell, B. Clausen, S. Cottrell, J. U. Hoffmann, P. R. Jemian, D. Männicke *et al.*, “The nexus data format,” *Applied Crystallography*, vol. 48, no. 1, pp. 301–305, 2015. [Page 19.]

- [39] M. S. Breitenfeld, “Parallel i/o with hdf5 and performance tuning techniques,” Presentation Slides, 2022, available from the HDF Group (<https://portal.hdfgroup.org>). [Page 19.]
- [40] B. Xie, H. Tang, S. Byna, J. Hanley, Q. Koziol, T. Li, and S. Oral, “Battle of the defaults: Extracting performance characteristics of hdf5 under production load,” in *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2021. doi: 10.1109/CCGrid51090.2021.00015 pp. 51–60. [Page 19.]
- [41] Amazon Web Services, “Amazon S3 update – strong read-after-write consistency,” <https://aws.amazon.com/blogs/aws/amazon-s3-update-strong-read-after-write-consistency/>, 2020, accessed 31 May 2025. [Page 20.]
- [42] —, “Amazon S3 pricing,” <https://aws.amazon.com/s3/pricing/>, 2025, accessed 31 May 2025. [Page 20.]
- [43] —, “Best practices for amazon S3 performance,” <https://docs.aws.amazon.com/AmazonS3/latest/userguide/optimizing-performance.html>, 2024, accessed 31 May 2025. [Page 21.]
- [44] —, “Amazon EFS performance,” <https://docs.aws.amazon.com/efs/latest/ug/performance.html>, 2025, accessed 31 May 2025. [Page 21.]
- [45] —, “Uploading objects,” <https://docs.aws.amazon.com/AmazonS3/latest/userguide/upload-objects.html>, 2025, accessed 31 May 2025. [Page 21.]
- [46] —, “Downloading objects using range GETs,” <https://docs.aws.amazon.com/AmazonS3/latest/userguide/download-objects.html>, 2025, accessed 31 May 2025. [Page 21.]
- [47] C. Durbin, P. Quinn, and D. Shum, “Task 51 – cloud-optimized format study,” NASA Goddard Space Flight Center, Greenbelt, MD, United States, Technical Note GSFC-E-DAA-TN77973, Jan. 2020. [Online]. Available: <https://ntrs.nasa.gov/citations/20200001178> [Pages 22 and 34.]
- [48] TileDB, Inc. (2025) Data layout: Columnar attributes, offsets, and coordinates. [Online]. Available: <https://cloud.tiledb.com/academy/structure/arrays/foundation/key-concepts/storage/data-layout/> [Pages 24 and 27.]

- [49] D. Kang, O. Rübel, S. Byna, and S. Blanas, “Predicting and comparing the performance of array management libraries,” in *2020 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2020, pp. 906–915. [Pages 24 and 34.]
- [50] D. Vohra, “Apache parquet,” in *Practical Hadoop Ecosystem: A Definitive Guide to Hadoop-Related Frameworks and Tools*. Springer, 2016, pp. 325–335. [Pages 24 and 27.]
- [51] TileDB, Inc. (2025) Consolidate fragment metadata. [Online]. Available: <https://cloud.tiledb.com/academy/structure/arrays/tutorials/advanced/advanced-writes/consolidation/fragment-meta/> [Pages 24 and 28.]
- [52] M. Folk, G. Heber, Q. Koziol, E. Pourmal, and D. Robinson, “An overview of the hdf5 technology suite and its applications,” in *Proceedings of the EDBT/ICDT 2011 Workshop on Array Databases*, ser. AD ’11. New York, NY, USA: Association for Computing Machinery, 2011. doi: 10.1145/1966895.1966900. ISBN 9781450306140 p. 36–47. [Online]. Available: <https://doi.org/10.1145/1966895.1966900> [Pages 24 and 25.]
- [53] J. Readey, “HSDS: A REST service for HDF5,” 2020, white paper. [Online]. Available: https://www.hdfgroup.org/wp-content/uploads/2020/10/HSDS_Rest_Service_HDF5_Readey.pdf [Pages 24, 25, and 34.]
- [54] The HDF Group, “Chunking in HDF5: Design and performance issues,” 2024, accessed 31 May 2025. [Online]. Available: https://support.hdfgroup.org/documentation/hdf5-docs/advanced_topics/chunking_in_hdf5.html [Page 25.]
- [55] —, “Cloud storage options for HDF5,” 2022, blog post, accessed 31 May 2025. [Online]. Available: <https://www.hdfgroup.org/2022/08/08/cloud-storage-options-for-hdf5/> [Page 25.]
- [56] C. Draxl, J. Wang, L. Sheridan, C. Jung, N. Bodini, S. Buckhold, C. T. Aghili, K. Peco, R. Kotamarthi, A. Kumler, C. Phillips, A. Purkayastha, E. Young, E. Rosenlieb, and H. Tinnesand, “WTK-LED: The WIND Toolkit Long-Term Ensemble Dataset,” National Renewable Energy Laboratory, Golden, CO, Technical Report NREL/TP-5000-88457, Oct. 2024, contract No. DE-AC36-08GO28308; Operated by the Alliance for Sustainable Energy, LLC; Prepared for the U.S.

- Department of Energy, Office of Energy Efficiency & Renewable Energy, Wind Energy Technologies Office. [Online]. Available: <https://docs.nrel.gov/docs/fy25osti/88457.pdf> [Page 25.]
- [57] Zarr Developers, “Zarr Core Specification, version 3,” <https://zarr-specs.readthedocs.io/en/latest/v3/core/index.html>, 2025, accessed 31 May 2025. [Page 26.]
- [58] GDAL Project, “Zarr driver—gdal documentation,” <https://gdal.org/en/stable/drivers/raster/zarr.html>, 2024, accessed 31 May 2025. [Page 26.]
- [59] Amazon Web Services, “Amazon athena – pricing,” 2025, “Save up to 90 % per query and improve performance by converting data to columnar formats”, accessed 1 June 2025. [Online]. Available: <https://aws.amazon.com/athena/pricing/> [Page 27.]
- [60] Apache Iceberg Community, “Iceberg table format — specification v1,” <https://iceberg.apache.org/spec/>, 2025, 1 June 2025. [Pages 27 and 128.]
- [61] pandas developers, “pandas.read_parquet — api reference,” https://pandas.pydata.org/docs/reference/api/pandas.read_parquet.html, 2025, pandas built-in Parquet I/O; accessed 1 June 2025. [Page 27.]
- [62] Google Cloud, “Loading parquet data from cloud storage into bigquery,” 2025, accessed 1 June 2025. [Online]. Available: <https://cloud.google.com/bigquery/docs/loading-data-cloud-storage-parquet> [Page 27.]
- [63] Amazon Web Services, “Using the parquet format in aws glue,” <https://docs.aws.amazon.com/glue/latest/dg/aws-glue-programming-etl-format-parquet-home.html>, 2025, accessed 1 June 2025. [Page 27.]
- [64] Amazon Web Services, Inc., “Use columnar storage formats,” Amazon Athena User Guide, AWS Documentation, 2025, accessed 1 June 2025. [Online]. Available: <https://docs.aws.amazon.com/athena/latest/ug/using-columnar-formats.html> [Page 27.]
- [65] S. Papadopoulos, K. Datta, S. Madden, and T. Mattson, “The tiledb array data storage manager,” in *Proceedings of the VLDB Endowment*, vol. 10, no. 4, 2016, pp. 349–360. [Online]. Available: https://people.csail.mit.edu/stavrosp/papers/vldb2017/VLDB17_TileDB.pdf [Page 27.]

- [66] TileDB, Inc. (2025) Backends: S3, azure blob, gcs, minio. [Online]. Available: <https://docs.tiledb.com/main/how-to/backends> [Page 27.]
- [67] ——. (2025) Amazon s3 backend: Multipart append and configuration. [Online]. Available: <https://docs.tiledb.com/main/how-to/backends/s3> [Page 27.]
- [68] ——. (2025) Fragment metadata: Non-empty domains, tile stats, offsets. [Online]. Available: <https://cloud.tiledb.com/academy/structure/arrays/foundation/key-concepts/storage/fragment-metadata/> [Pages 27 and 28.]
- [69] ——. (2025) Tiling sparse arrays: Mbr indexing and pruning. [Online]. Available: <https://tiledb-inc-tiledb.readthedocs-hosted.com/en/1.6.3/tutorials/tiling-sparse.html> [Page 28.]
- [70] ——. (2025) Reading arrays: Row-/col-major and global/hilbert orders. [Online]. Available: <https://tiledb-inc-tiledb.readthedocs-hosted.com/en/1.6.3/tutorials/reading.html> [Page 28.]
- [71] ——. (2025) Arrays: Time traveling by timestamp range. [Online]. Available: <https://cloud.tiledb.com/academy/structure/arrays/tutorials/advanced/advanced-reads/time-traveling/> [Page 28.]
- [72] ——. (2025) Compression options: Gzip, zstd, lz4, rle, bzip2, double-delta. [Online]. Available: <https://tiledb-inc-tiledb.readthedocs-hosted.com/en/1.6.3/tutorials/compression.html> [Page 28.]
- [73] ——. (2025) Checksum filters: Md5 and sha-256. [Online]. Available: https://www.rdocumentation.org/packages/tiledb/versions/0.30.2/topics/tiledb_filter [Page 28.]
- [74] ——. (2025) Encryption: Aes-256-gcm for arrays at rest. [Online]. Available: <https://docs.tiledb.com/main/background/internal-mechanics/encryption> [Page 28.]
- [75] J. Blomer, P. Canal, A. Naumann, and D. Piparo, “Evolution of the root tree i/o,” in *EPJ Web of Conferences*, vol. 245. EDP Sciences, 2020, p. 02030. [Page 28.]
- [76] ROOT, “Rntuple code architecture,” CERN, 2025, accessed 1 June 2025. [Online]. Available: https://root.cern.ch/doc/master/md_tree_2ntuple_2v7_2doc_2Architecture.html [Page 29.]

- [77] J. Blomer, P. Canal, F. de Geus, J. Hahnfeld, A. Naumann, J. Lopez-Gomez, G. L. Miotto, and V. E. Padulano, “Root’s rntuple i/o subsystem: The path to production,” in *EPJ Web of Conferences*, vol. 295. EDP Sciences, 2024, p. 06020. [Page 29.]
- [78] P. Deutsch, “Gzip file format specification version 4.3,” RFC 1952, 1996. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc1952> [Page 32.]
- [79] X. Delaunay, A. Courtois, and F. Gouillon, “Evaluation of lossless and lossy algorithms for the compression of scientific datasets in netcdf-4 or hdf5 files,” *Geoscientific Model Development*, vol. 12, no. 9, pp. 4099–4113, 2019. [Pages 32 and 34.]
- [80] X. Chen, J. Tian, I. Beaver, C. Freeman, Y. Yan, J. Wang, and D. Tao, “Fcbench: Cross-domain benchmarking of lossless compression for floating-point data,” *arXiv preprint arXiv:2312.10301*, 2023. [Pages 32, 33, and 34.]
- [81] LZ4 Project, “Lz4: A lossless compression algorithm,” 2025, accessed 7 June 2025. [Online]. Available: <https://lz4.org/> [Page 33.]
- [82] M. Bartík, S. Ubik, and P. Kubalik, “Lz4 compression algorithm on fpga,” in *2015 IEEE International Conference on Electronics, Circuits, and Systems (ICECS)*, 2015. doi: 10.1109/ICECS.2015.7440278 pp. 179–182. [Page 33.]
- [83] S. Almeida, V. Oliveira, A. Pina, and M. Melle-Franco, “Two high-performance alternatives to zlib scientific-data compression,” in *Computational Science and Its Applications–ICCSA 2014: 14th International Conference, Guimarães, Portugal, June 30–July 3, 2014, Proceedings, Part IV 14*. Springer, 2014, pp. 623–638. [Pages 33 and 34.]
- [84] Y. Collet and M. Kucherawy, “Zstandard compression and the ‘application/zstd’ media type,” RFC 8878, 2018. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8878> [Page 33.]
- [85] Y. Collet, “zstd(1) manual page, version 1.5.6,” <https://manpages.debian.org/testing/zstd/zstd.1.en.html>, 2025, accessed 7 June 2025. [Page 33.]
- [86] L. Promberger, R. Schwemmer, and H. Fröning, “Characterization of data compression across cpu platforms and accelerators,” *Concurrency*

and Computation: Practice and Experience, vol. 35, no. 20, p. e6465, 2023. [Page 34.]

- [87] The HDF Group. (2025) Installation using docker with aws s3 storage. Accessed 2025-09-20. [Online]. Available: https://github.com/HDFGroup/hsds/blob/master/docs/docker_install_aws.md [Page 57.]

Appendix A

Supporting materials

A.1 Methodological details and statistical settings

A.1.1 Cloud testbed and storage configuration

Cloud/region. Single **AWS** region with the **S3** bucket co-located (eu-north-1).

Bucket and prefixes. Single bucket `scatterin-thesis` in eu-north-1 with per-format prefixes (**HDF5/HSDS**, Zarr, TileDB, **ROOT/TTree**, Parquet) as summarized in Appendix A.1.7.

Networking. Private **S3** via Gateway VPC endpoint with no CloudFront, no Transfer Acceleration, no cross-region traffic.

Client node. One **EC2** `c6i.8xlarge` instance for ingestion and reads, running Amazon Linux 2023 (`al2023-ami-2023.8.20250707.0-kernel-6.1-x86_64`, `ami-09278528675a8d54e`) on private IPv4.

Connections. Persistent **HTTP** keep-alive and connection pooling with default retry policy.

HSDS configuration. **HSDS** ran as a Docker service on the client node and wrote to `scatterin-thesis`. Key environment variables: `BUCKET_NAME=scatterin-thesis`, `AWS_REGION=eu-north-1`, `AWS_S3_GATEWAY=http://s3.amazonaws.com`, `SN_PORT=5101`, `HSDS_ENDPOINT=http://localhost:5101`, `LOG_LEVEL=INFO`.

A.1.2 Run protocol and quality control

Sessions. One per {format, codec, pattern} with no overlap across formats.

Warm-ups and repetitions. 5 warm-ups, then $n=50$ timed runs per cell with idle gap ≥ 2 min before timing.

Cache regime. No explicit cache-clearing between repetitions (no OS page-cache drops and no manual library-cache resets). Timed runs therefore represent warm, within-session behavior after warm-ups; absolute latencies are likely optimistic relative to cold-open first-touch reads, especially for metadata-heavy layouts.

Slice warm-ups. Slice frame indices are sampled without replacement, so warm-ups touch distinct frames; warm-up mainly primes connections and metadata rather than the timed pixel payload.

Randomization. Fixed seed with slice pattern uses a precomputed index list; full scan iterates all frames.

Timer scope. First dispatch $\rightarrow$ last byte; includes transfer, decompression, client decode.

Inclusion rules. Warm-ups excluded; only successful reads included; failures retried.

QC. Q–Q on $\ln(\text{time})$, lag-1 autocorrelation, run-order trend. See Figures [A.1](#), [A.2](#), [A.3](#), [A.4](#).

A.1.3 Statistical settings

Scale. Analyze $\ln(\text{time})$; report **GM** time with 95% intervals; p_{95} on original scale. For full scans, **GM** time is the primary completion metric, and transferred-byte throughput (GiB/s) is derived from downloaded bytes/time and analyzed on the log scale.

Intervals. Moving-block bootstrap (**MBB**) with $\ell \in \{4, 6, 8\}$, $B=10,000$; report the widest 95% **CI** across ℓ ; sensitivity extends to $\ell=10$.

Tails. Main tables report the empirical (sample) p_{95} with **MBB** CIs (using $\ell \in \{4, 6, 8\}$). Harrell–Davis p_{95} with **MBB** CIs over $\ell \in \{4, 6, 8, 10\}$ is reported as a tail-robustness sensitivity check.

Contrasts. Descriptive, session-conditional; ratios reference Zarr within codec $\times$ pattern.

A.1.4 Instrumentation and metrics

Source. Amazon **S3** server-side request metrics in CloudWatch at 1-minute periods, scoped to bucket and session prefixes. **Inventory.** Stored bytes and

object counts per {format, codec} were obtained from **S3** inventory listings for the benchmark bucket and prefixes.

Identifiers and statistics

- **GetRequestsSum**: `GetRequests`, statistic Sum (Count).
- **BytesDownloadedSum**: `BytesDownloaded`, statistic Sum (Bytes).
- **FirstByteLatencyAverage/P95**: `FirstByteLatency`, statistics Average/p95 (ms).
- **TotalRequestLatencyAverage/P95**: `TotalRequestLatency`, statistics Average/p95 (ms).

Derivations used

$$\text{GETs/run} = \frac{\text{GetRequestsSum}}{n_{\text{runs}}}, \quad \text{GETs/GiB} = \frac{\text{GetRequestsSum}}{\text{BytesDownloadedSum}/2^{30}},$$

$$\text{Throughput (GiB/s)} = \frac{\text{BytesDownloadedSum}/2^{30}}{\text{total scan time (s)}}$$

(See Table **A.3** and Table **A.4**.)

Client timing logs

Per {format, codec, pattern} cell, the notebook wrote a timing log and a CloudWatch export:

- **Timing logs (client)**. One **JSON** file per cell, with filename pattern `<format>_<codec>_<pattern>_<run_id>.json`. Each file contains an array of objects

```
[ {"i": <rep_index>, "t_total_ns": <int>}, ... ]
```

where `i` is the repetition index (1...50 for timed runs; warm-ups are not logged) and `t_total_ns` is the wall-clock time in nanoseconds for that read.

- **CloudWatch exports (server).** One **JSON** file per cell with aggregated CloudWatch metrics for the same time window, containing the identifiers listed above (GetRequestsSum, BytesDownloadedSum, FirstByteLatencyAverage/P95, TotalRequestLatencyAverage/P95).

A.1.5 Libraries and access paths

Client stack: numpy, boto3/botocore, fsspec/s3fs, zarr, numcodecs (gzip, zstd, lz4/Blosc), h5pyd (**HSDS**), tiledb (TileDB-py), pyarrow/parquet, ROOT/uproot, fabio, tqdm, matplotlib. Access modes: **HSDS** REST endpoint, s3fs URLs, TileDB **S3** VFS.

Versioning. Package versions are pinned in Experiment/requirements.txt; ROOT/PyROOT is installed from the CERN **ROOT** distribution (see note in Experiment/requirements.txt).

A.1.6 Reproducibility and data availability

Repository. The project repository is hosted at <https://github.com/AsperKristiansson/KTH-MSc-Thesis-Computer-Science>.

Public artifacts. Code/notebooks (Experiment/, Report/); seed frames (Experiment/Base Data/); run logs and CloudWatch exports (Experiment/Experiment Result/); derived tables (Experiment/tables/).

Entry points. Dataset synthesis:

Experiment/base_data_generation.ipynb. Ingestion to backends: Experiment/data_generation.ipynb. Analysis/plots: Experiment/plots.ipynb and tables under Experiment/tables/.

Not publicly hosted. The expanded synthetic dataset (~20 GiB) and the ingested **S3** prefixes are not distributed; rerunning timed experiments requires generating and ingesting the dataset into a user-controlled bucket under comparable provisioning (Appendix A.1.1).

Reproducibility scope. Reported tables/figures are exactly reproducible from the published run logs, while timed end-to-end performance on **AWS** is session-conditional and will vary across reruns due to cloud variability, cache state, and provisioning details.

A.1.7 Per-format schema specifics

Common logical shape $(N, 2880, 2880)$, $N=1296$, uint16; one frame per chunk/tile $(1, 2880, 2880)$.

HDF5/HSDS. Domain `/hdf5/{gzip|lz4|zstd}/all_data.h5`; dataset `/data`; compression `{gzip, lz4, zstd}`. Frames were written in batches of 16 into contiguous slices of the dataset using the **HSDS** client with host-level threading for frame decode and I/O.

Zarr. Store `s3://.../zarr/{gzip|lz4|zstd}.zarr`; array `/data`; codec paths: `gzip` via `GzipCodec(level=3)`, `zstd` via `ZstdCodec(level=3)`, and `lz4` via `BloscCodec(cname=lz4, clevel=3)` in the Python client stack. Frames were written in batches of 16 contiguous slices, with threaded frame loading and a single chunk per frame.

TileDB. URI `s3://.../tiledb/{gzip|lz4|zstd}.tdb`; domain `dims file: [0, N-1], y: [0, 2879], x: [0, 2879]`; tile extents $(1, 2880, 2880)$. Writes were grouped into fragments of 16 frames each, so the expected fragment count is $\lceil N/16 \rceil = 81$.

ROOT/TTree. File `s3://.../root/{gzip|lz4|zstd}.root`; `TTree img_tree` with branch `data` $(2880 \times 2880 \text{ uint16})$; one entry per frame. Entries were appended sequentially, one frame per event, with `gzip`, `lz4`, or `zstd` compression.

Parquet (single-object pilot only). Single file; 2880 list-of-2880 uint16 columns; one row per frame; dictionaries disabled; rows appended sequentially with `gzip` or `lz4` compression only.

A.1.8 Data synthesis parameters

Table A.1: Dataset facts and size definitions for the benchmark detector stack. The reported ~ 20 GiB refers to the uncompressed pixel payload B_{raw} , not the on-disk size of **CBF** inputs or the stored size of any backend.

Item	Value
Seed source (raw)	80 proprietary CBF 1.5 seed frames from one detector acquisition (header convention <code>PILATUS_1.2</code> ; detector: Varex XRD-4343CT). The CBF binary section uses byte-offset encoding (<code>x-CBF_BYTE_OFFSET</code>). Seed files are versioned under <code>Experiment/Base Data/</code> in the project repository; the upstream acquisition dataset beyond the seed set is not redistributed.
Sample/feature definition	One frame is one sample; each sample contains $2880 \times 2880 = 8,294,400$ pixel features (<code>uint16</code> , little-endian).
Synthetic expansion	Frames are sampled from the seed set and perturbed with deterministic RNG (below) until $N=1296 \approx 1,300$ frames. Outputs are named <code>slice_{index}.cbf</code> (zero-based indexing).
Per-frame raw size	$B_{\text{frame}} = 2880 \times 2880 \times 2 = 16,588,800$ bytes = 15.82 MiB.
Raw baseline size	$B_{\text{raw}} = N \cdot B_{\text{frame}} = 21,499,084,800$ bytes = 20.02 GiB.
Stored size definition	$B_{f,c}^{\text{stored}}$ is the sum of all S3 object sizes under a <code>format×codec prefix (data+metadata)</code> . Observed $B_{f,c}^{\text{stored}}$ for the four timed backends spans 10.26 GiB–19.17 GiB (Appendix Table A.5); the Parquet pilot is excluded (Appendix A.1.9).
Transferred bytes (full scans)	$B_{f,c}^{\text{dl}}$ is CloudWatch <code>BytesDownloadedSum</code> over the timed full-scan window (Appendix A.1.4); this is the volume term used for reported scan throughput.
Compressibility	Compression is reported as $R_{f,c} = B_{\text{raw}}/B_{f,c}^{\text{stored}}$ (Eq. 5.1); observed ratios for the four timed backends span ≈ 1.04 – ≈ 1.95 depending on codec and backend (Appendix Table A.5).

Seeds/RNG. Global seed 979969114. Python random and NumPy Generator (PCG64) seeded deterministically; per-frame seeds derived from the global seed and frame index.

Augmentation. From 80 real CBF frames: intensity gain $g \in [0.85, 1.15]$; local contrast $c \in [0.9, 1.1]$; spatial jitter with prob 0.8 (integer circular shifts up to ± 16 px); additive Gaussian noise with prob 0.9 ($\sigma \approx 20$ counts); clip to $[0, 65535]$; cast to `uint16`.

Targeting. Target the raw baseline $B_{\text{raw}} \approx 20.02$ GiB by generating $N=1296$ frames; write `slice_{index}.cbf`; ingest to stores.

A.1.9 Parquet pilot notes

This thesis includes Parquet only as a pilot to document one concrete mapping of an n -D detector stack into Parquet and to surface operational pitfalls of a single-object layout on [S3](#). It is not a quantitative benchmark of Parquet as a format family.

Tested layout (single-object pilot).

- One Parquet file per codec (one [S3](#) object); codecs: `gzip` and `lz4` only.
- Schema: 2880 columns `row_0...row_2879`, each `list<fixed_size=2880, uint16>`; one appended row per frame ($N=1296$).
- Row groups: effectively one row group per frame because each write call appended a one-row table (PyArrow defaults).

Observed during pilot.

- Stored size grew to a single ~ 34 GiB object for a ~ 20 GiB raw stack.
- Metadata open required 3 min to 4 min on the [EC2](#) node; repeated attempts to open the `gzip` file caused intermittent instability (including instance crashes).

How to interpret this pilot. The outcomes above are evidence about a *single-file, single-object* Parquet layout for dense image stacks. In contrast, Parquet is commonly deployed as a *partitioned* dataset of many Parquet data files (often managed by table formats such as Iceberg [\[60\]](#)) with tuned file sizing and row-group sizing for scan/filter workloads; such variants were not evaluated here.

A.2 Supplementary tables and figures

Reading note. The main text consolidates the primary performance tables for readability. The supplementary figures below intentionally retain detailed diagnostic, bootstrap, and sensitivity artifacts so that the reported rankings can be audited from the run logs. Readers interested only in the main storage decision can focus on the consolidated tables in Chapter 5 and the decision-oriented synthesis in Chapter 7.

A.2.1 Timing diagnostics (Q–Q and drift)

Table A.2: Summary diagnostics by cell: Shapiro p on $\ln(\text{time})$ and lag-1 autocorrelation ρ_1 .

Cell (format-codec-pattern)	Shapiro p	ρ_1	p95/GM	Spearman p
HDF5/HSDS-gzip-full	3.43e-09	0.66	1.02	0.837
ROOT/TTree-gzip-full	0.305	0.75	1.05	8.09e-06
tiledb-gzip-full	0.067	0.61	1.10	0.531
zarr-gzip-full	0.0571	0.79	1.02	4.63e-15
HDF5/HSDS-gzip-slice	0.00656	-0.12	1.42	0.337
ROOT/TTree-gzip-slice	0.983	0.02	1.16	0.161
tiledb-gzip-slice	0.334	0.03	1.34	0.00376
zarr-gzip-slice	0.773	-0.19	1.23	0.786
HDF5/HSDS-LZ4-full	0.249	0.34	1.01	0.831
ROOT/TTree-LZ4-full	0.0396	0.77	1.03	0.036
tiledb-LZ4-full	0.07	0.69	1.06	0.0948
zarr-LZ4-full	0.311	0.71	1.03	0.689
HDF5/HSDS-LZ4-slice	0.481	0.07	1.23	0.509
ROOT/TTree-LZ4-slice	0.432	0.32	1.16	0.00312
tiledb-LZ4-slice	0.00033	0.05	1.28	0.178
zarr-LZ4-slice	0.0164	-0.09	1.99	0.585
HDF5/HSDS-ZSTD-full	0.000154	0.57	1.01	0.00385
ROOT/TTree-ZSTD-full	0.556	0.60	1.17	0.395
tiledb-ZSTD-full	0.00838	0.75	1.11	0.33
zarr-ZSTD-full	0.0821	0.57	1.02	0.00347
HDF5/HSDS-ZSTD-slice	0.0038	-0.21	1.21	0.625
ROOT/TTree-ZSTD-slice	0.101	-0.17	1.16	0.0225
tiledb-ZSTD-slice	0.339	0.01	1.41	0.0662
zarr-ZSTD-slice	0.00425	0.00	1.26	0.236

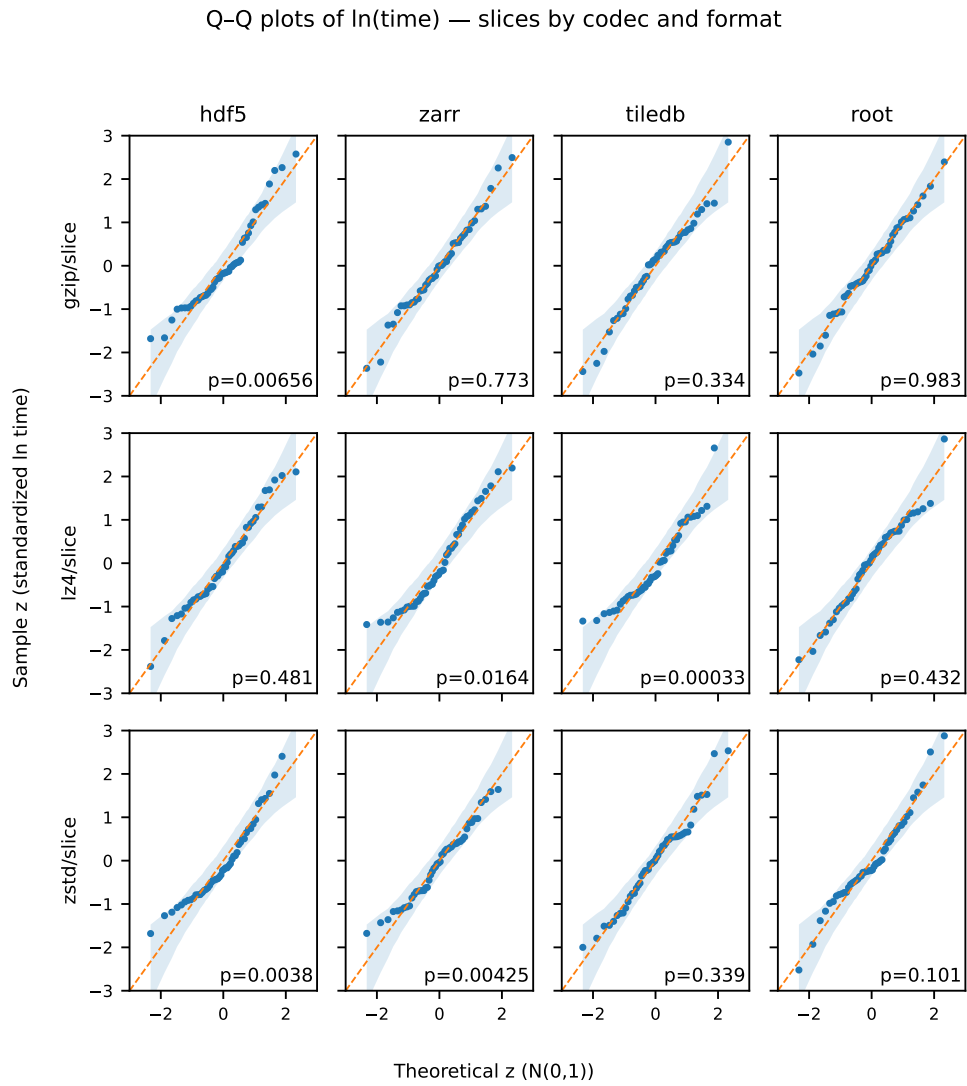


Figure A.1: Q-Q plots of $\ln(\text{time})$ for slice workloads. Rows: codecs {gzip, LZ4, ZSTD}. Columns: formats {HDF5/HSDS, Zarr, TileDB, ROOT/TTree}. Shaded band is a 95% pointwise envelope; per-panel Shapiro p shown.

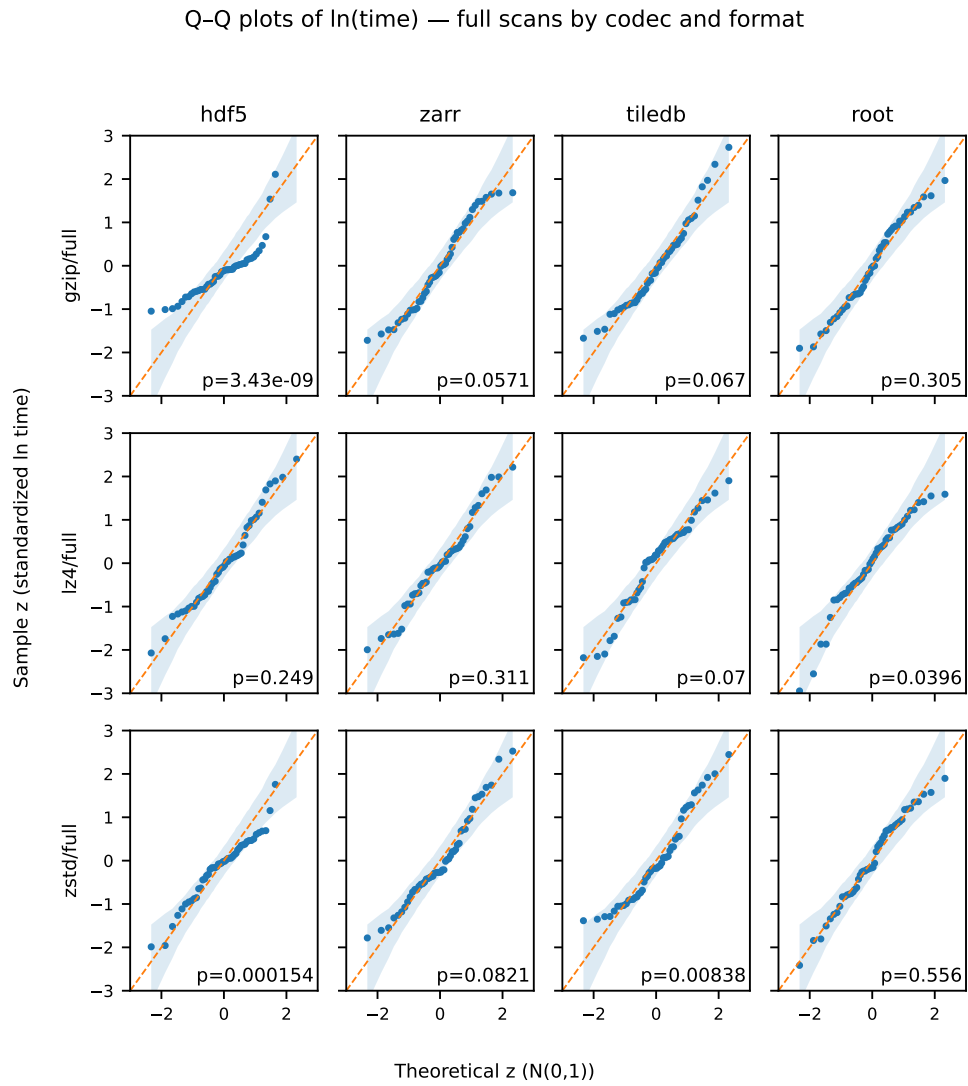


Figure A.2: Q-Q plots of $\ln(\text{time})$ for full-scan workloads. Rows: codecs {gzip, LZ4, ZSTD}. Columns: formats {HDF5/HSDS, Zarr, TileDB, ROOT/TTree}. Shaded band is a 95% pointwise envelope; per-panel Shapiro p shown.

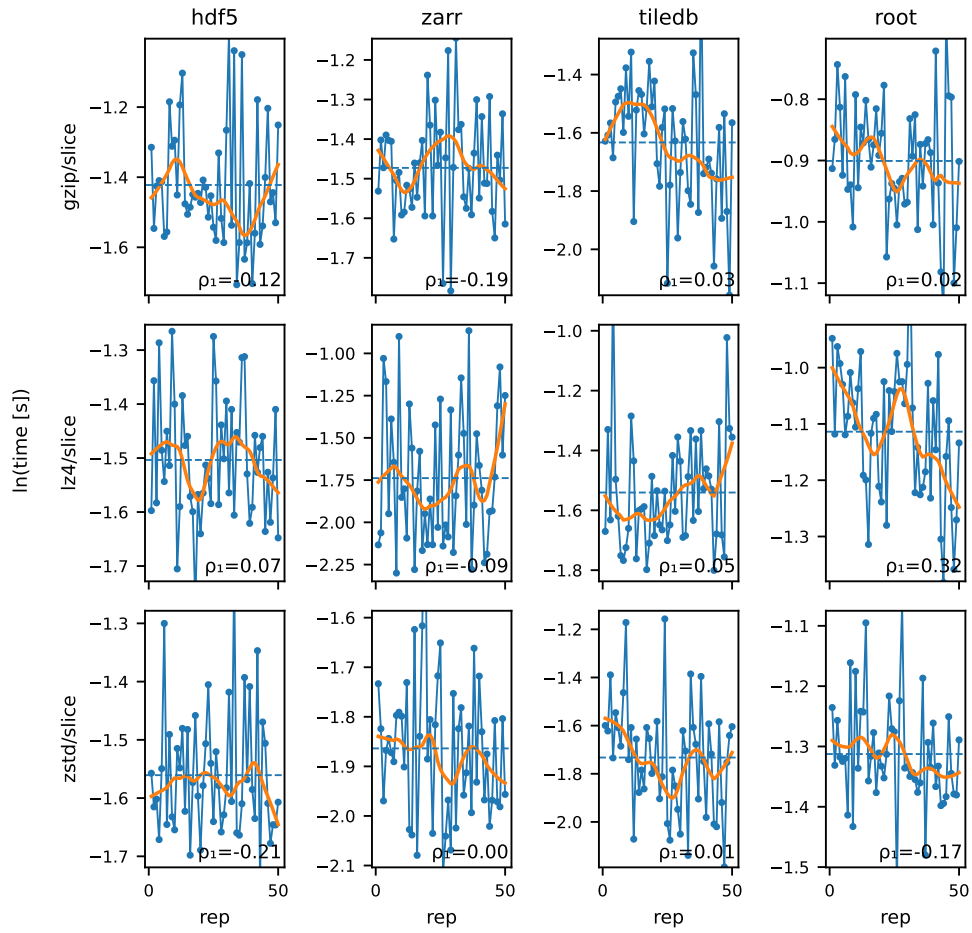
Run-order traces of $\ln(\text{time})$ — slices by codec and format

Figure A.3: Run-order traces of $\ln(\text{time})$ for slice workloads. Rows and columns as in Figure A.1. Solid line: LOWESS smoother; dashed: $\ln$ geometric mean. Per-panel robust y -limits; panel shows lag-1 autocorrelation ρ_1 .

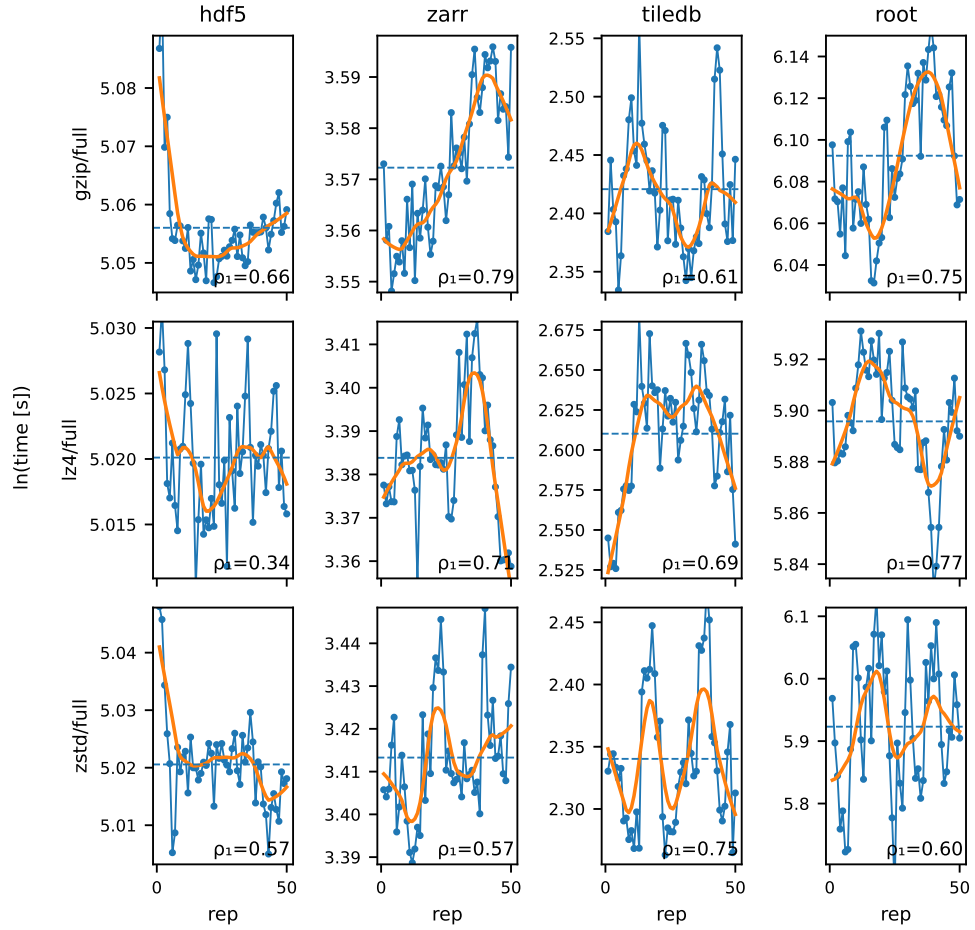
Run-order traces of $\ln(\text{time})$ — full scans by codec and format

Figure A.4: Run-order traces of $\ln(\text{time})$ for full-scan workloads. Rows and columns as in Figure A.2. Solid line: LOWESS smoother; dashed: $\ln$ geometric mean. Per-panel robust y -limits; panel shows lag-1 autocorrelation ρ_1 .

A.2.2 Request intensity and S3 service latencies

Table A.3: S3 request intensity per cell: point estimates and exact Poisson 95% CIs.

Format	Codec	Pattern	GETs/rep [95% CI]	GETs/GiB [95% CI]	G	Bytes (GiB)	n
HDF5/HSDS	gzip	full	1561.06 [1550.13, 1572.05]	137.75 [136.78, 138.72]	78,053	566.634	50
ROOT/TTTree	gzip	full	1321.92 [1311.86, 1332.04]	58.47 [58.03, 58.92]	66,096	1130.349	50
tiledb	gzip	full	1104.42 [1095.23, 1113.67]	94.61 [93.82, 95.40]	55,221	583.670	50
zarr	gzip	full	1320.90 [1310.85, 1331.01]	116.80 [115.91, 117.70]	66,045	565.441	50
HDF5/HSDS	gzip	slice	1.98 [1.61, 2.41]	482.17 [391.88, 587.02]	99	0.205	50
ROOT/TTTree	gzip	slice	1.26 [0.97, 1.61]	61.28 [47.09, 78.41]	63	1.028	50
tiledb	gzip	slice	1.20 [0.92, 1.54]	113.27 [86.44, 145.80]	60	0.530	50
zarr	gzip	slice	1.30 [1.00, 1.66]	126.66 [97.75, 161.43]	65	0.513	50
HDF5/HSDS	LZ4	full	1557.22 [1546.30, 1568.20]	79.64 [79.09, 80.21]	77,861	977.612	50
ROOT/TTTree	LZ4	full	1321.92 [1311.86, 1332.04]	48.71 [48.34, 49.08]	66,096	1356.963	50
tiledb	LZ4	full	1486.92 [1476.25, 1497.65]	93.68 [93.01, 94.36]	74,346	793.618	50
zarr	LZ4	full	1351.58 [1341.41, 1361.81]	124.53 [123.59, 125.47]	67,579	542.664	50
HDF5/HSDS	LZ4	slice	4.80 [4.21, 5.45]	270.49 [237.35, 306.96]	240	0.887	50
ROOT/TTTree	LZ4	slice	1.26 [0.97, 1.61]	51.06 [39.23, 65.32]	63	1.234	50
tiledb	LZ4	slice	1.20 [0.92, 1.54]	83.21 [63.50, 107.10]	60	0.721	50
zarr	LZ4	slice	1.30 [1.00, 1.66]	134.99 [104.18, 172.05]	65	0.482	50
HDF5/HSDS	ZSTD	full	1547.32 [1536.44, 1558.26]	142.19 [141.19, 143.19]	77,366	544.117	50
ROOT/TTTree	ZSTD	full	1321.92 [1311.86, 1332.04]	63.18 [62.70, 63.66]	66,096	1046.131	50
tiledb	ZSTD	full	1061.58 [1052.57, 1070.65]	96.89 [96.06, 97.71]	53,079	547.847	50
zarr	ZSTD	full	1320.90 [1310.85, 1331.01]	123.16 [122.23, 124.11]	66,045	536.236	50
HDF5/HSDS	ZSTD	slice	4.82 [4.23, 5.47]	488.01 [428.33, 553.66]	241	0.494	50
ROOT/TTTree	ZSTD	slice	1.26 [0.97, 1.61]	66.20 [50.87, 84.70]	63	0.952	50
tiledb	ZSTD	slice	1.20 [0.92, 1.54]	120.72 [92.12, 155.39]	60	0.497	50
zarr	ZSTD	slice	1.30 [1.00, 1.66]	136.61 [105.49, 174.22]	65	0.476	50

Table A.4: **S3** CloudWatch service latencies per cell (ms). Values summarize 1-minute CloudWatch datapoints within the timed window of each session. Averages are request-weighted across 1-minute periods; p_{95} is the mean of the 1-minute p_{95} datapoints (descriptive; not a true session-wide per-request p_{95}).

Format	Codec	Pattern	FirstByte avg	FirstByte p_{95}	Total avg	Total p_{95}
HDF5/HSDS	gzip	full	46.622	66.833	102.149	132.430
ROOT/TTree	gzip	full	63.206	80.167	204.174	245.199
tiledb	gzip	full	22.614	29.192	116.425	128.060
zarr	gzip	full	24.882	40.508	93.128	103.176
HDF5/HSDS	gzip	slice	75.051	105.347	89.152	135.756
ROOT/TTree	gzip	slice	117.619	140.110	266.603	334.332
tiledb	gzip	slice	62.917	89.851	89.546	239.641
zarr	gzip	slice	87.597	132.875	118.938	181.698
HDF5/HSDS	LZ4	full	45.517	63.953	107.108	137.238
ROOT/TTree	LZ4	full	75.456	95.540	245.737	293.689
tiledb	LZ4	full	22.567	29.158	117.307	126.492
zarr	LZ4	full	24.785	34.685	89.638	96.150
HDF5/HSDS	LZ4	slice	63.621	86.445	77.800	106.719
ROOT/TTree	LZ4	slice	120.524	141.858	294.825	363.410
tiledb	LZ4	slice	57.133	77.672	97.454	259.554
zarr	LZ4	slice	102.016	171.055	161.938	330.963
HDF5/HSDS	ZSTD	full	43.157	60.930	94.571	105.935
ROOT/TTree	ZSTD	full	60.790	79.221	205.670	272.696
tiledb	ZSTD	full	22.754	29.244	113.891	125.190
zarr	ZSTD	full	23.604	32.837	88.483	96.587
HDF5/HSDS	ZSTD	slice	56.191	77.999	65.813	93.934
ROOT/TTree	ZSTD	slice	68.444	85.236	189.476	250.441
tiledb	ZSTD	slice	62.633	106.719	80.901	207.882
zarr	ZSTD	slice	83.932	94.424	133.233	182.695

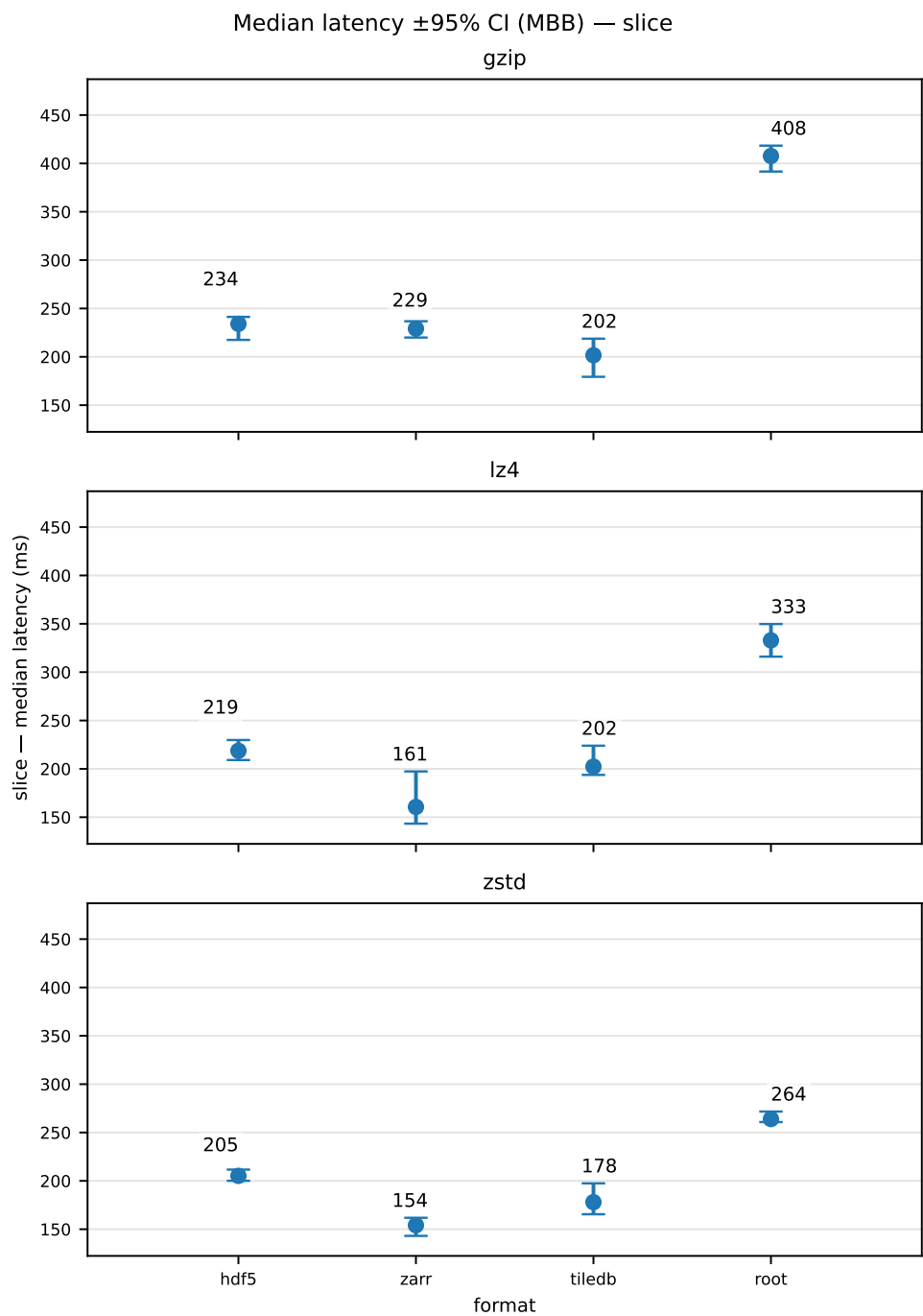


Figure A.5: Slice: median latency $\pm$ 95% CI (MBB); one row with three panels (gzip, LZ4, ZSTD).

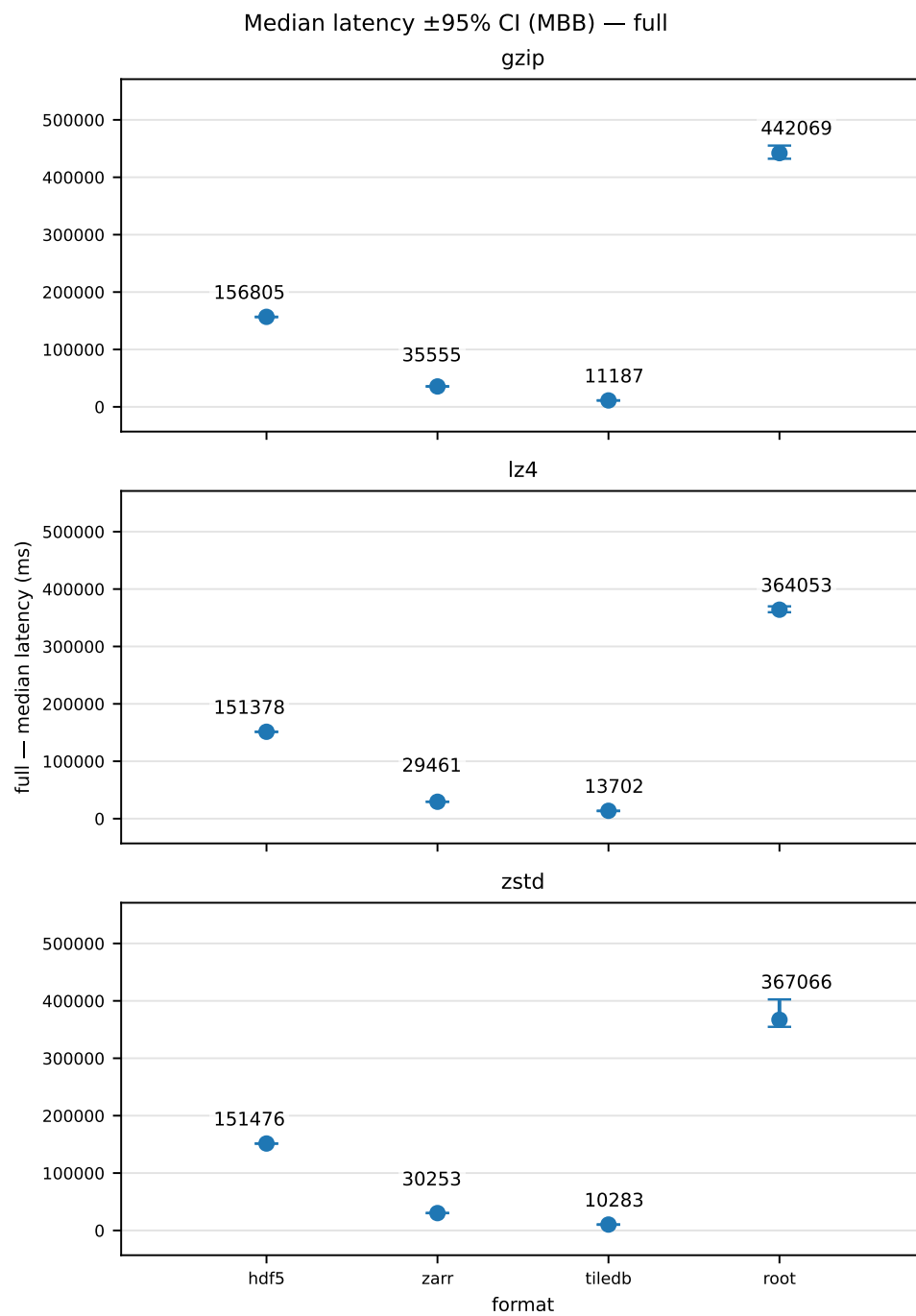


Figure A.6: Full scan: median latency $\pm 95\%$ CI (MBB); one row (columns = gzip, LZ4, ZSTD).

(Appendix) Slice / gzip: **HD** p_{95} with **MBB** 95% **CI**s by block length ℓ .

Format	ℓ	p_{95} (ms)	95% CI lo	95% CI hi	CI width
HDF5/HSDS	4	344.70	304.34	366.43	62.09
ROOT/TTree	4	479.58	456.47	502.96	46.50
TileDB	4	274.99	240.45	330.95	90.50
Zarr	4	293.99	268.92	313.38	44.46
HDF5/HSDS	6	344.70	302.51	366.57	64.06
ROOT/TTree	6	479.58	455.17	503.84	48.67
TileDB	6	274.99	239.68	330.97	91.30
Zarr	6	293.99	268.01	313.72	45.71
HDF5/HSDS	8	344.70	302.25	366.59	64.34
ROOT/TTree	8	479.58	454.89	503.95	49.06
TileDB	8	274.99	240.75	330.89	90.14
Zarr	8	293.99	267.83	314.08	46.25
HDF5/HSDS	10	344.70	302.48	366.70	64.22
ROOT/TTree	10	479.58	453.33	503.94	50.61
TileDB	10	274.99	240.25	330.76	90.51
Zarr	10	293.99	266.30	314.06	47.76

(Appendix) Slice / lz4: **HD** p_{95} with **MBB** 95% **CI**s by block length ℓ .

Format	ℓ	p_{95} (ms)	95% CI lo	95% CI hi	CI width
HDF5/HSDS	4	274.94	256.95	280.35	23.40
ROOT/TTree	4	393.55	373.21	439.37	66.16
TileDB	4	321.57	264.30	407.78	143.48
Zarr	4	370.84	305.30	409.14	103.84
HDF5/HSDS	6	274.94	257.91	280.22	22.31
ROOT/TTree	6	393.55	373.36	438.80	65.45
TileDB	6	321.57	263.41	400.90	137.49
Zarr	6	370.84	301.63	407.59	105.97
HDF5/HSDS	8	274.94	258.75	280.23	21.48
ROOT/TTree	8	393.55	374.39	438.73	64.34
TileDB	8	321.57	262.65	407.79	145.14
Zarr	8	370.84	300.03	406.13	106.10
HDF5/HSDS	10	274.94	261.20	280.08	18.88
ROOT/TTree	10	393.55	375.22	438.54	63.32
TileDB	10	321.57	261.70	407.99	146.29
Zarr	10	370.84	298.23	404.56	106.33

(Appendix) Slice / zstd: **HD** p_{95} with **MBB** 95% **CI**s by block length ℓ .

Format	ℓ	p_{95} (ms)	95% CI lo	95% CI hi	CI width
HDF5/HSDS	4	264.69	242.89	288.24	45.35
ROOT/TTree	4	321.45	298.81	339.12	40.31
TileDB	4	274.79	227.79	307.66	79.87
Zarr	4	206.34	179.83	253.80	73.97
HDF5/HSDS	6	264.69	242.43	286.94	44.51
ROOT/TTree	6	321.45	298.36	338.44	40.08
TileDB	6	274.79	232.27	306.35	74.08
Zarr	6	206.34	179.64	253.85	74.21
HDF5/HSDS	8	264.69	242.79	286.87	44.08
ROOT/TTree	8	321.45	298.49	338.12	39.63
TileDB	8	274.79	233.38	303.42	70.04
Zarr	8	206.34	178.68	253.85	75.18
HDF5/HSDS	10	264.69	243.18	285.72	42.53
ROOT/TTree	10	321.45	298.59	337.31	38.72
TileDB	10	274.79	235.41	302.40	66.99
Zarr	10	206.34	178.90	253.86	74.96

(Appendix) Full / gzip: **HD** p_{95} with **MBB** 95% **CI**s by block length ℓ .

Format	ℓ	p_{95} (ms)	95% CI lo	95% CI hi	CI width
HDF5/HSDS	4	160 558	157 224	162 903	5 679
ROOT/TTree	4	465 371	457 976	469 618	11 642
TileDB	4	12 528	11 843	12 841	998
Zarr	4	36 417	36 153	36 444	291
HDF5/HSDS	6	160 558	157 173	162 903	5 729
ROOT/TTree	6	465 371	456 192	469 620	13 428
TileDB	6	12 528	11 830	12 834	1 004
Zarr	6	36 417	36 059	36 443	384
HDF5/HSDS	8	160 558	157 154	162 905	5 751
ROOT/TTree	8	465 371	453 826	469 620	15 794
TileDB	8	12 528	11 833	12 833	1 000
Zarr	8	36 417	35 955	36 444	489
HDF5/HSDS	10	160 558	157 142	162 905	5 763
ROOT/TTree	10	465 371	451 293	469 620	18 327
TileDB	10	12 528	11 844	12 823	979
Zarr	10	36 417	35 871	36 445	574

(Appendix) Full / 1 z 4: **HD** p_{95} with **MBB** 95% **CI**s by block length ℓ .

Format	ℓ	p_{95} (ms)	95% CI lo	95% CI hi	CI width
HDF5/HSDS	4	152 821	152 328	153 067	738
ROOT/TTree	4	375 321	371 899	376 256	4 357
TileDB	4	14 427	14 130	14 572	442
Zarr	4	30 297	29 811	30 407	597
HDF5/HSDS	6	152 821	152 394	153 060	666
ROOT/TTree	6	375 321	370 722	376 259	5 536
TileDB	6	14 427	14 099	14 579	480
Zarr	6	30 297	29 755	30 408	653
HDF5/HSDS	8	152 821	152 458	153 053	595
ROOT/TTree	8	375 321	369 906	376 272	6 366
TileDB	8	14 427	14 068	14 582	513
Zarr	8	30 297	29 749	30 409	659
HDF5/HSDS	10	152 821	152 552	153 041	490
ROOT/TTree	10	375 321	369 349	376 319	6 970
TileDB	10	14 427	14 065	14 579	513
Zarr	10	30 297	29 738	30 409	670

(Appendix) Full / z std: **HD** p_{95} with **MBB** 95% **CI**s by block length ℓ .

Format	ℓ	p_{95} (ms)	95% CI lo	95% CI hi	CI width
HDF5/HSDS	4	154 112	152 199	155 549	3 350
ROOT/TTree	4	441 405	423 524	453 744	30 219
TileDB	4	11 567	10 956	11 810	854
Zarr	4	31 207	30 815	31 389	573
HDF5/HSDS	6	154 112	152 193	155 549	3 356
ROOT/TTree	6	441 405	424 400	453 657	29 257
TileDB	6	11 567	10 831	11 809	978
Zarr	6	31 207	30 809	31 379	570
HDF5/HSDS	8	154 112	152 178	155 549	3 371
ROOT/TTree	8	441 405	425 586	453 278	27 692
TileDB	8	11 567	10 800	11 809	1 009
Zarr	8	31 207	30 820	31 377	557
HDF5/HSDS	10	154 112	152 192	155 549	3 357
ROOT/TTree	10	441 405	426 816	452 899	26 083
TileDB	10	11 567	10 790	11 809	1 019
Zarr	10	31 207	30 847	31 376	529

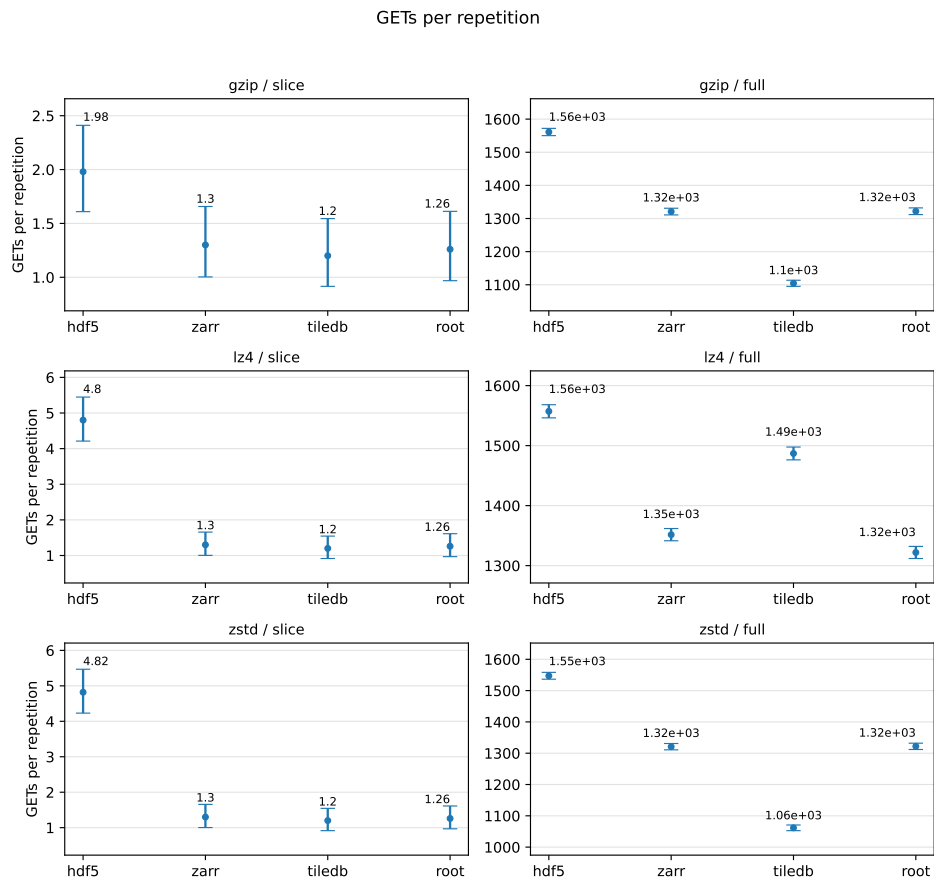


Figure A.7: **S3** request intensity per repetition ($\hat{\lambda}_{\text{rep}}$). Rows: codecs; columns: access patterns. Points are per session; descriptive only.

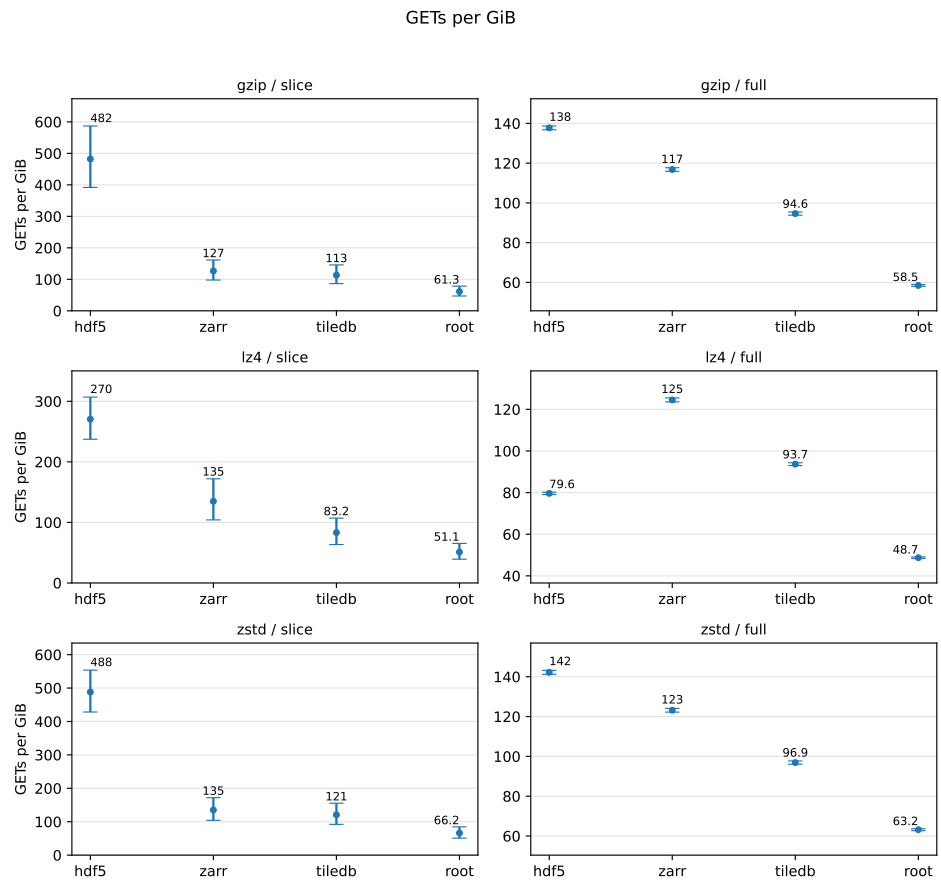


Figure A.8: **S3** request intensity per GiB ($\hat{\lambda}_{\text{GiB}}$). Rows: codecs; columns: access patterns. Points are per session; descriptive only.

A.2.3 Objectization and storage footprint

Table A.5: Object counts by store and codec with stored size and compression ratio $R = B_{\text{raw}}/B^{\text{stored}}$ ($B_{\text{raw}} = 20.02$ GiB; metadata included). **HSDS** sizes tallied from shared db / by dataset UUID.

Format	Codec	Objects	Stored [GiB]	R	Avg [KiB/object]	Layout
HSDS	gzip	5,183	11.11	1.80	2,247.8	shard
HSDS	LZ4	5,183	19.17	1.04	3,878.1	shard
HSDS	ZSTD	5,183	10.67	1.88	2,158.4	shard
ROOT/TTTree	gzip	1	11.09	1.81	11,626,174.2	single-object
ROOT/TTTree	LZ4	1	13.31	1.50	13,956,389.1	single-object
ROOT/TTTree	ZSTD	1	10.26	1.95	10,759,260.2	single-object
TileDB	gzip	244	11.44	1.75	49,183.8	fragment
TileDB	LZ4	244	15.56	1.29	66,874.7	fragment
TileDB	ZSTD	244	10.74	1.86	46,165.2	fragment
Zarr	gzip	1,297	11.09	1.81	8,963.5	chunk-per-object
Zarr	LZ4	1,297	10.40	1.93	8,407.2	chunk-per-object
Zarr	ZSTD	1,297	10.51	1.90	8,500.5	chunk-per-object

S3 FirstByteLatency: avg vs p95

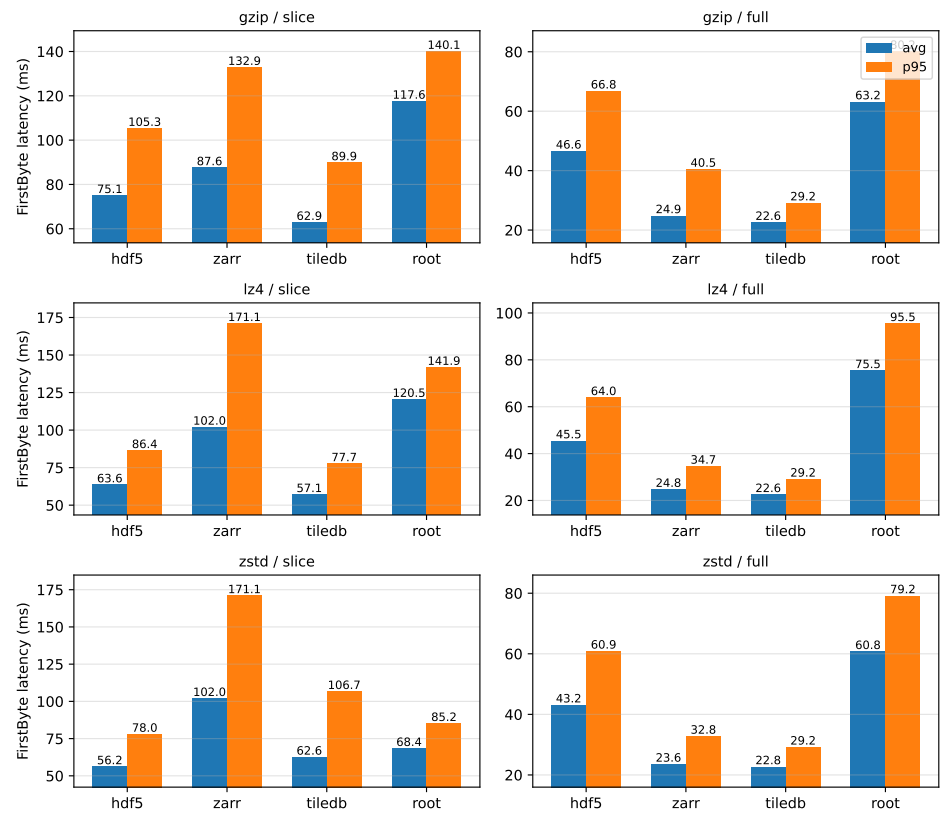


Figure A.9: FirstByteLatency per cell: per-session average (avg) and p_{95} as side-by-side bars. Rows: codecs; columns: patterns. Descriptive only.

S3 TotalRequestLatency: avg vs p95

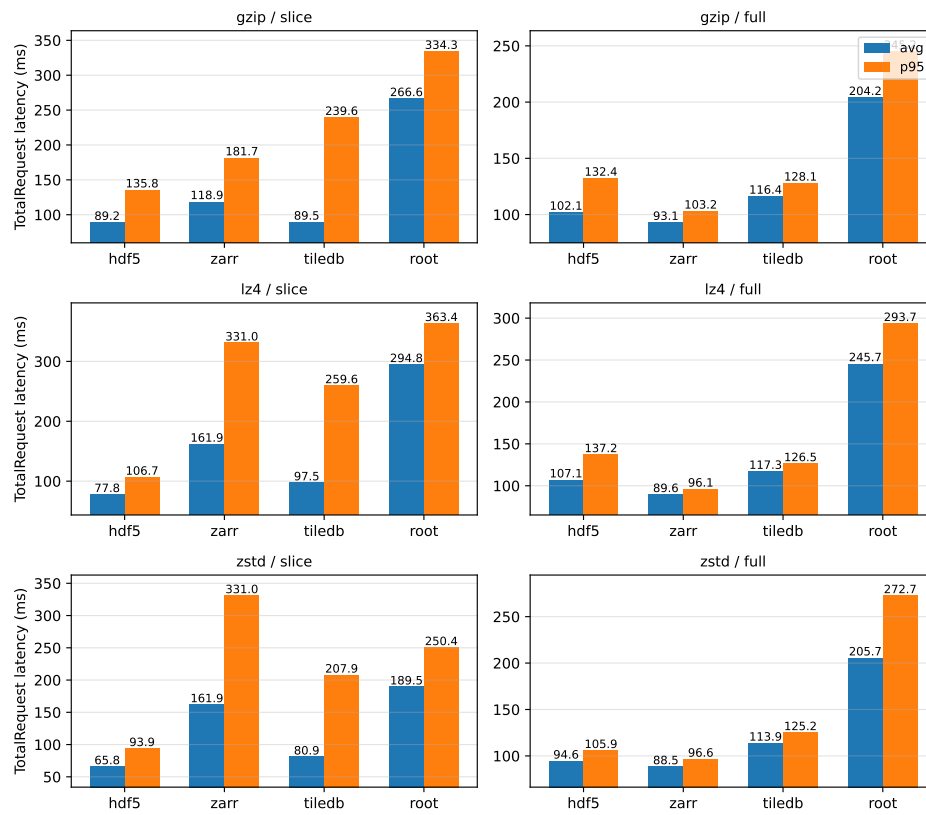


Figure A.10: TotalRequestLatency per cell: per-session average (avg) and p_{95} as side-by-side bars. Rows: codecs; columns: patterns. Descriptive only.

A.2.4 Slice and full latency details

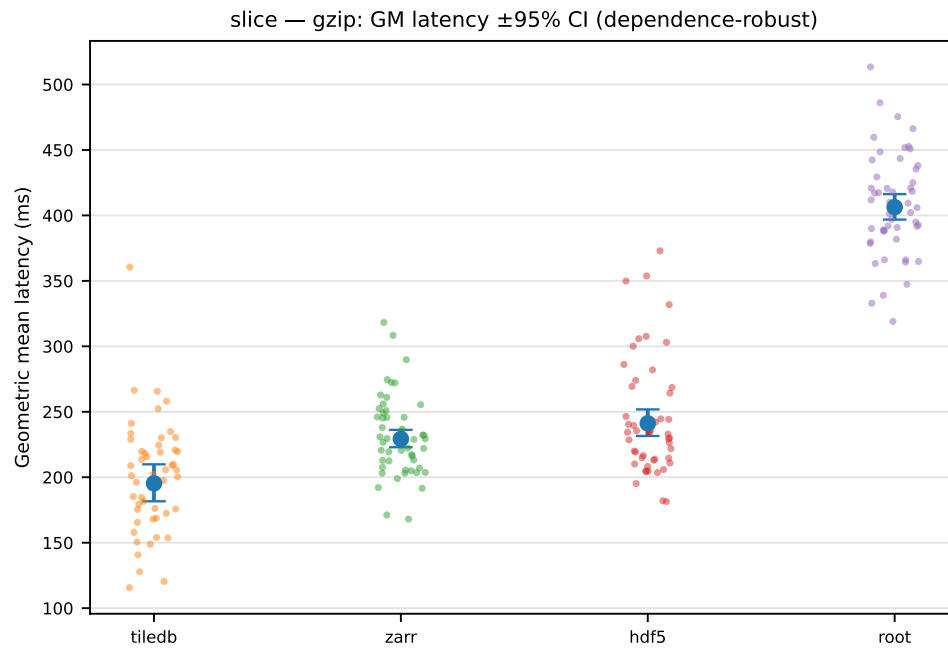


Figure A.11: gzip/slice: geometric mean latency with dependence-robust 95% CIs.

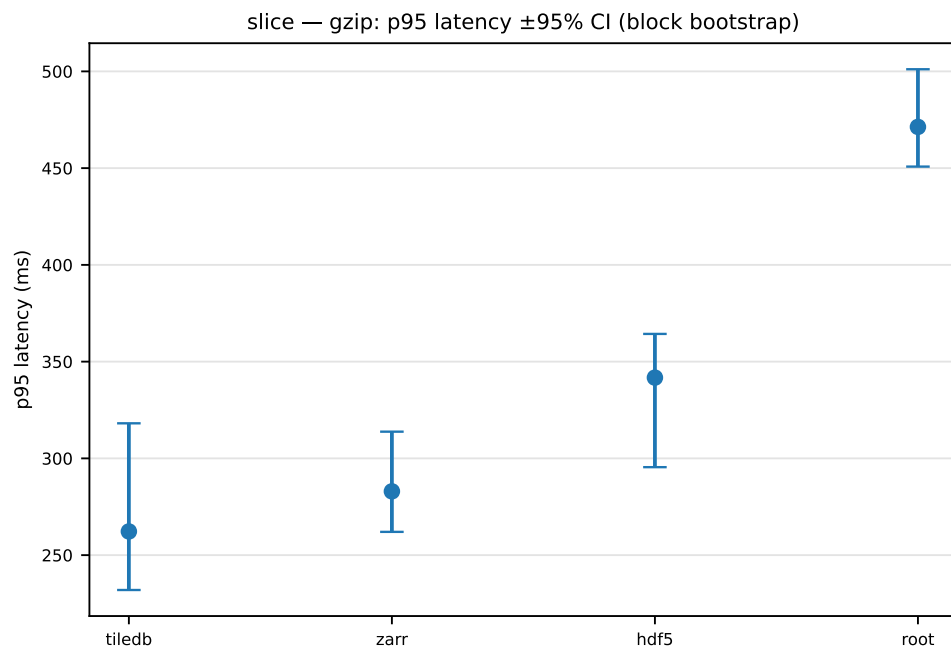


Figure A.12: `gzip/slice`: p_{95} latency with dependence-robust 95% CIs.

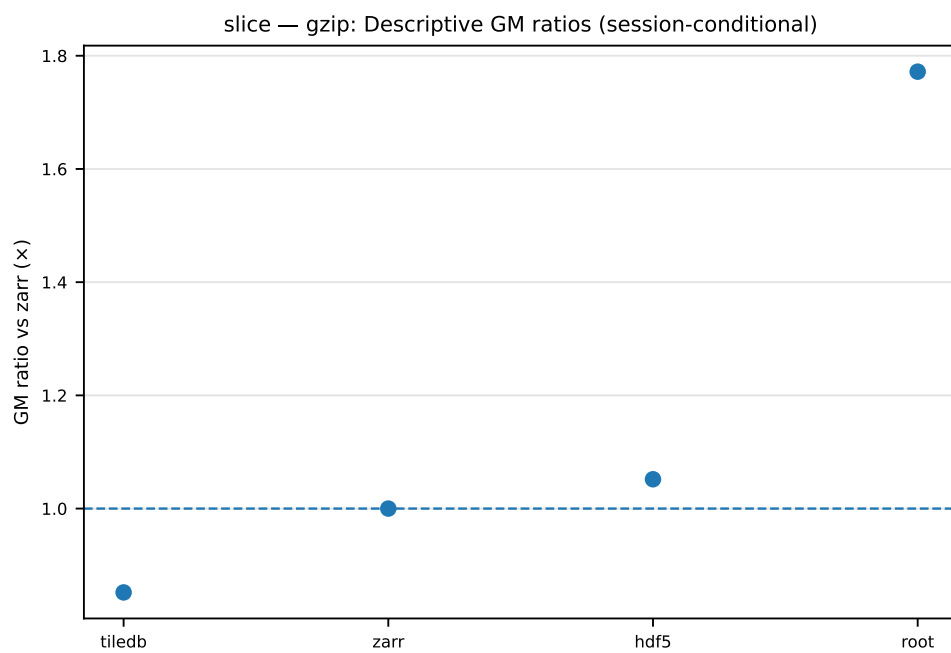


Figure A.13: `gzip/slice`: descriptive **GM** time ratios vs Zarr (session-conditional).

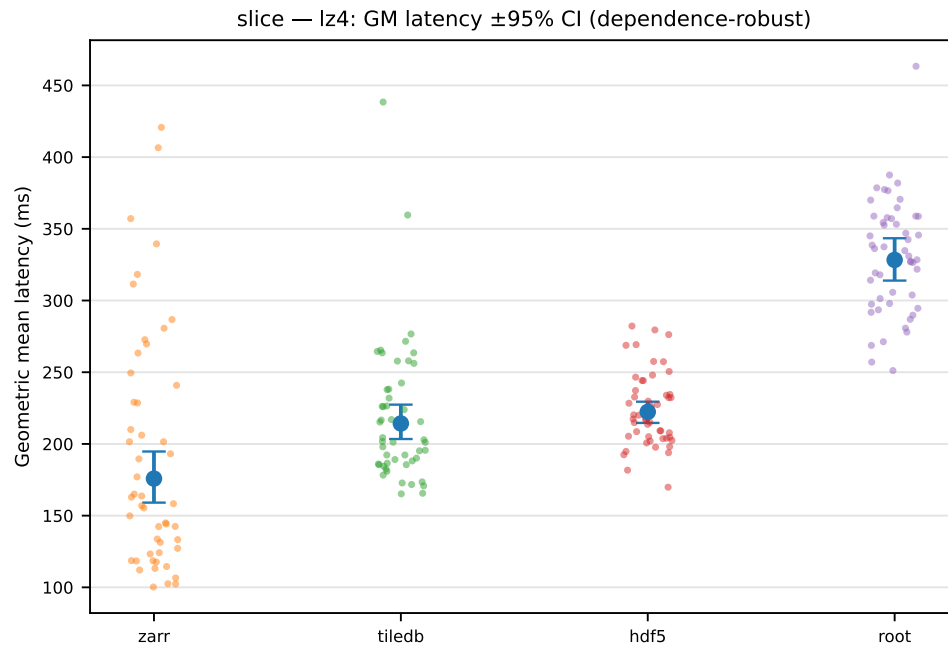


Figure A.14: `lz4/slice`: geometric mean latency with dependence-robust 95% CIs.

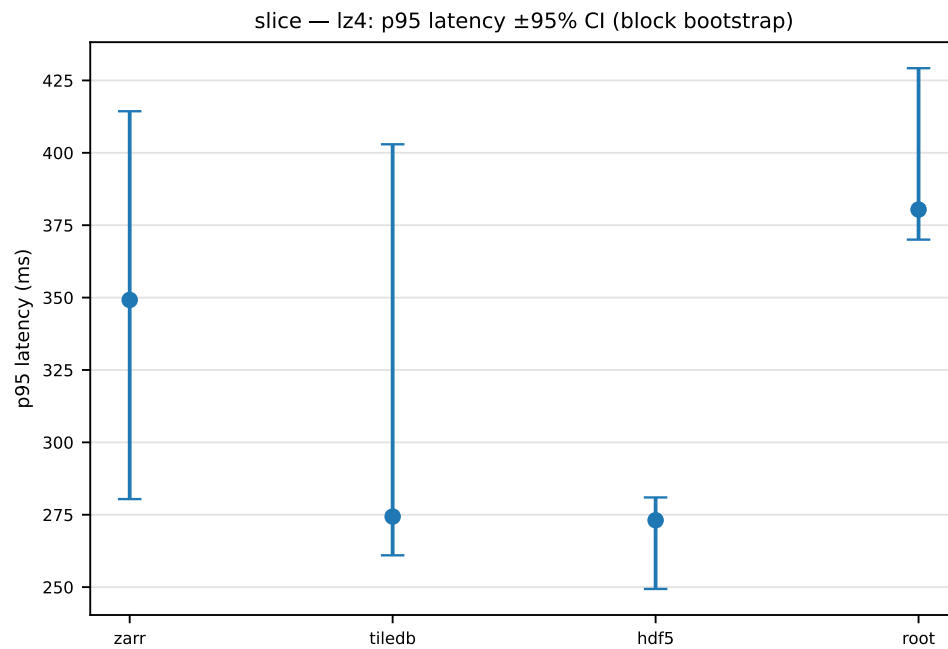


Figure A.15: `lz4/slice`: p₉₅ latency with dependence-robust 95% CIs.

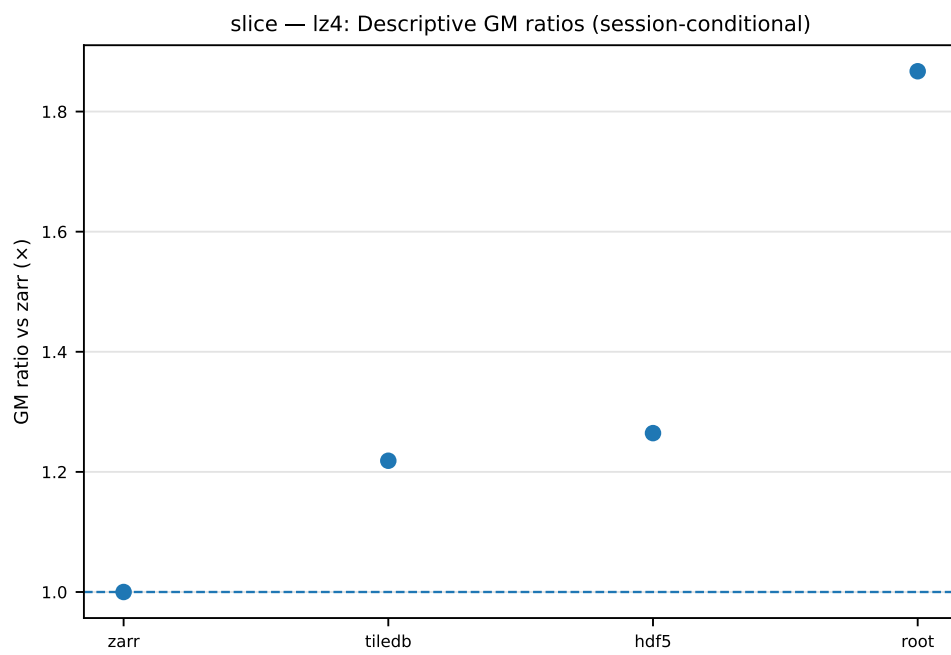


Figure A.16: lz4/slice: descriptive GM time ratios vs Zarr (session-conditional).

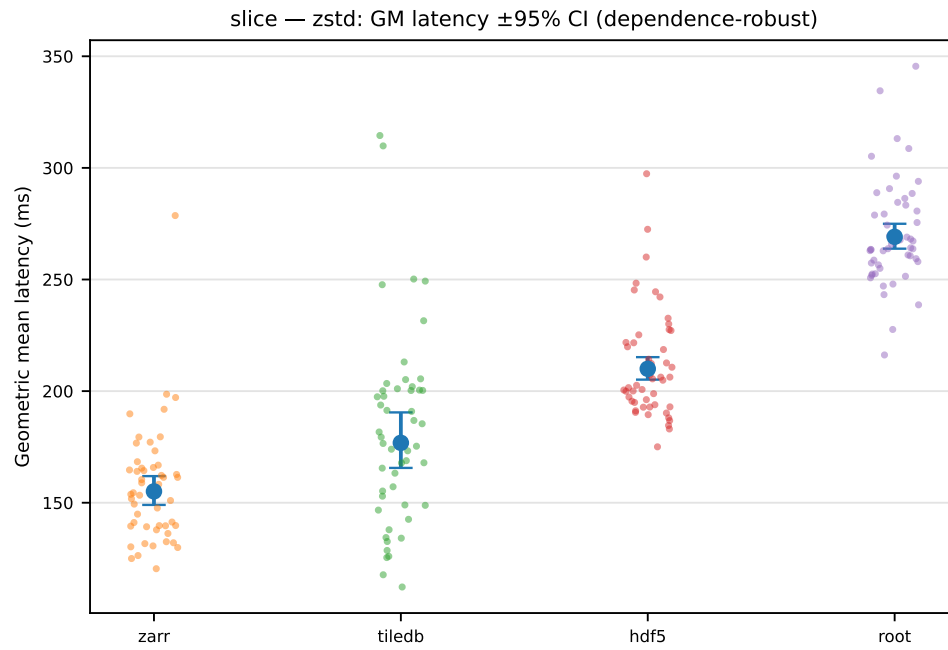


Figure A.17: zstd/slice: geometric mean latency with dependence-robust 95% CIs.

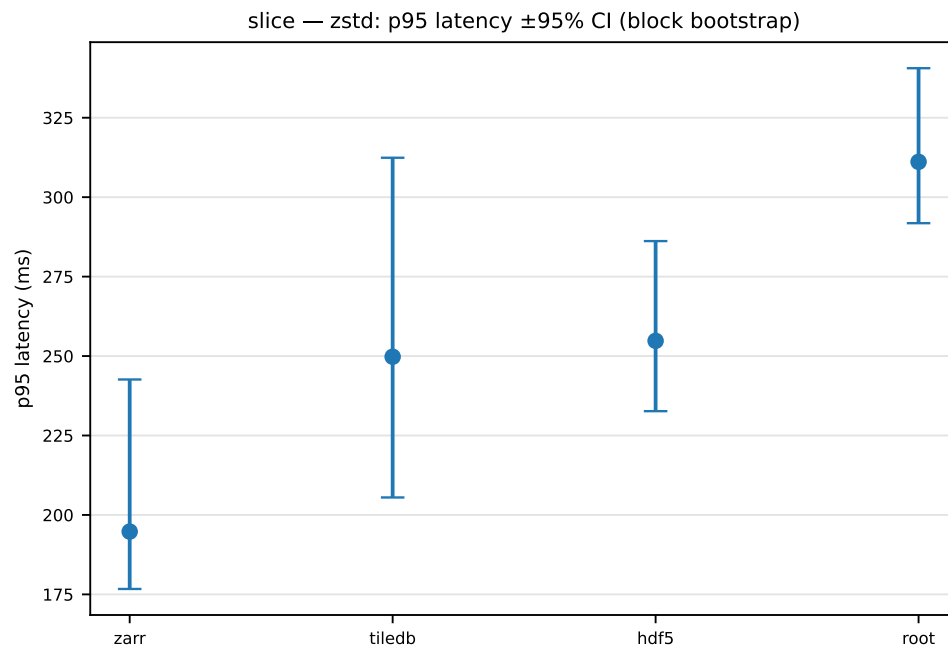


Figure A.18: zstd/slice: p_{95} latency with dependence-robust 95% CIs.

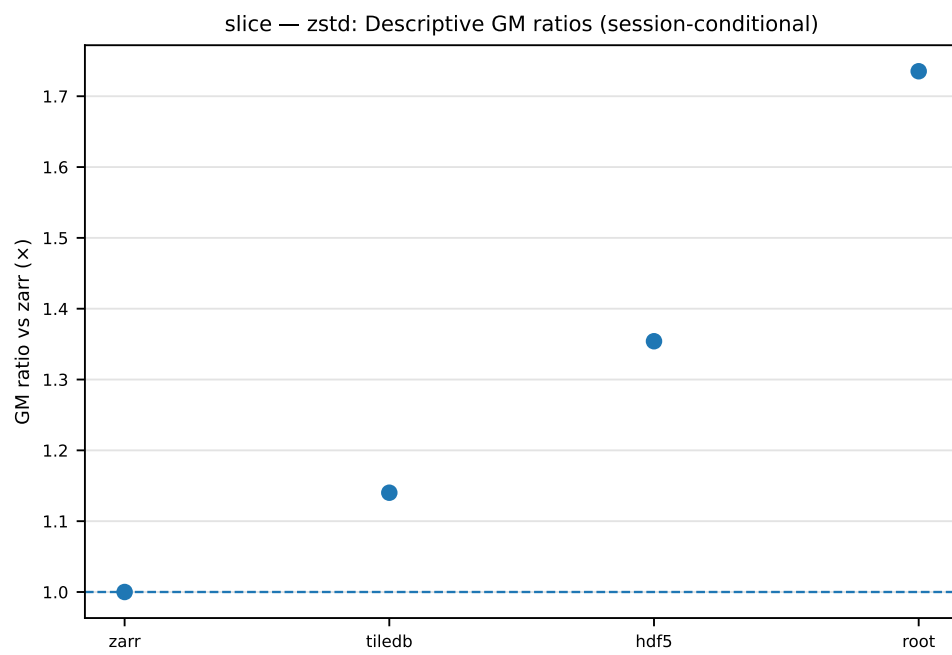


Figure A.19: `zstd/slice`: descriptive **GM** time ratios vs Zarr (session-conditional).

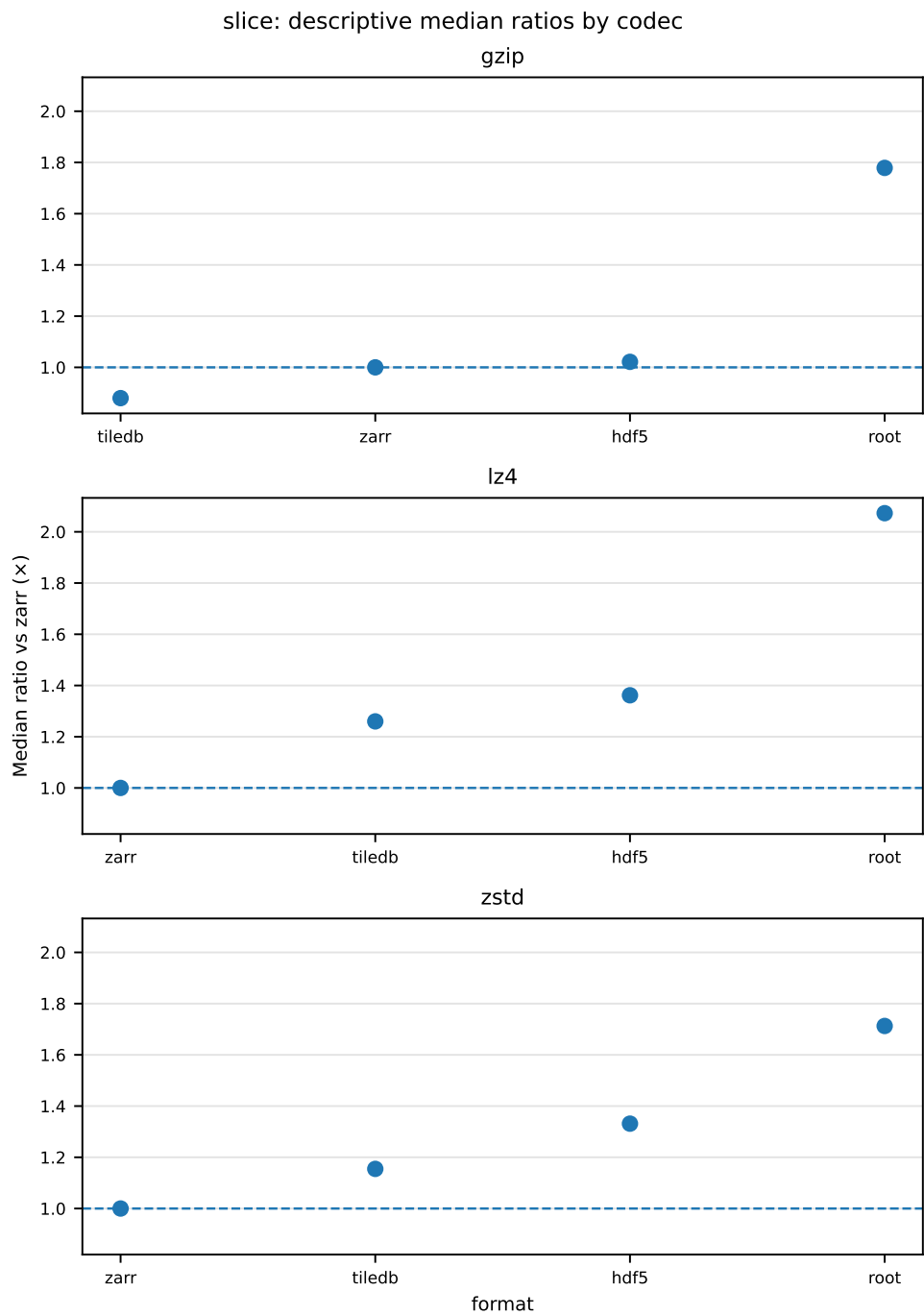


Figure A.20: Slice: descriptive median latency ratios vs Zarr across codecs (gzip, LZ4, ZSTD).

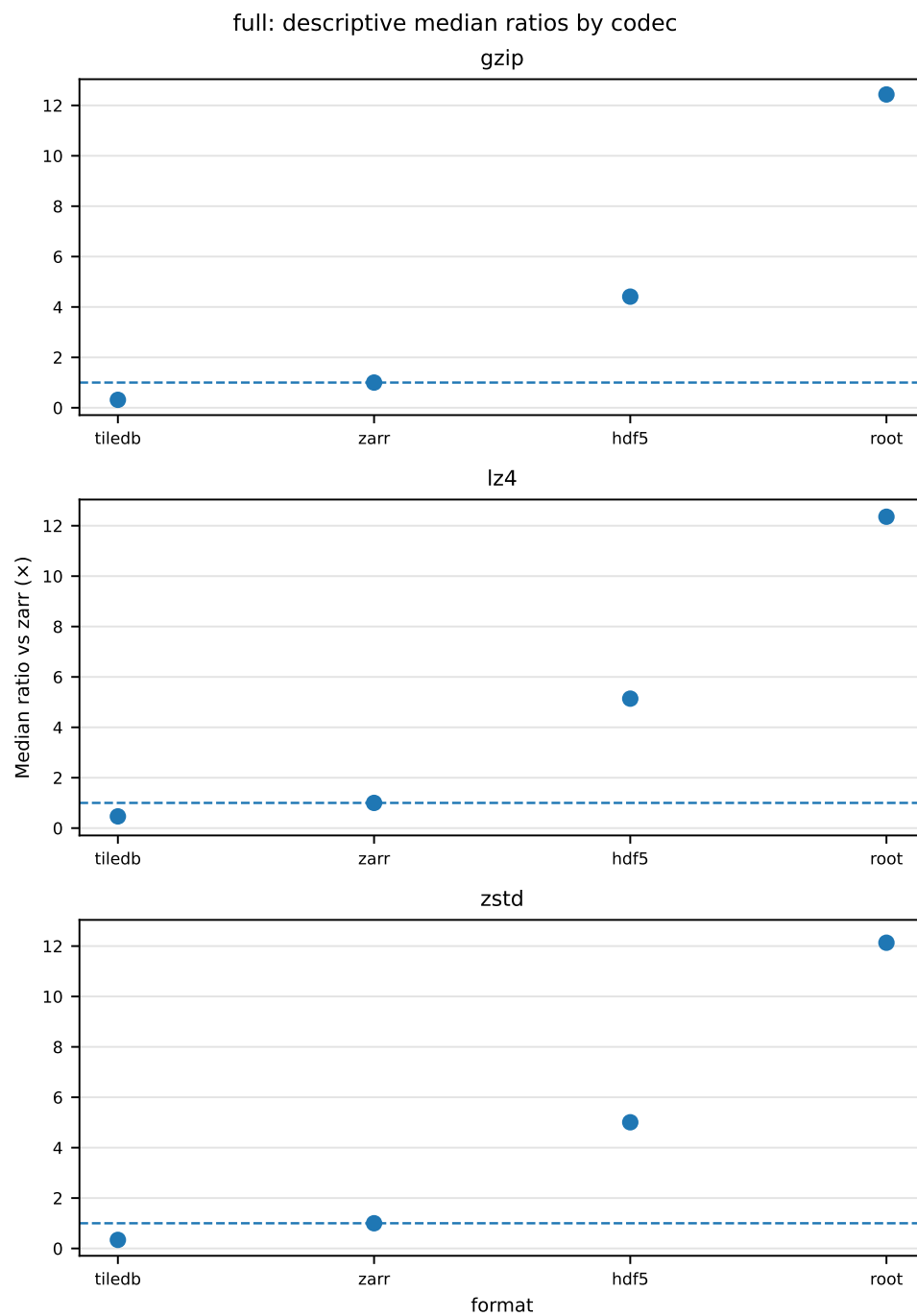


Figure A.21: Full scan: descriptive median latency ratios vs Zarr across codecs (gzip, LZ4, ZSTD).

A.2.5 Tail sensitivity

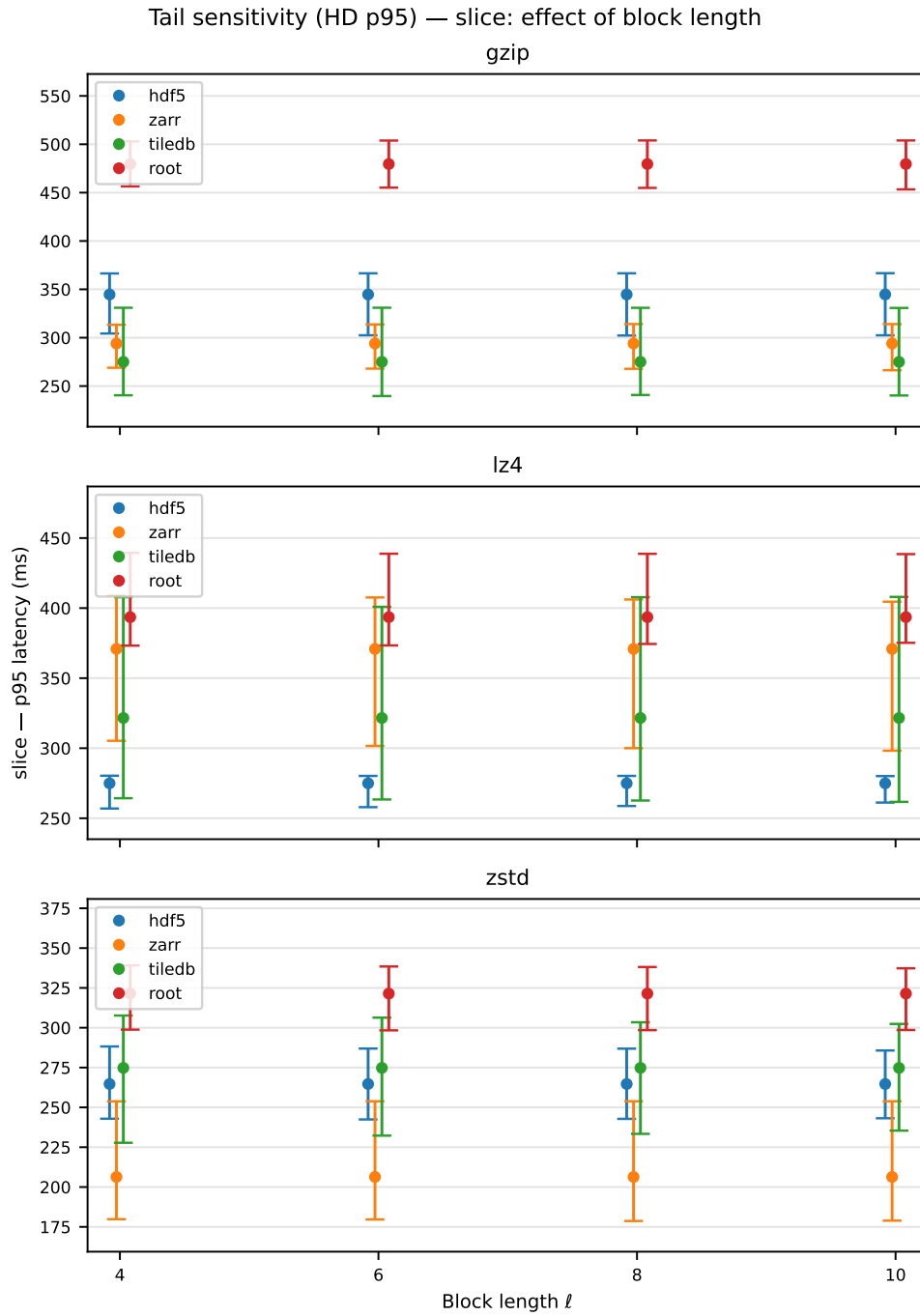


Figure A.22: Slice: **HD** p_{95} latency with **MBB** 95% CIs versus block length ℓ (columns: gzip, **LZ4**, **ZSTD**; legend shows formats).

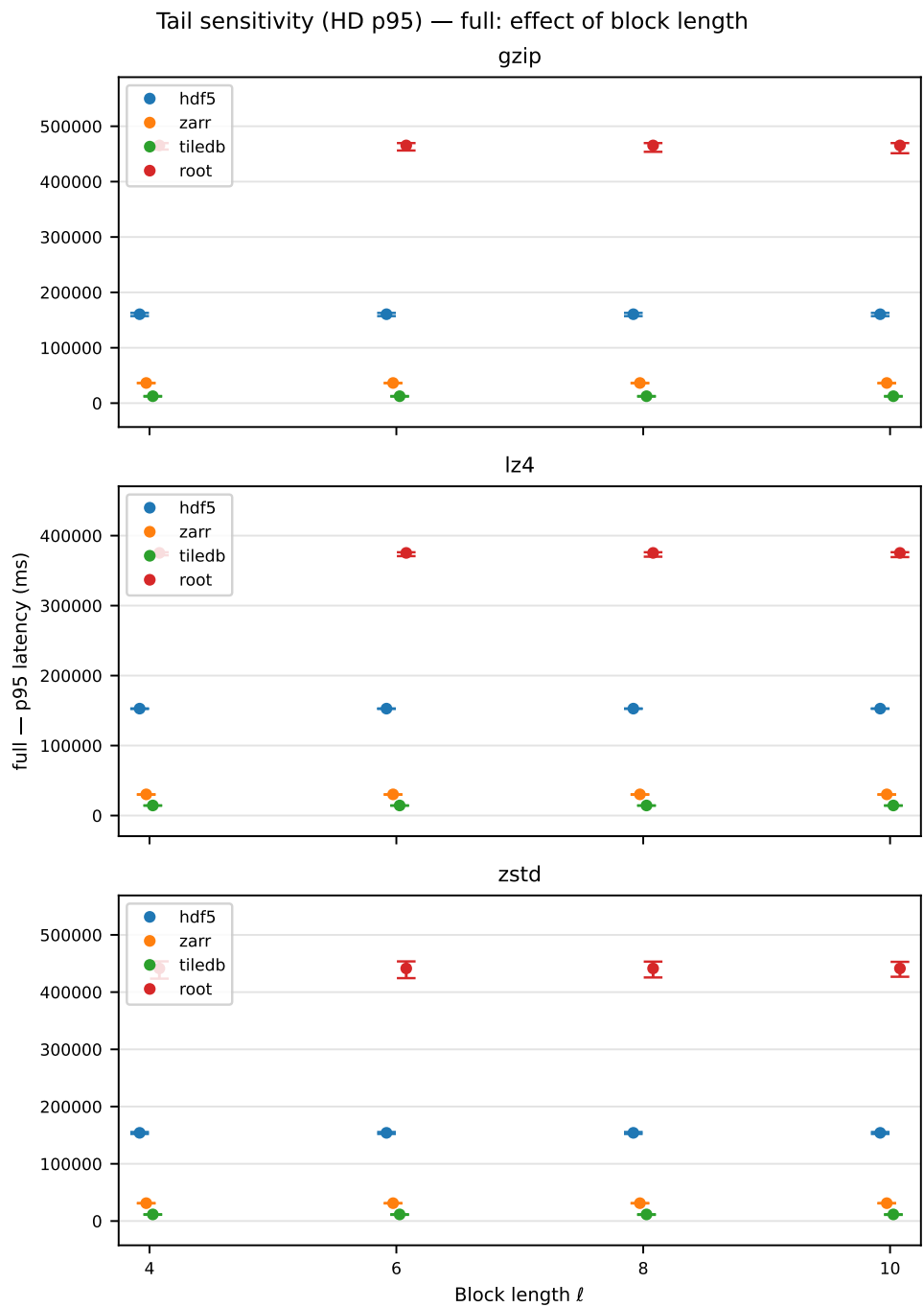


Figure A.23: Full scan: HD p_{95} latency with MBB 95% CIs versus block length ℓ (columns: gzip, LZ4, ZSTD; legend shows formats).

A.2.6 League tables for configuration selection

The main text reports representative league tables for slice `gzip` and **ZSTD** and for full scans under **ZSTD**. This appendix lists the remaining league tables used for the decision matrix.

Slice.

Table A.6: Slice — `lz4`: league table.

Rank	Format	GM (ms) [CI]	p_{95} (ms) [CI]	GETs/GiB	Stored GiB
1	<code>zarr</code>	176 [159, 195]	349 [281, 414]	135.0	10.40
2	<code>tiledb</code>	214 [203, 227]	274 [261, 403]	83.2	15.56
3	HDF5/HSDS	222 [215, 230]	273 [251, 281]	270.5	19.17
4	ROOT/TTree	328 [314, 344]	380 [369, 429]	51.1	13.31

Full scan.

Table A.7: Full — `gzip`: league table.

Rank	Format	GM thr (GiB/s) [CI]	GM time (ms) [CI]	GETs/GiB	Stored GiB
1	<code>tiledb</code>	1.037 ([1.013, 1.062])	11 254 ([10 994, 11 529])	94.6	11.44
2	<code>zarr</code>	0.318 ([0.315, 0.321])	35 598 ([35 280, 35 926])	116.8	11.09
3	HDF5/HSDS	0.072 ([0.072, 0.072])	156 966 ([156 394, 157 761])	137.7	11.11
4	ROOT/TTree	0.051 ([0.050, 0.052])	442 484 ([434 012, 451 398])	58.5	11.09

Table A.8: Full — 1 z 4: league table.

Rank	Format	GM thr (GiB/s) [CI]	GM time (ms) [CI]	GETs/GiB	Stored GiB
1	tiledb	1.167 ([1.144, 1.197])	13 602 ([13 265, 13 871])	93.7	15.56
2	zarr	0.368 ([0.365, 0.371])	29 483 ([29 255, 29 741])	124.5	10.40
3	HDF5/HSDS	0.129 ([0.129, 0.129])	151 427 ([151 201, 151 666])	79.6	19.17
4	ROOT/TTree	0.075 ([0.075, 0.076])	363 490 ([358 672, 367 871])	48.7	13.31

A.2.7 Score matrices for configuration selection

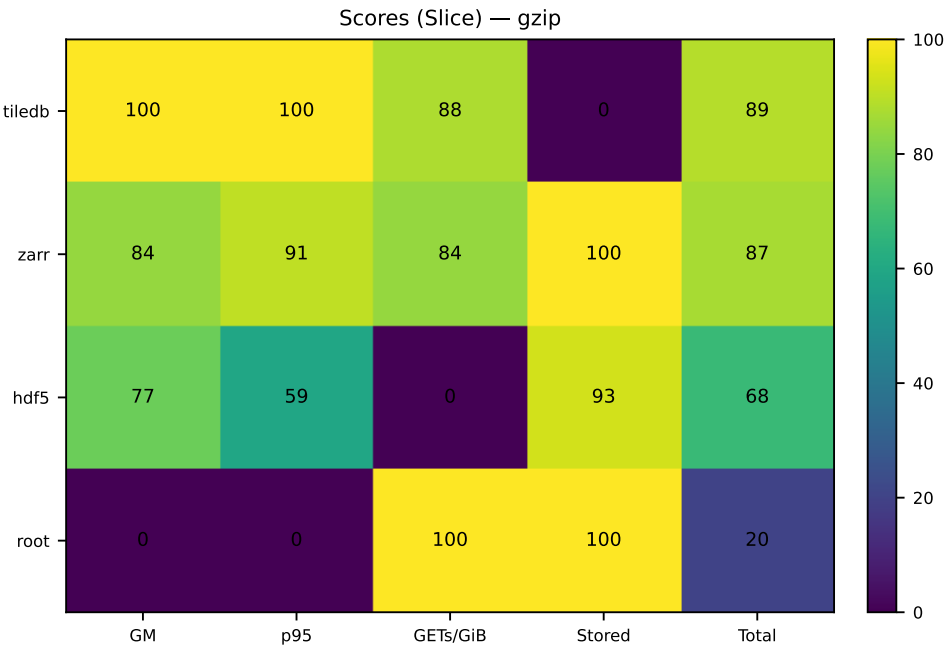


Figure A.24: Slice/gzip: normalized scores (higher is better); column “Total” uses the stated weights.

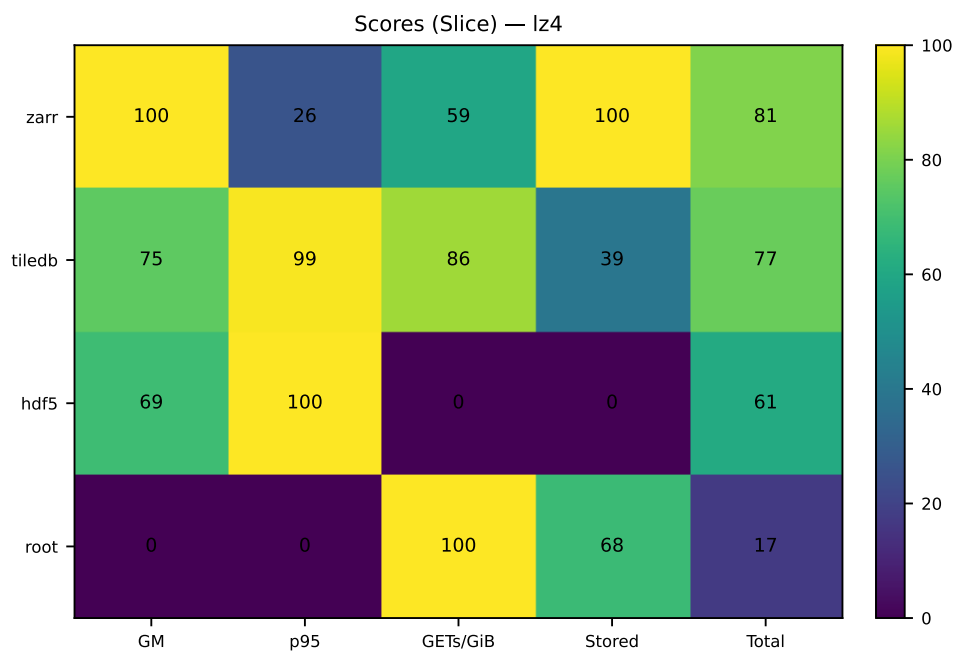


Figure A.25: Slice/lz4: normalized scores.

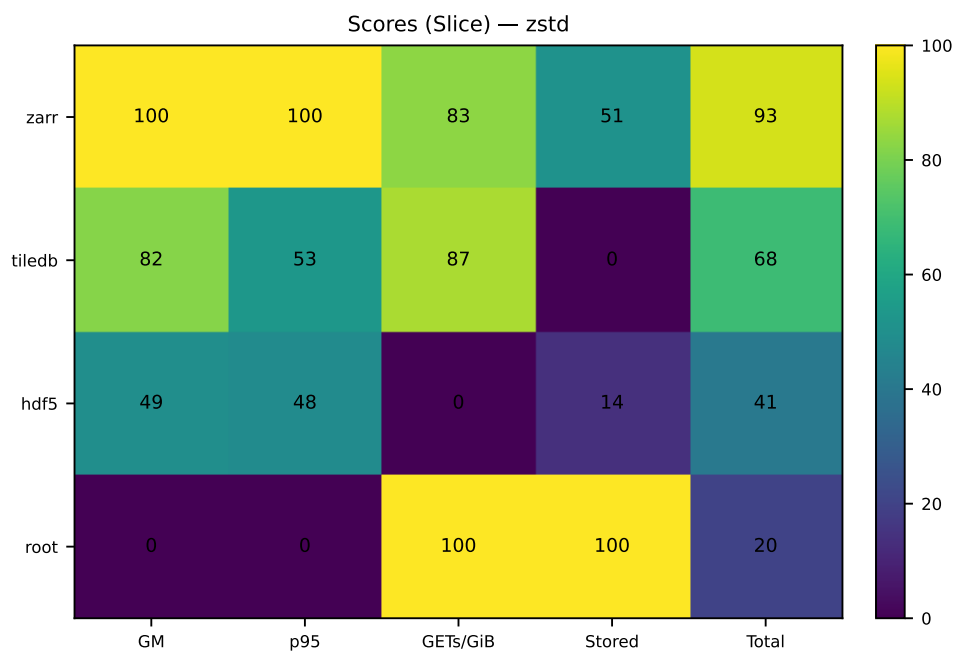


Figure A.26: Slice/zstd: normalized scores.

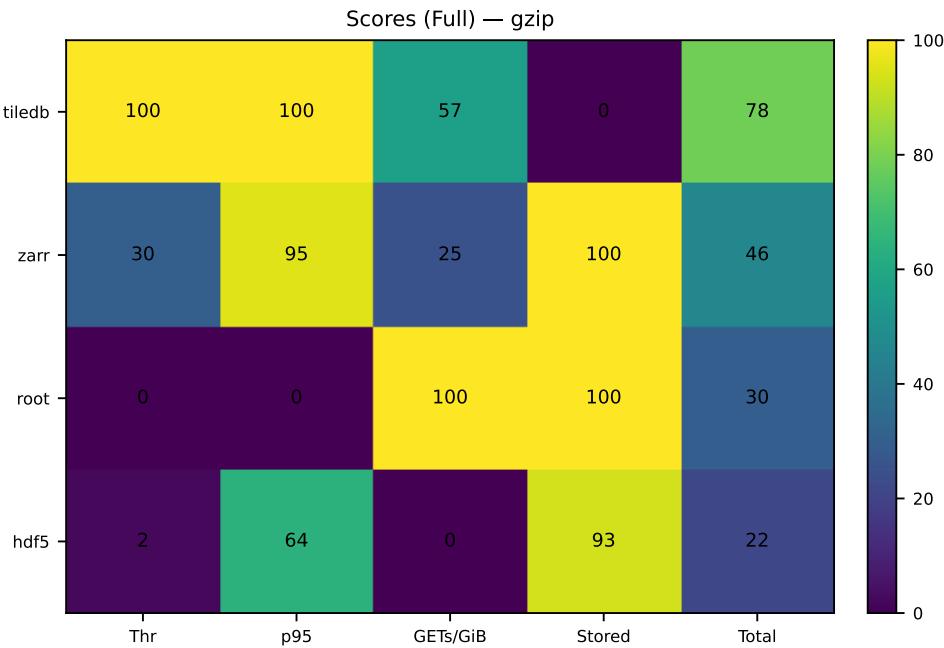


Figure A.27: Full/gzip: normalized scores.

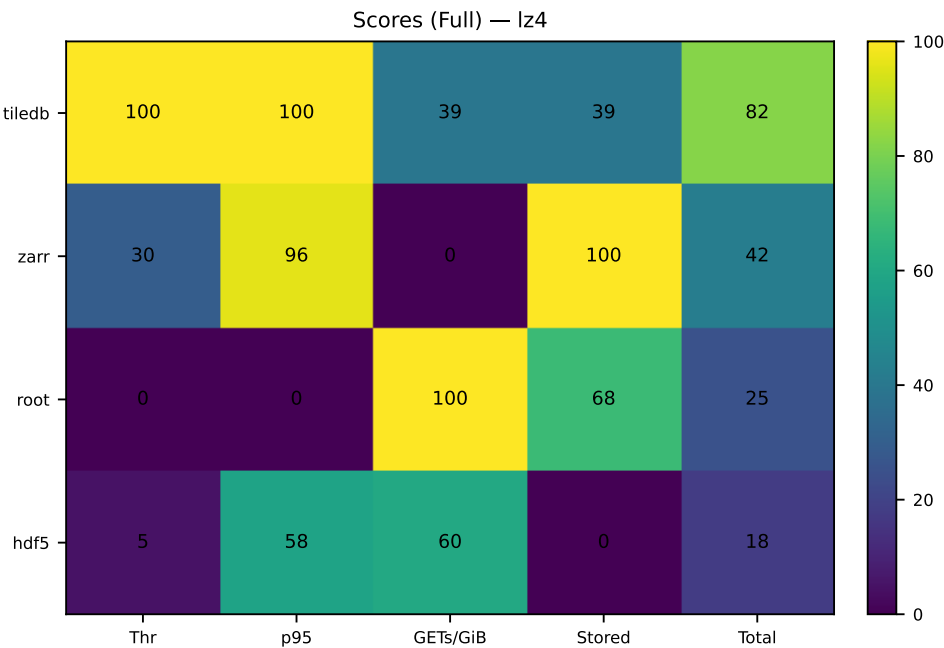


Figure A.28: Full/lz4: normalized scores.

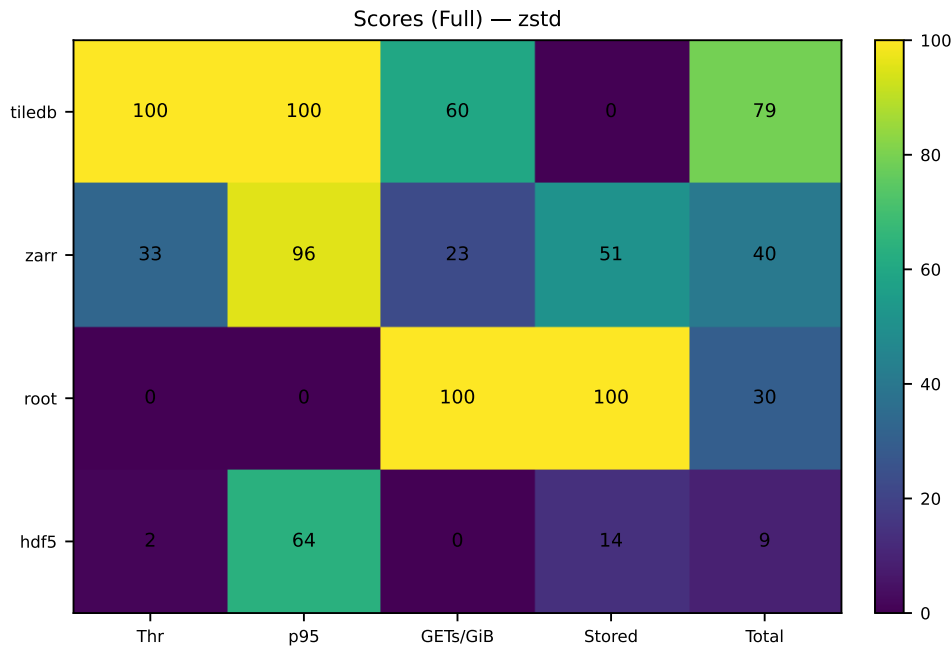


Figure A.29: Full/zstd: normalized scores.

We aggregate latency, tail behavior, request intensity, and stored size into 0–100 normalized scores per codec×pattern to support configuration choices. Figures A.24–A.29 and Tables A.9–A.14 show the resulting score matrices. For slices, TileDB and Zarr dominate the Pareto frontier for gzip and ZSTD, while Zarr is favored under LZ4. For full scans, TileDB clearly dominates on throughput and overall score for all codecs.

Slice. Scores are 0–100 per metric using robust min–max (5th–95th percentiles) within each codec×pattern where higher is better. Weights: GM 0.60, p_{95} 0.20, GETs/GiB 0.10, Stored 0.10.

Table A.9: Slice/gzip: score matrix (normalized 0–100).

Format	GM	p_{95}	GETs	Stored	Total	DoM. /by
tiledb	100.0	100.0	87.7	0.0	88.8	/
zarr	84.1	90.6	84.0	100.0	86.9	HDF5/HSDS /
HDF5/HSDS	77.5	59.0	0.0	93.3	67.6	/ zarr
ROOT/TTree	0.0	0.0	100.0	100.0	20.0	/

Table A.10: Slice/lz4: score matrix (normalized 0–100).

Format	GM	p_{95}	GETs	Stored	Total	DoM. /by
zarr	100.0	25.9	59.3	100.0	81.1	/
tiledb	75.0	99.0	85.9	39.4	77.3	/
HDF5/HSDS	68.8	100.0	0.0	0.0	61.3	/
ROOT/TTree	0.0	0.0	100.0	68.3	16.8	/

Table A.11: Slice/zstd: score matrix (normalized 0–100).

Format	GM	p_{95}	GETs	Stored	Total	DoM. /by
zarr	100.0	100.0	83.2	50.8	93.4	HDF5/HSDS /
tiledb	81.9	53.1	87.2	0.0	68.4	/
HDF5/HSDS	49.3	48.1	0.0	13.8	40.6	/ zarr
ROOT/TTree	0.0	0.0	100.0	100.0	20.0	/

Full scan. Scores are 0–100 per metric, using robust min–max normalization within codec×pattern where higher scores are better. Weights: throughput 0.60, GETs/GiB 0.15, Stored 0.15, p_{95} 0.10.

Table A.12: Full/gzip: score matrix (normalized 0–100).

Format	Thr	p_{95}	GETs/GiB	Stored	Total	DoM. /by
tiledb	100.0	100.0	56.6	0.0	78.5	/
zarr	30.1	94.9	25.2	100.0	46.3	HDF5/HSDS /
ROOT/TTree	0.0	0.0	100.0	100.0	30.0	/
HDF5/HSDS	2.1	64.3	0.0	93.3	21.7	/ zarr

Table A.13: Full/lz4: score matrix (normalized 0–100).

Format	Thr	p_{95}	GETs/GiB	Stored	Total	DoM. /by
tiledb	100.0	100.0	39.4	39.4	81.8	/
zarr	29.6	95.8	0.0	100.0	42.3	/
ROOT/TTree	0.0	0.0	100.0	68.3	25.2	/
HDF5/HSDS	4.8	58.1	60.5	0.0	17.8	/

Table A.14: Full/zstd: score matrix (normalized 0–100).

Format	Thr	p_{95}	GETs/GiB	Stored	Total	DoM. /by
tiledb	100.0	100.0	59.7	0.0	79.0	/
zarr	33.1	95.6	22.7	50.8	40.4	HDF5/HSDS /
ROOT/TTree	0.0	0.0	100.0	100.0	30.0	/
HDF5/HSDS	1.5	63.5	0.0	13.8	9.3	/ zarr

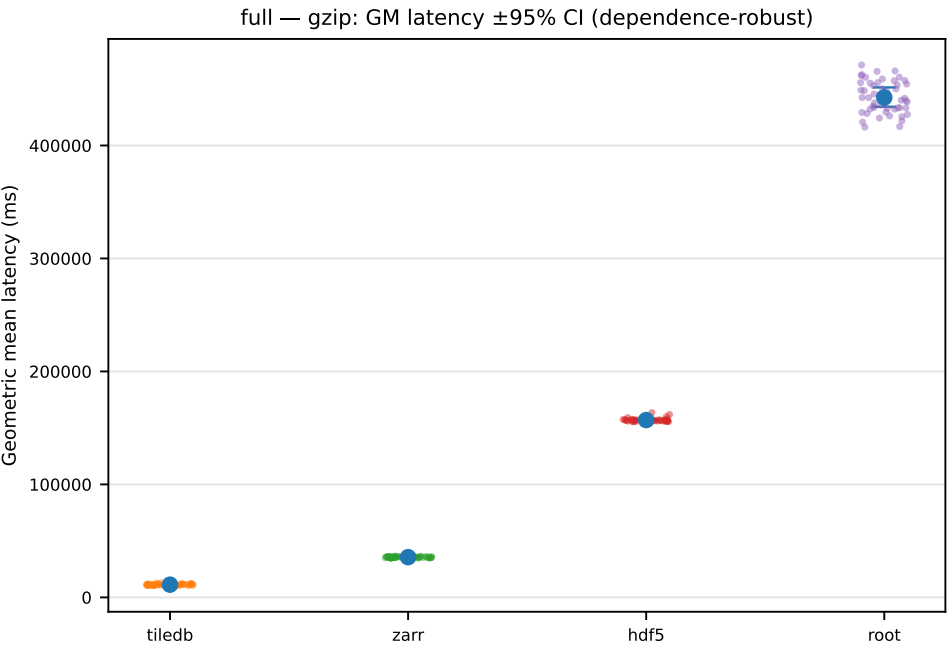


Figure A.30: gzip/full: geometric-mean time (ms) with dependence-robust 95% CIs.

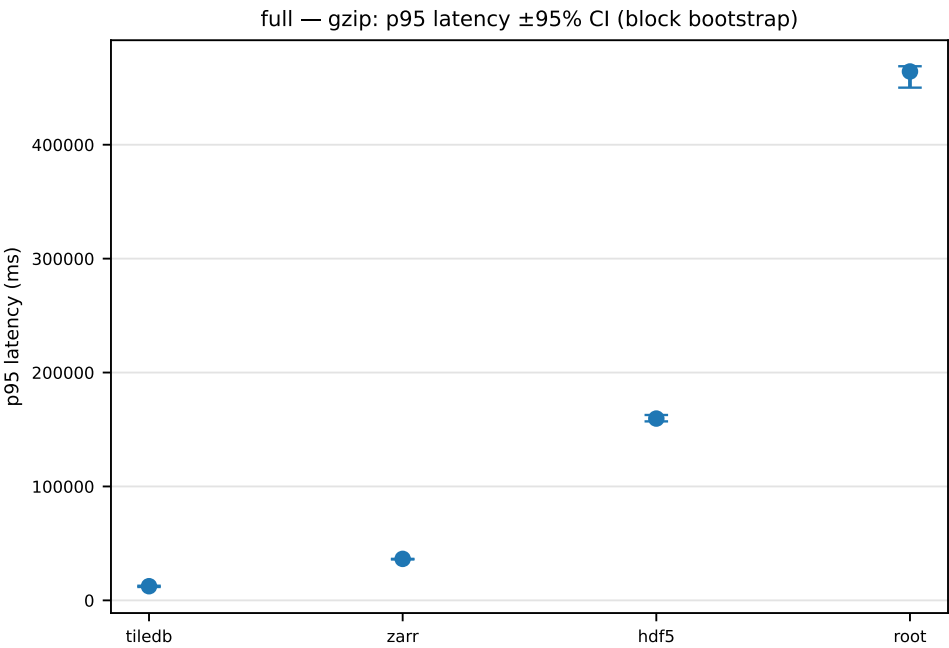


Figure A.31: `gzip/full`: p_{95} time with dependence-robust 95% CIs.

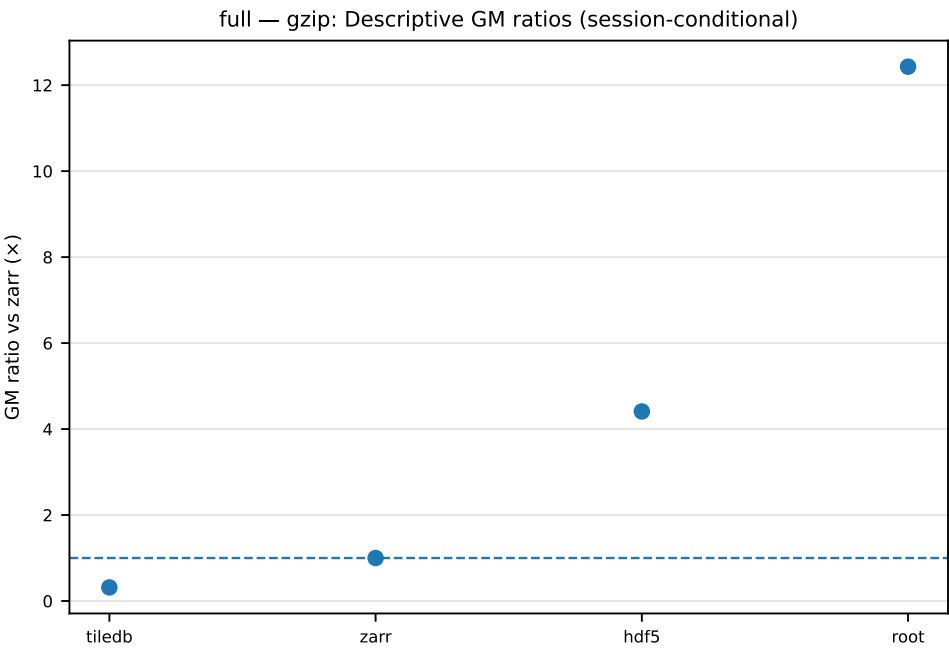


Figure A.32: `gzip/full`: descriptive **GM** time ratios vs Zarr (session-conditional).

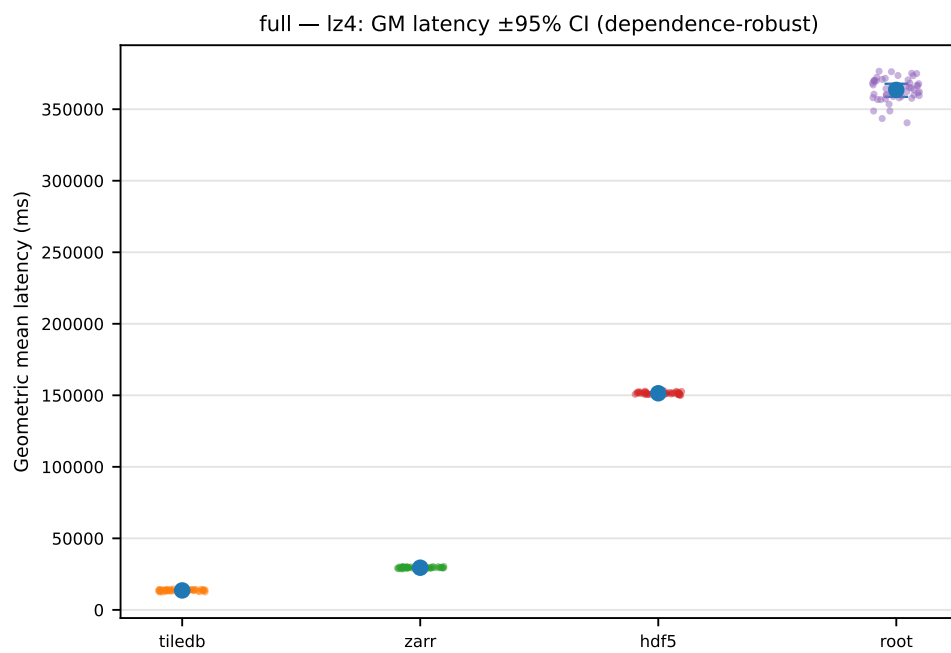


Figure A.33: $1z4/full$: geometric-mean time (ms) with dependence-robust 95% CIs.

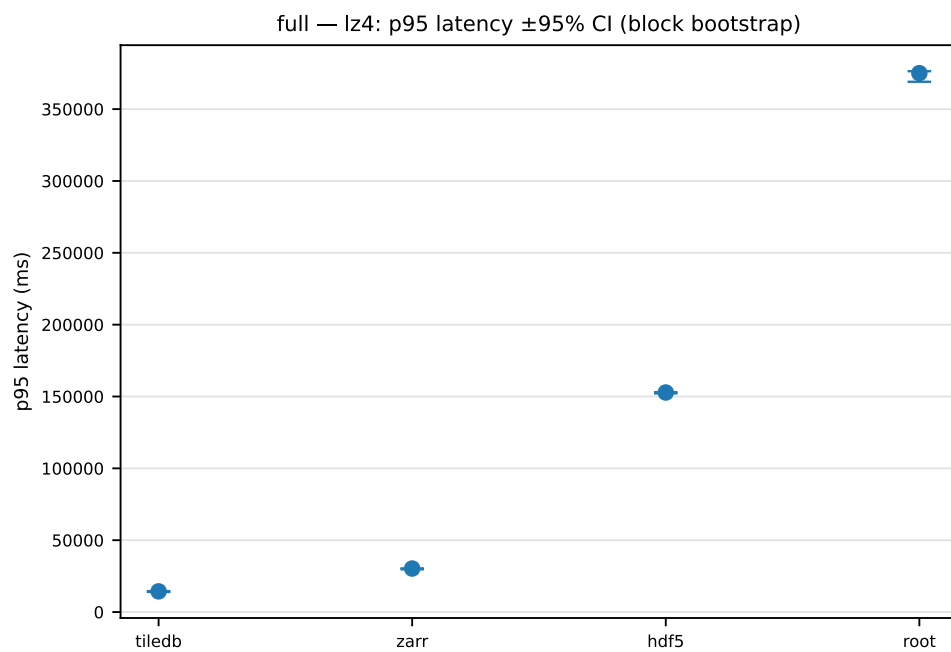


Figure A.34: $1z4/full$: p_{95} time with dependence-robust 95% CIs.

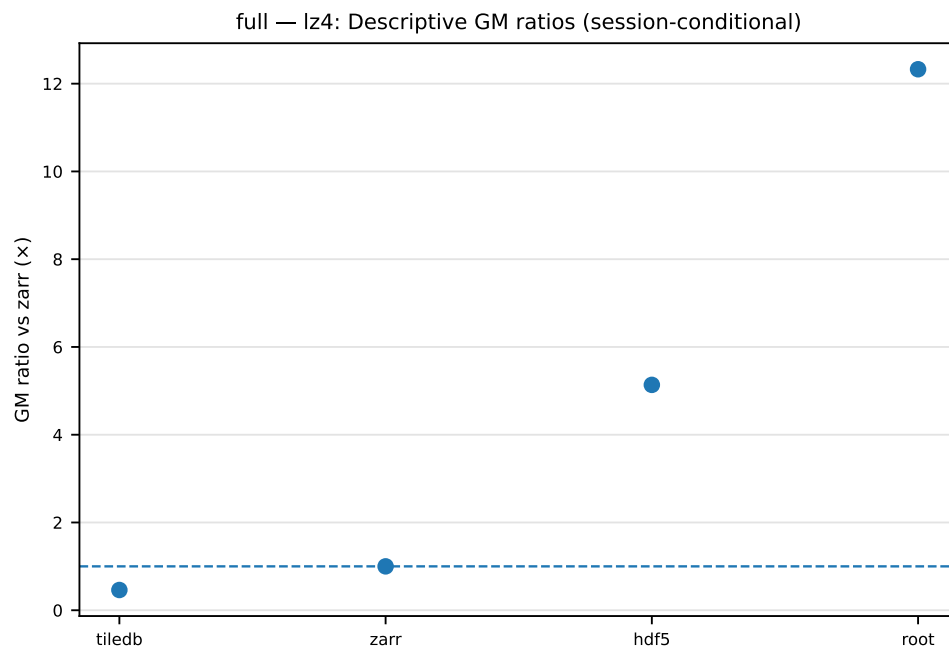


Figure A.35: lz4/full: descriptive **GM** time ratios vs Zarr (session-conditional).

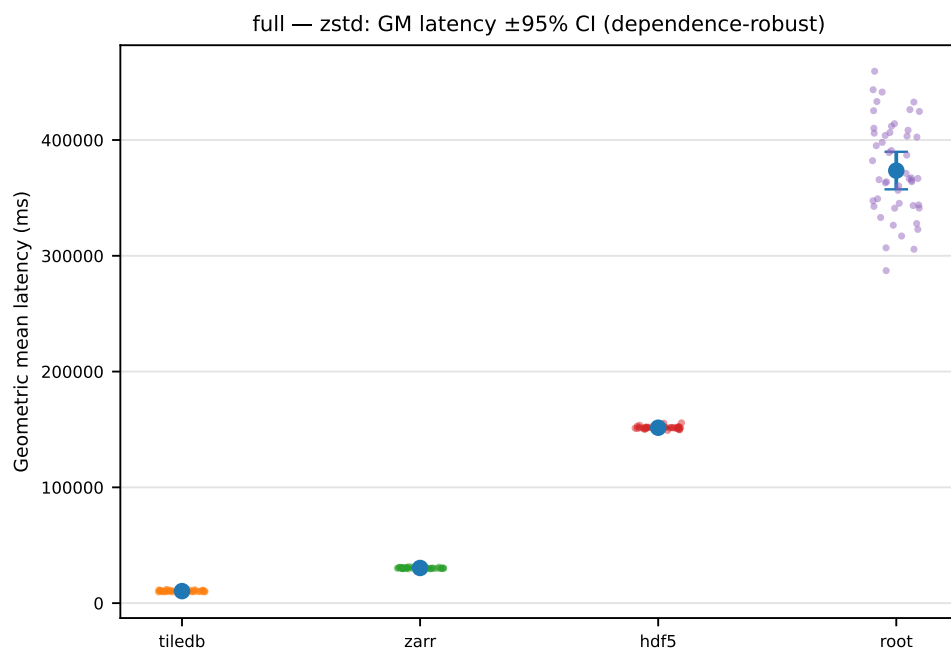


Figure A.36: `zstd/full`: geometric-mean time (ms) with dependence-robust 95% CIs.

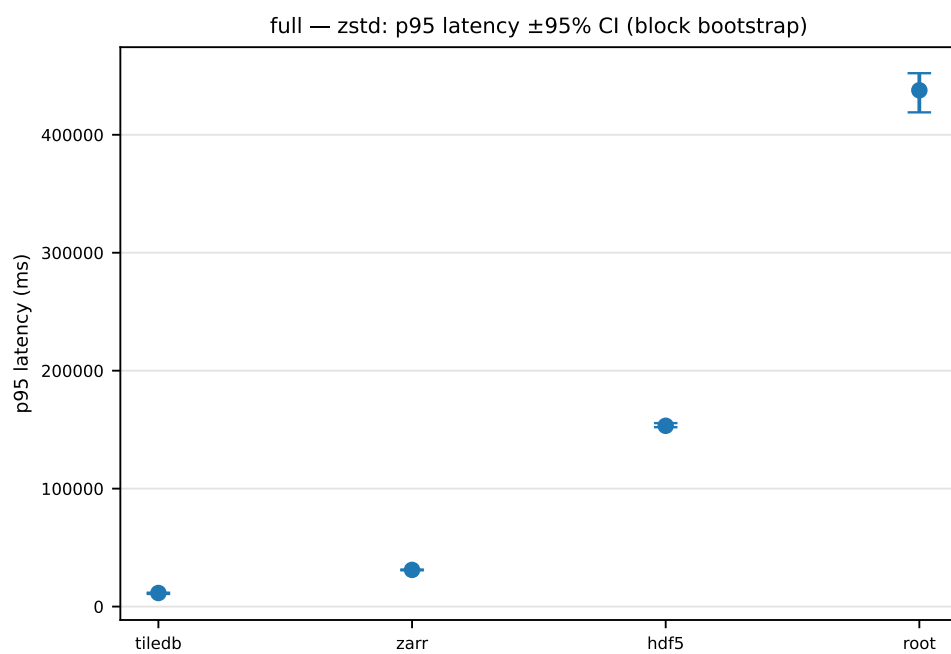


Figure A.37: `zstd/full`: p_{95} time with dependence-robust 95% CIs.

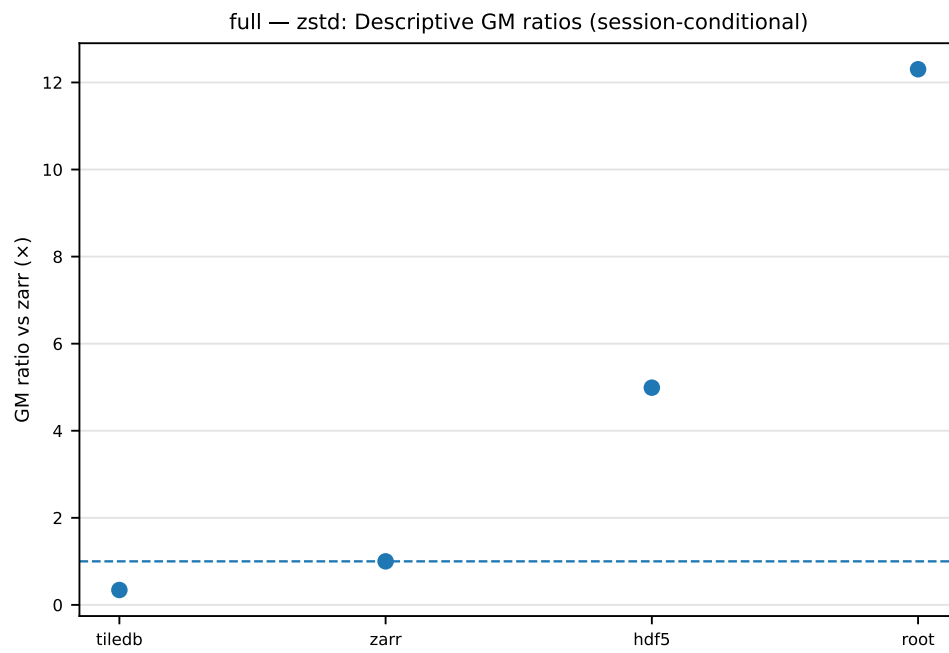


Figure A.38: `zstd/full`: descriptive **GM** time ratios vs Zarr (session-conditional).

€€€€ For DIVA €€€€

```
{
  "Author1": { "Last name": "Kristiansson",
    "First name": "Casper Ove",
    "Local User Id": "casperkr",
    "E-mail": "casperkr@kth.se",
    "organisation": { "L1": "School of Electrical Engineering and Computer Science",
    }
  },
  "Cycle": "2",
  "Course code": "DA231X",
  "Credits": "30.0",
  "Degree1": { ("Educational program": "Master's Programme, Computer Science, 120 credits"
    , "programcode": "TCSCM"
    , "Degree": "Degree of Master of Science in Engineering"
    , "subjectArea": "Technology"
  )
  },
  "Title": {
    "Main title": "From Experiment to Insight: A Comparative Study of Storage Approaches for Large-Scale Synchrotron and Neutron Scattering Data on AWS",
    "Subtitle": "Benchmarking HDF5/HSDS, Zarr, TileDB, and ROOT/TTree for Random Single-Frame Reads and Full Sequential Scans",
    "Language": "eng" },
    "Alternative title": {
      "Main title": "Från experiment till insikt: en jämförande studie av lagringsmetoder för storskaliga synkrotron- och neutrondata på AWS",
      "Subtitle": "Prestandautvärdering av HDF5/HSDS, Zarr, TileDB och ROOT/TTree för slumpmässiga läsningar av enskilda bilder och sekventiell genomläsning av hela dataset",
      "Language": "swe"
    },
    "Supervisor1": { "Last name": "Dalleiger",
      "First name": "Sebastian",
      "Local User Id": "u100003",
      "E-mail": "sdall@kth.se",
      "organisation": { "L1": "",
        "L2": "Theoretical Computer Science" }
    },
    "Supervisor2": { "Last name": "Yildiz",
      "First name": "Ahmet Bahadir",
      "E-mail": "sb@kth.se",
    },
    "Examiner1": { "Last name": "Papadimitratos",
      "First name": "Panagiotis",
      "Local User Id": "u100004",
      "E-mail": "papadim@kth.se",
      "organisation": { "L1": "School of Electrical Engineering and Computer Science",
        "L2": "Software & Computer Systems" }
    },
    "Cooperation": { "Partner_name": "Scatterin AB",
      "National Subject Categories": "10201, 10206",
      "Other information": { "Year": "2026", "Number of pages": "1,169",
        "Copyright": "copyright",
        "Series": { "Title of series": "TRITA – EECS-EX" , "No. in series": "2024:0000" },
        "Opponents": { "Name": "Axel Sparr",
          "Presentation": { "Date": "2026-05-26 10:00"
            , "Language": "eng"
            , "Room": "via Zoom https://kth-se.zoom.us/j/63369824065"
            , "City": "Stockholm" } },
          "Number of lang instances": "2",
          "Abstract[eng ]": €€€€
        }
      }
    }
  }
  Modern synchrotron and neutron beamlines can stream at gigabytes per second, turning single experiments into multi-terabyte datasets and exposing latency limits of parallel file systems. This thesis benchmarks cloud-native storage layouts on AWS S3 using a single synthetic ~20 GiB, ~1,300-frame detector stack generated from 80 real CBF seed frames, focusing on slice-level latency, scan throughput, and cost drivers. Four backends (HDF5 via HSDS, Zarr v3, TileDB, ROOT/TTree) and three codecs (gzip, LZ4, zstd) are evaluated under two workloads: random single-frame reads and full sequential scans. Client timings are analyzed on a log scale with dependence-robust confidence intervals, with S3 request counts and stored size providing context.
  Results are descriptive and session-conditional for this dataset and AWS setup. For interactive slices, Zarr is fastest with zstd (GM ≈155 ms) and with LZ4; TileDB is fastest with gzip. For full scans, TileDB completes the same logical dataset in 2.2–3.2× lower GM scan time than Zarr across codecs; the transferred-byte throughput summaries follow the same ordering (e.g., zstd ≈1.06 GiB/s for 20 GiB). ROOT/TTree is slow for frame-random reads despite a few GETs, indicating single-object paging and decode overheads. HDF5/HSDS trails on slice latency and scan throughput with LZ4, storing 19.17 GiB versus Zarr 10.40 GiB for the same payload.
  The contribution is a reproducible AWS-native benchmarking method and a decision matrix for the tested synthetic detector stack and workloads: choose Zarr+zstd when slice latency dominates, and choose TileDB+zstd/LZ4 when throughput dominates. €€€€,
  "Keywords[eng ]": €€€€
  Synchrotron; Neutron scattering; Object storage; AWS S3; EC2; CloudWatch; Cloud-native; HDF5; HSDS; Zarr; TileDB; ROOT; TTree; Parquet; Partial retrieval; Random reads; Full scans; Chunking; Fragments; Metadata; Compression; zstd; lz4; gzip; Latency; Throughput; Request cost; Decision matrix; Scientific data management; Benchmarking; €€€€,
  "Abstract[swe ]": €€€€
  Vid moderna synkrotron- och neutronanläggningar genereras dataflöden på flera gigabyte per sekund. Enskilda experiment kan därmed leda till
```

datamängder på flera terabyte, vilket tydliggör latensbegränsningar i parallella filsystem. I denna avhandling jämförs molnativa lagringsformat på AWS S3 med en syntetiskt utökad datamängd om cirka 20 GiB och cirka 1 300 bilder, genererad från 80 verkliga CBF-bilder, som testfall. Fokus ligger på latens vid läsning av enskilda bilder, datahastighet vid sekventiell genomläsning av hela datamängden samt kostnadsdrivande faktorer. Fyra lagringslösningar (HDF5 via HSDS, Zarr v3, TileDB och ROOT/TTree) och tre kodekar (gzip, LZ4 och zstd) utvärderas för två åtkomstmönster: slumpmässig läsning av enskilda bilder och sekventiell genomläsning av hela datamängden. Körtider på klientsidan analyseras på logaritmisk skala med beroenderobusta konfidensintervall, och antalet S3-förfrågningar samt lagrad datamängd används som kompletterande mått. Resultaten är deskriptiva och gäller den studerade datamängden i den aktuella AWS-miljön; de bör därför tolkas som sessionsbundna. För interaktiv läsning av enskilda bilder är Zarr snabbast med zstd (GM $\approx$ 155 ms) och även med LZ4, medan TileDB är snabbast med gzip. Vid sekventiell genomläsning av hela datamängden läser TileDB igenom samma logiska datamängd på $2,2\text{--}3,2\times$ lägre GM-skanningstid än Zarr för samtliga kodekar; sammanfattningarna av överförd datahastighet följer samma ordning (exempelvis zstd $\approx$ 1,06 GiB/s för 20 GiB). ROOT är långsamt vid slumpmässig läsning av bilder trots få GET-förfrågningar, vilket tyder på overhead från paginering och avkodning i en lagringslayout med ett enda objekt. HDF5/HSDS presterar sämre både vad gäller latens vid bildläsning och datahastighet vid helavläsning med LZ4 och lagrar 19,17 GiB jämfört med 10,40 GiB för Zarr för samma nyttolast. Avhandlingens huvudsakliga bidrag är en reproducerbar metod för prestandamätning på AWS samt en beslutsmatris för den studerade syntetiska datamängden och de studerade användningsfallen: välj Zarr+zstd när latens vid läsning av enskilda bilder är viktigast, och välj TileDB+zstd/LZ4 när datahastighet prioriteras. €€€€, "Keywords[swe]": €€€€ Synkrotron; Neutronspredning; Objektlagring; AWS S3; EC2; CloudWatch; Molnativa lagringsformat; HDF5; HSDS; Zarr; TileDB; ROOT; TTree; Parquet; Partiell dataåtkomst; Slumpmässiga läsningar; Sekventiella genomläsningar; Chunkning; Fragment; Metadata; Komprimering; zstd; lz4; gzip; Latens; Genomströmning; Förfrågningskostnader; Beslutsmatris; Vetenskaplig datahantering; Prestandajämförelse; €€€€, }

acronyms.tex

```
%%% Local Variables:
%%% mode: latex
%%% TeX-master: t
%%% End:
% The following command is used with glossaries-extra
\setabbreviationstyle[acronym]{long-short}

\newacronym{HPC}{HPC}{High-Performance Computing}
\newacronym{AWS}{AWS}{Amazon Web Services}
\newacronym{SThree}{S3}{Simple Storage Service}
\newacronym{HDFfive}{HDF5}{Hierarchical Data Format version~5}
\newacronym{GPFS}{GPFS}{General Parallel File System}
\newacronym{MPIIO}{MPI-IO}{Message Passing Interface Input/Output}
\newacronym{POSIX}{POSIX}{Portable Operating System Interface}
\newacronym{CPU}{CPU}{Central Processing Unit}
\newacronym{ROOT}{ROOT}{ROOT data analysis framework}
\newacronym{RNTuple}{RNTuple}{ROOT Ntuple}
\newacronym{IPC}{IPC}{Inter-Process Communication}
\newacronym{HSDS}{HSDS}{Highly Scalable Data Service}
\newacronym{ECtwo}{EC2}{Elastic Compute Cloud}
\newacronym{EBS}{EBS}{Elastic Block Store}
\newacronym{ANOVA}{ANOVA}{analysis of variance}
\newacronym{HSD}{HSD}{Honestly Significant Difference}
\newacronym{ORC}{ORC}{Optimized Row Columnar}
\newacronym{CBF}{CBF}{Crystallographic Binary File}
\newacronym{TCO}{TCO}{Total Cost of Ownership}
\newacronym{DAQ}{DAQ}{Data Acquisition}
\newacronym{NVMe}{NVMe}{Non-Volatile Memory Express}
\newacronym{SSD}{SSD}{Solid-State Drive}
\newacronym{CMOS}{CMOS}{Complementary Metal--Oxide--Semiconductor}
\newacronym{sCMOS}{sCMOS}{scientific Complementary Metal--Oxide--Semiconductor}
\newacronym{NIST}{NIST}{National Institute of Standards and Technology}
\newacronym{NeXT}{NeXT}{Neutron and X-ray Tomography}
\newacronym{SLS}{SLS}{Swiss Light Source}
\newacronym{BeeGFS}{BeeGFS}{Bee Global File System}
\newacronym{TOMCAT}{TOMCAT}{Tomographic Microscopy and Coherent Radiology Experiments}
\newacronym{SAXS}{SAXS}{Small-angle X-ray Scattering}
\newacronym{WAXS}{WAXS}{Wide-angle X-ray Scattering}
\newacronym{EXAFS}{EXAFS}{Extended X-ray Absorption Fine Structure}
\newacronym{MDS}{MDS}{Metadata Server}
\newacronym{OSS}{OSS}{Object Storage Server}
\newacronym{NSD}{NSD}{Network Shared Disk}
\newacronym{SAN}{SAN}{Storage Area Network}
\newacronym{DNE}{DNE}{Distributed Namespace Environment}
\newacronym{OST}{OST}{Object Storage Target}
\newacronym{WAN}{WAN}{Wide Area Network}
\newacronym{PFS}{PFS}{Parallel File System}
\newacronym{DoM}{DoM}{Data on Metadata}
\newacronym{EFS}{EFS}{Elastic File System}
\newacronym{NREL}{NREL}{National Renewable Energy Laboratory}
\newacronym{GDAL}{GDAL}{Geospatial Data Abstraction Library}
\newacronym{OME}{OME}{Open Microscopy Environment}
\newacronym{JSON}{JSON}{JavaScript Object Notation}
\newacronym{ROMIO}{ROMIO}{MPI-IO implementation}
\newacronym{CSV}{CSV}{Comma-Separated Values}
\newacronym{ZSTD}{ZSTD}{Zstandard}
\newacronym{CERN}{CERN}{European Organization for Nuclear Research}
\newacronym{TTree}{TTree}{ROOT Tree data structure}
\newacronym{HTTP}{HTTP}{Hypertext Transfer Protocol}
\newacronym{DAOS}{DAOS}{Distributed Asynchronous Object Storage}
\newacronym{netCDF}{netCDF}{Network Common Data Form}
\newacronym{RFC}{RFC}{Request for Comments}
\newacronym{LZseventyseven}{LZ77}{Lempel--Ziv 1977}
\newacronym{LZfour}{LZ4}{Lempel--Ziv 4}
\newacronym{ZLIB}{zlib}{zlib compression library}
\newacronym{FSE}{FSE}{Finite State Entropy}
\newacronym{CFOSAT}{CFOSAT}{China--France Oceanography Satellite}
\newacronym{SWOT}{SWOT}{Surface Water and Ocean Topography}
\newacronym{LHCb}{LHCb}{Large Hadron Collider beauty}
\newacronym{VLDB}{VLDB}{Very Large Data Bases}
\newacronym{FCBench}{FCBench}{Floating-Point Compression Benchmark}
\newacronym{NLSII}{NLS-II}{National Synchrotron Light Source II}
\newacronym{ECHO}{ECHO}{ECHO object store}
\newacronym{IRIS}{IRIS}{IRIS cloud analysis service}
\newacronym{API}{API}{Application Programming Interface}
\newacronym{Ceph}{Ceph}{Ceph distributed storage system}
```

```
\newacronym{MinIO}{MinIO}{MinIO object storage}
\newacronym{DSR}{DSR}{Design Science Research}
\newacronym{ART}{ART}{Aligned Rank Transform}
\newacronym{CUR}{CUR}{Cost and Usage Report}
\newacronym{TCP}{TCP}{Transmission Control Protocol}
\newacronym{Pninetyfive}{P95}{95\textsuperscript{th} percentile}
\newacronym{CI}{CI}{Confidence Interval}
\newacronym{GM}{GM}{geometric mean}
\newacronym{MBB}{MBB}{moving-block bootstrap}
\newacronym{HD}{HD}{Harrell--Davis}
```